Chanakya

An offline AI teacher: narrated, animated lessons in nine languages

Team name	The Last Commit
Members	Aryan Dubey, Priyadarshini Prasad, Niharika K P, Dhiraj Shetty
College	RV University, Bengaluru
Course	B.Tech (Hons)
Branch	Computer Science Engineering
Year of study	2nd year

Complete project documentation · v0.16.0
https://github.com/signingoff-dubey/UNLOX-TEACHER

Contents

Overview	
README: overview, features, models, languages, tech stack, API	`README.md`
Product	
Product requirements (PRD)	`prd.md`
Problem statement	`problem_statement.md`
Pitch	`pitch.md`
Plan	`PLAN.md`
To do	`todo.md`
Engineering log	
Progress log and git history	`progress.md`
Issues and fixes log	`issues.md`
Design notes	
Orchestration	`docs/orchestration.md`
Long videos in parts	`docs/plan-long-videos.md`
Presentation mode	`docs/presentation-mode.md`
Voice assistant	`docs/voice-assistant.md`
RAG and OCR: design	`docs/superpowers/specs/2026-10-27-rag-ocr-design.md`
RAG and OCR: implementation plan	`docs/superpowers/plans/2026-10-27-rag-ocr.md`
Earlier detailed notes and per-version changelog	`docs/README-notes.md`
Teammate modules	
Blender 3D lessons (Dhiraj)	`blender 3d/README.md`
Learnverse metaverse (Niharika)	`metaverse/README.md`
Niharika: voice assistant and metaverse notes	`Niharika/README.md`
Niharika 01: current state	`Niharika/01-current-state.md`
Niharika 02: architecture	`Niharika/02-architecture.md`
Niharika 03: speech to text	`Niharika/03-speech-to-text.md`
Niharika 04: talk-back text to speech	`Niharika/04-talkback-tts.md`
Niharika 05: topic knowledge	`Niharika/05-topic-knowledge.md`
Niharika 06: live demo	`Niharika/06-ltx-live-demo.md`
Niharika 07: roadmap, testing and risks	`Niharika/07-roadmap-testing-risks.md`

Dhiraj: RAG and OCR build notes `Dhiraj/README.md`

Each document starts on a new page. The first part is the README, which is the best single overview; the rest is the product definition, the engineering log (every version and every bug with its fix), design notes and the modules written by each teammate.

Overview

README: overview, features, models, languages, tech stack, API and the documents that follow

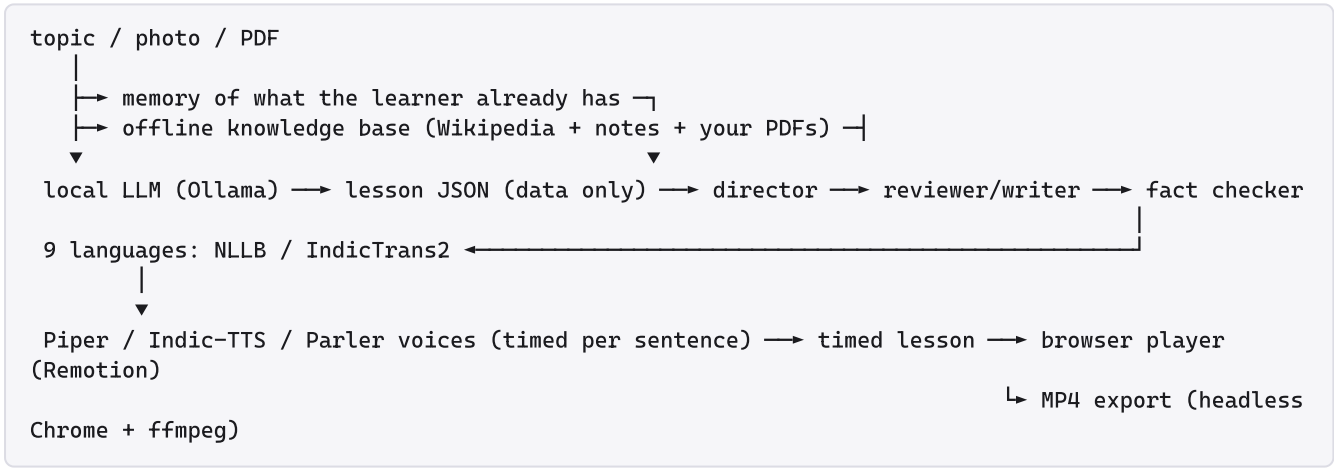
README.md

Chanakya

> Type a topic, attach a photo or a PDF, and get a narrated, animated video lesson in English and eight Indian languages, with quizzes, study tools and a tutor. Everything runs offline on one laptop (and on a phone). No outside AI service is ever called.

Latest release: v0.16.0 · Built for the Unlox hackathon · Repo: https://github.com/signingoff-dubey/UNLOX-TEACHER

Team name	The Last Commit
Members	Aryan Dubey, Priyadarshini Prasad, Niharika K P, Dhiraj Shetty
College	RV University, Bengaluru
Course	B.Tech (Hons)
Branch	Computer Science Engineering
Year of study	2nd year



The rule that shapes everything: the AI only ever returns *data* (a JSON lesson that is checked against a schema). It picks one of 11 scene templates and fills in text; our own code draws, times and renders every frame. Nothing the model writes is ever executed.

Contents

1. Features
2. Flash and Pro
3. How a lesson is made
4. Models used
5. Languages and voices
6. Tech stack

7. Quick start
8. Configuration
9. API
10. Project layout
11. Quality, safety and fact checking
12. Android app
13. Testing
14. Performance and hardware
15. Known limits
16. Licences
17. Docs and team
18. Changelog

Features

Make a lesson

- **Any topic, any length from 30 s to 4 min.** Longer videos are written in parts; a course is 1 to 3 videos, and "Auto" lets the AI decide how many concepts a topic has.
- **11 scene templates:** `title`, `bullets`, `flow`, `cycle`, `compare`, `diagram`, `quiz`, `summary`, `graph` (11 curve types: line, parabola, cubic, sine, cosine, exp, abs, sqrt, reciprocal, circle, ellipse, plus points, segments, tangents and chords), `code` (syntax-coloured, a highlight that follows the narration, program output), and `network` (boxes and arrows laid out automatically: trees, linked structures, pipelines, architectures).
- **Modes:** Explanation, Study (key terms, memory tricks, a recall quiz), Homework (teaches the method, writes the answers into a separate file you open in the browser) and Revise (re-teaches what you keep getting wrong).
- **Level:** Beginner or Intermediate. The learner's age (Profile) shapes the wording.
- **Photos:** attach a photo of a page, a board or a worksheet; a local vision model reads it. The Create button never waits for it: the photo is read inside the job.
- **PDFs up to 2 GB / 1,000 pages:** a chapter ("chapter 4"), a page range ("pages 120 to 135") or the whole book. Text-layer pages are read as Markdown (headings, lists, tables kept), scans go through OCR, pictures inside the PDF are described by the vision model, and study notes are written while the book is read. Citations such as `Source: Science 8, Chapter 2, pp. 41–44` appear in the recap.
- **Ask in your own words:** "teach me about the solar system" becomes the title "The solar system"; "how does the rainbow have 7 colours, why does it come only when sunlight and rain are together" is answered as two questions.
- **Memory:** the planner is told which related videos you already have and does not repeat them.
- **Reference notes you can add:** drop `.md` / `.txt` files in `data/kb_extra/` (this repo ships notes on data structures and AVL trees, MERN, coordinate geometry and Python) and rebuild the index.

Watch and study

- **Player** with scene list, captions timed to the audio, language tabs, auto-advance to the next video of a course, and early playback (Flash starts about 6 to 18 s after you ask).

- **Tutor chat** (text or voice) that knows the lesson. Say "generate one more video on ..." and it builds one, shows its first frame in the chat and adds it to the playlist.
- **Study tools** written from the lesson in your languages: flashcards, quiz, mind map, study guide, FAQ, glossary, timeline.
- **Review** (spaced repetition of what you got wrong), **Teach it back** (explain it and be graded), **Rewards** (streak, score, level, daily goal, badges), **Progress**, **Profile**.
- **Learnverse:** a walkable 3D world (three.js) the local model builds from your topic or from everything you studied, with quiz gates, points and a shop.

Take it away

- MP4 export (720p or 1080p, H.264 + AAC), `.srt` captions per language, narration-only audio, the whole lesson as one `.chanakya.zip` with its own player (copy it to a USB stick, import it in Library), worksheets and question papers (HTML).
- Backend address is configurable, so the UI can use a backend on another laptop.

Built to keep working

- Jobs resume after a crash; GPU out-of-memory falls back to the CPU and retries the episode; a memory gate waits when the laptop is short of RAM or VRAM; the PyTorch allocator is capped so the language model is never starved.
- Translation is checked for script and meaning (back-translation); a bad translation is replaced by English rather than shown.
- `GET /api/ready` and a start-up self-check say exactly what is missing and what to run.

Flash and Pro

Pick it on the Home page (the "Build" switch) or send `"quality": "flash" | "pro"`. Both use the same model.

	Flash	Pro
Idea	Scenes appear within seconds and flow into each other while the rest is still being written	The whole video is planned first; nothing is shown until it is complete
How	A short outline, then parts of about 40 s written one after the other, each told what the earlier parts taught; the first scenes stream to the player token by token	A scene-by-scene outline of the whole video first (`longplan.beat_sheet`); the first minute is written in one call; each further minute is patched on in a call that sees the whole outline and the full text so far (`plan_pro`)
Fact check	50 s budget	100 s budget
Measured ("teach me what avl trees are", 2 min)	133 s, first scenes after 18 s	167 s, shown at the end; covers balance factor, single and double rotation, insertion, deletion and uses
Choose it when	You want to start watching now	You want the most coherent video and can wait

How a lesson is made

1. **Clean the request** (`content.clean_topic`): "Teach me about dsa in java" is "DSA in Java". Questions are left as they are. A request with two questions must be answered twice.
2. **Remember** (`memory.py`): related lessons you already have are summarised for the planner.
3. **Retrieve** (`knowledge.py` , `rag/`): top passages from the offline knowledge base, your PDF first if you attached one.
4. **Plan** (`planner.py`): a prompt built for the subject (maths gets graph scenes, programming gets code scenes in the language you named, trees and structures get network scenes); Flash or Pro as above. The reply is parsed and repaired by `schema.normalize` (small models slip on types) and sent back with feedback if it repeats headings or code, has narration under 18 words, ignores a question, shows a technical topic only as text, or leans on everyday comparisons.
5. **Clean** (`content.py`): repeated scenes, headings and openings are dropped, one summary is kept, empty title cards removed, repeated everyday comparisons stripped.
6. **Direct** (`director.py`): a visual director replaces weak scene choices (a list that should be a flow, a graph with nothing to draw).
7. **Review** (`review.py`): a writer pass lengthens short narration; Python snippets are parse-checked and repaired.
8. **Fact check** (`facts.py`): see below.
9. **Translate** (`translate.py`): batched by length, cached, pre-translated while the planner is still writing.
10. **Narrate** (`tts.py` , `expression.py`): each sentence is spoken with the delivery that fits it; each scene lasts exactly as long as its audio plus 0.5 s.
11. **Show**: the Remotion player draws the timed JSON in the browser, or the renderer exports an MP4.

Models used

Everything below runs locally. Nothing is called over the internet at run time, and models whose name ends in `-cloud` / `:cloud` (remote servers) are refused.

Role	Model	Runs on	Notes
Planner, director, writer, fact checker, tutor, study tools, Learnverse builder	**Qwen2.5-7B-Instruct** Q4_K_M (`qwen2.5:7b-instruct-q4_K_M`) through **Ollama**	GPU (5.5 GB, 100 % resident), about 43 tokens/s	Apache-2.0. Context 6144 tokens. Same model for Flash and Pro
Photo reader, pictures inside PDFs	**LLaVA 7B** (`llava:7b`)	GPU, loaded only when needed	The language model steps aside while it runs
Knowledge-base embeddings	**nomic-embed-text** (Ollama)	CPU/GPU	768 dimensions
PDF passage embeddings	**multilingual-e5-small** (sentence-transformers)	CPU	

Role	Model	Runs on	Notes
Re-ranking	**bge-reranker-v2-m3**	CPU/GPU	Re-ranks retrieved passages (PDF search)
Translation	**NLLB-200 distilled 600M** (default), **AI4Bharat IndicTrans2 dist-200M** (optional, gated), Ollama as a fallback	GPU, released after 10 idle minutes	Back-translation check
Narration	**Piper** voices (en, hi, te, ml, mr, bn), **AI4Bharat Indic-TTS** int8 ONNX (en, ta, kn, gu, plus bn/mr), **Indic Parler-TTS** (GPU fallback, takes a tone description)	CPU (Piper, Indic-TTS), GPU (Parler)	`expression.py` sets per-sentence speed, pauses and emphasis
Speech to text	**faster-whisper** (`STT_MODEL` , default medium; small is bundled)	GPU if 2.2 GB is free, else CPU	Loaded on the first spoken question, cached
Scanned pages	**Tesseract** (fast) then **Unlimited-OCR** (accurate, when the GPU is idle)	CPU / GPU	Gives way to lessons
Phone	**Gemma 4 E2B** (LiteRT-LM), **Piper** via sherpa-onnx, **Whisper base** and **Vosk**, **ML Kit** translation and OCR	On the phone	See Android app

A larger model was measured and rejected for this card: `qwen3.5:9b` needs 8.8 GB with the context window, so only 72 % fits in 8 GB of VRAM; it ran at 17 tokens/s against 43 and returned invalid JSON on the test plan (see `issues.md` , ISSUE-046).

Languages and voices

Nine languages. English is always available; the lesson is sized for the slowest voice you pick.

Language	Code	Translation	Voice engine	Speed (words/s)
English	`en`	none	Piper `en_US-lessac` / `en_GB-alan` , Indic-TTS	2.7
Hindi	`hi`	NLLB / IndicTrans2	Piper `hi_IN-priyamvada` , `hi_IN-pratham`	1.9
Marathi	`mr`	NLLB / IndicTrans2	Piper `mr_IN-google` , Indic-TTS	1.8
Bengali	`bn`	NLLB / IndicTrans2	Piper `bn_BD-google` , Indic-TTS	1.8
Gujarati	`gu`	NLLB / IndicTrans2	Indic-TTS, Parler	1.8
Tamil	`ta`	NLLB / IndicTrans2	Indic-TTS, Parler	1.7
Kannada	`kn`	NLLB / IndicTrans2	Indic-TTS, Parler	1.7
Telugu	`te`	NLLB / IndicTrans2	Piper `te_IN-padmavathi` , `te_IN-venkatesh`	1.6
Malayalam	`ml`	NLLB / IndicTrans2	Piper `ml_IN-meera`	1.6

Fonts: Noto Sans for every script (SIL OFL), bundled so the browser and the renderer draw Devanagari, Telugu, Tamil and the rest correctly, including conjuncts. If a voice is missing the next engine is tried, and finally silent audio with correct timing.

Tech stack

Layer	Technology
Language model runtime	Ollama (local), Qwen2.5-7B, LLaVA, nomic-embed-text
Backend	Python 3.12, FastAPI 0.115, Uvicorn, Pydantic 2 (schema and validation), `requests`, SQLite (jobs, learning data, passage store)
ML	PyTorch (CUDA 12.8 wheels for RTX 50-series), Transformers 4.57, sentence-transformers, FAISS, ONNX Runtime, piper-tts, faster-whisper, NLLB-200, IndicTrans2
Documents	PyMuPDF + pymupdf4llm (PDF to Markdown), Tesseract (pytesseract), Unlimited-OCR, Pillow
Frontend	React 18, TypeScript 5.8, Vite 6, Remotion 4.0.539 (player and scene templates), three.js (Learnverse), Mermaid 11 (mind maps), Figtree and Chakra Petch fonts
Video export	Remotion renderer (headless Chrome) + bundled ffmpeg, Node 20+
Android	Kotlin, Jetpack Compose (Material 3), Room, WorkManager, Media3, LiteRT-LM (Gemma), sherpa-onnx, Vosk, ML Kit; min SDK 31
3D lessons (teammate)	Headless Blender pipeline in `blender 3d/`
Tests	pytest (345 backend tests), `tsc --noEmit` for the web app

Quick start

Windows, PowerShell. You also need **Ollama** (https://ollama.com).

```
git clone https://github.com/signingoff-dubey/UNLOX-TEACHER.git
cd UNLOX-TEACHER
.\setup.ps1 -Gpu      # once, online. Leave out -Gpu for CPU only (Piper voices + Ollama
translation)
.\start.ps1           # opens http://127.0.0.1:8000  (add -Lan for other devices)
```

Manual setup of the backend (Python 3.12):

```
cd backend
py -V:3.12 -m venv venv
.\venv\Scripts\pip install -r requirements.txt
.\venv\Scripts\pip install torch --index-url https://download.pytorch.org/whl/cu128   # GPU
.\venv\Scripts\pip install -r requirements-ml.txt -r requirements-rag.txt
ollama pull qwen2.5:7b-instruct-q4_K_M
ollama pull nomic-embed-text
ollama pull llava:7b                                          # photos and pictures in PDFs
.\venv\Scripts\python scripts\download_models.py [--ml]       # voices, fonts, models still missing
.\venv\Scripts\python scripts\build_knowledge.py              # the offline knowledge base (online
once); later: --no-fetch
cd ..\web ; npm install ; npm run bundle ; npm run build      # UI + video renderer
cd ..\backend ; .\run.ps1
```

Then open http://127.0.0.1:8000 and check **http://127.0.0.1:8000/api/ready**, which lists what is missing and the command that fixes it.

Before a demo: set `HF_HUB_OFFLINE=1` and `TRANSFORMERS_OFFLINE=1`, turn Wi-Fi off, and make one lesson end to end. Development: run the backend, then `cd web; npm run dev` (UI on :5173, proxied to :8000); `npm run studio` previews every template without the backend; `LLM_BACKEND=mock` runs the pipeline without Ollama.

Configuration

Set in `backend/.env` (copy `.env.example`) or the environment. Common ones:

Variable	Default	Meaning
`OLLAMA_MODEL`	`qwen2.5:7b-instruct-q4_K_M`	The language model (must be in `ollama list`)
`VISION_MODEL`	`llava:7b`	Reads photos and PDF pictures
`EMBED_MODEL`	`nomic-embed-text`	Knowledge-base embeddings
`OLLAMA_NUM_CTX`	`6144`	Context window; the same for every call (changing it reloads the model)
`LLM_RETRIES`	`2`	Extra tries when Ollama is busy or restarting
`RAG_ENABLED`, `RAG_TOP_K`, `RAG_MIN_SCORE`	`1`, `6`, `0.68`	Retrieval
`FACTCHECK`, `FACTCHECK_BUDGET_S`	`1`, `50`	The fact checker and its time budget (Pro doubles it)
`TRANSLATE_BACKEND`	`nllb`	`nllb`, `indictrans2`, `ollama`, `none`
`BACKCHECK`	`1`	Translate back to English and compare
`TTS_ENGINE`, `TTS_FALLBACK`, `TTS_EXPRESSIVE`	`piper`, `parler`, `1`	Voices and expressive delivery
`STT_MODEL`, `PRELOAD_STT`	`medium`, `0`	Speech recognition
`DEVICE`, `OLLAMA_NUM_GPU`	`auto`	GPU/CPU choice

Variable	Default	Meaning
`TORCH_VRAM_FRACTION`	`0.35`	Share of the card PyTorch may hold
`MIN_FREE_RAM_GB`, `MIN_FREE_VRAM_GB`, `MODEL_IDLE_S`	`1.5`, `0.8`, `600`	The resource gate
`PDF_FIGURES`, `PDF_MAX_FIGURES`	`1`, `40`	Describe pictures inside PDFs
`PUBLIC_BASE_URL`, `CORS_ORIGINS`, `NODE_BIN`		Serving and export

Secrets are never committed (`.env` is git-ignored).

API

Long work runs as a job: POST returns a job, then poll `GET /api/jobs/{id}` until `status` is `done` or `error`.

Group	Endpoints
Status	`GET /api/ready` (what is missing, what to run), `GET /api/health`, `GET /api/languages`, `GET /api/jobs/{id}`
Lessons	`POST /api/lessons` (`concept`, `languages`, `level`, `duration_s` 30-240, `mode`, `quality`, `upload`, `focus`, `images`), `GET /api/lessons`, `GET /api/lessons/{id}`, `POST /api/lessons/{id}/export`
Courses	`POST /api/playlists` (same fields plus `episodes` 0-3), `GET /api/playlists[/{id}]`, `POST /api/playlists/{id}/export`
Sources	`POST /api/uploads` (PDF), `GET /api/uploads/{sha}[/notes]`, `POST /api/search`, `GET /api/knowledge/search?q=...`, `POST /api/images`, `GET /api/images/{sha}`
Homework	`POST /api/homework`, `GET /api/homework[/{id}[/download]]`
Learning	`POST /api/lessons/{id}/chat`, `.../studio[/{kind}]`, `.../teachback`, `GET /api/review/due`, `/api/review/stats`, `POST /api/review/answer`, `/api/weak*`, `/api/profile`, `/api/rewards*`
Take away	`GET /api/lessons/{id}/subtitles`, `POST .../podcast`, `.../say`, `GET .../bundle`, `POST /api/bundle/import`, `/api/papers*`
Voice and world	`POST /api/stt`, `POST /api/metaquiz`, `POST /api/metaworld`, `GET/POST /api/metaverse/save`
Files	`GET /media/...` (audio, MP4, `.srt`), `GET /fonts/...`

The lesson JSON shape is mirrored in `web/src/types.ts`; the plan schema is `backend/app/schema.py`. To render a track in any React app, give it to `LessonVideo` (`web/src/remotion/Lesson.tsx`) inside Remotion's `<Player>` with `inputProps={{ track, assetBase: "" }}`.

Project layout

Path	What
`backend/app/planner.py`, `longplan.py`	Prompts by subject, Flash parts, Pro outline and blocks
`backend/app/content.py`, `memory.py`, `facts.py`	Quality rules, what already exists, fact checker
`backend/app/pipeline.py`, `playlist.py`, `streaming.py`	Orchestration, courses, scenes that play while they are made
`backend/app/schema.py`, `director.py`, `review.py`	Lesson schema and repair, visual director, writer/reviewer
`backend/app/knowledge.py`, `rag/`, `ocr/`	Retrieval, PDF ingestion, OCR, Markdown, figures, notes
`backend/app/translate.py`, `tts.py`, `expression.py`, `stt.py`, `languages.py`	Languages and voices
`backend/app/vision.py`, `homework.py`, `chat.py`, `videochat.py`, `studio.py`	Photos, homework, tutor, extra videos, study tools
`backend/app/jobs.py`, `resume.py`, `resources.py`, `gpu.py`, `selfcheck.py`	Jobs, crash resume, memory gate, GPU slot, readiness
`backend/scripts/`	`build_knowledge.py`, `fetch_library.py`, `download_models.py`
`backend/tests/`	345 tests
`web/src/remotion/`	Scene templates, graph, code, icons, themes
`web/src/pages/`, `components/`, `studio/`	Home, Lesson, Library, Rewards, Progress, Review, Profile, Settings; study tools
`metaverse/`	The walkable 3D world (Learnverse)
`Unlox_Hackethon/`	The Android app (Kotlin, Compose)
`blender 3d/`	Offline narrated 3D lessons with headless Blender (Dhiraj)
`data/topics.txt`, `data/kb_extra/`	Wikipedia topic list; reference notes (committed)
`data/curated/`	Hand-written lesson plans (git-ignored)
`docs/`	Design notes; `docs/README-notes.md` is the older detailed README

Quality, safety and fact checking

- **Schema first.** Every model reply is parsed as JSON, repaired and validated; bad plans are retried with the error, and a plan that is weak (repeats, short captions, an unanswered question, a technical topic with no code or drawing) is sent back with feedback. A minimal fallback lesson means a job never ends empty.

- **Grounded.** The planner reads retrieved passages (Simple English Wikipedia, reference notes in `data/kb_extra/`, your PDFs). Sources are listed with each lesson.
- **Fact checker (`facts.py`).** Runs on the English plan before translation, using no outside service: 1. *Calculation:* arithmetic and linear equations are worked out ("x = 20 / 3" for 3x + 5 = 20 is corrected to x = 5); a count that does not match its list ("7 colours: red ... indigo") and a few lists every child learns (rainbow, planets in order, continents, oceans, senses) are checked against the real list. 2. *Sources:* each statement is compared with passages that are about it; a verdict only counts if the judge quotes words that really are in the source, and a focused second question must agree before anything changes. 3. *Second opinion:* the model, when no source covers a statement, may only say "wrong" if the opposite question ("is it true?") agrees. 4. *Repair:* a correction must pass the same check, keep what the sentence taught and have no notes or brackets, else the statement is removed (never leaving a scene too thin). Quiz answers change only when a source and the model agree. Analogies are not treated as claims. The report (checked, confirmed, corrected, removed, sources) is saved with the lesson and shown in the notes panel.
- **Safe translation.** Script checks and back-translation; a failure keeps English.
- **No code runs.** Code scenes are text; Python is only parse-checked.
- **Privacy.** Everything stays on the machine; the learner profile is a local file.

Android app

`Unlox_Hackethon/` is the same product as a native app that runs fully on the phone (tested on a OnePlus 13R, 16 GB): lesson planner, tutor and homework solver on Gemma 4 E2B (LiteRT-LM), narration with Piper through sherpa-onnx (and the phone's voice for Tamil and Kannada), speech to text with Whisper + Vosk, translation and OCR with ML Kit, long videos in parts, a set of videos that plays on by itself, and "make one more video" in the chat. Models (about 4 GB) are not in git: copy them to `Android/data/com.kabir.unlox_hackethon/files/models/` (`llm/`, `stt/`, `tts/`, `embeddings/`). Build with `gradlew :app:assembleDebug` (JDK 21). The Android app does not have the Flash/Pro switch, the fact checker or the memory yet.

Testing

```
cd backend ; .\venv\Scripts\python -m pytest -q          # 345 tests; run with the backend
stopped (a few thread-timing tests are flaky under load)
cd ..\web ; npx tsc --noEmit ; npx vite build
cd ..\Unlox_Hackethon ; .\gradlew.bat :app:assembleDebug
```

Live checks used while building: lessons on the aquatic life of earth, the solar system, rainbows, the MERN stack, why the sky is blue, how steel is made, how a refrigerator works, and AVL trees, in Flash and Pro, with the fact checker on.

Performance and hardware

Built and measured on a laptop with an RTX 5060 (8 GB VRAM), 15 GB RAM, Windows 11.

- Language model: 43 tokens/s, 100 % on the GPU. First scene of a short lesson about 1.2 s warm; the player opens 6 to 18 s after you ask (Flash).
- A 2-minute video: about 2 to 3 minutes to build (planning, checking, translation, narration), then export about 1.5 times its length.
- PDF reading: text-layer pages 1 to 15 ms each, Markdown 50 to 500 ms, OCR about 1 s; a 1,000-page scan is searchable in under 3 minutes.
- Translation of 32 sentences: 2.9 s after batching and caching.
- The three GPU users (language model, translator, voices) share 8 GB by taking turns; the resource gate and a PyTorch cap keep the language model from being starved.

Known limits

- The 7B model still gets facts wrong now and then. The fact checker removes or corrects what it can prove wrong; the rest is reported as "not confirmed".
- Code is shown, not run; only Python is syntax-checked.
- Very short labels can be mistranslated for lack of context; IndicTrans2 is gated and needs a C compiler on Windows (NLLB is the default).
- Android lacks Flash/Pro, the fact checker and memory; its changes since v0.15.0 are compiled but not tested on the device.
- Only one extra chat video at a time.
- Hand-written lessons are not fact-checked.

Licences

Component	Licence
Qwen2.5-7B-Instruct	Apache-2.0
LLaVA, nomic-embed-text, bge-reranker, multilingual-e5-small	see each model card
NLLB-200	CC-BY-NC (research/education use; check before commercial use)
IndicTrans2, AI4Bharat Indic-TTS	MIT (gated download; ONNX repack is unofficial)
piper-tts	GPL-3.0-or-later (voices: per-voice cards)
Wikipedia text	CC BY-SA 4.0 (downloaded at setup, each lesson lists its articles)
Noto fonts	SIL OFL 1.1
Remotion	Remotion licence (free for individuals and small teams; read the terms)
ffmpeg	LGPL/GPL by build
NCERT / OpenStax (optional library)	NCERT: local use only; OpenStax: CC BY 4.0

Excluded: Meta MMS-TTS (CC-BY-NC), sarvam-translate (GPL tag + Gemma base). Please verify the licences of anything you redistribute.

Docs and team

`prd.md`, `problem_statement.md`, `pitch.md`, `progress.md` (versions and git history), `issues.md` (every bug and its fix), `todo.md`, `docs/` (long videos, voice assistant, presentation mode, RAG and OCR design), `blender 3d/README.md`, `CLAUDE.md`.

Team folders in the repo: `Dhiraj/` (Dhiraj Shetty's snapshot of the RAG, OCR and library work), `Niharika/` (Niharika K P's voice-assistant and metaverse notes), `Priya/` (Priyadarshini Prasad's screen designs). See the team table at the top.

Changelog

Current: **v0.16.0** (2026-10-29): Flash and Pro, memory of what already exists, technical topics shown with code and network scenes, reference notes, retries and readiness checks, fact checker. Earlier: v0.15.2 coherent long videos and courses, "Teach me" removed from titles, expressive narration, non-blocking photos; v0.15.0 subject-aware prompt and faster first scene, PDF progress, Android parts and chat video; v0.14.x PDF Markdown and figures, caching, Learnverse worlds, chat videos, resilience; v0.13 Android app; v0.10 photos, modes, homework; v0.9 grey-glass home, rewards, long videos. Full per-version notes: `progress.md` and `docs/README-notes.md`.

Product

Product requirements (PRD) and the documents that follow

Product Requirements Document

Product Name

Chanakya (repo: UNLOX-TEACHER) — Codestorm: Unlox 24-Hour Hackathon

Problem Statement (one line)

Turn a typed concept into a 30–90 s narrated visual lesson in English + Indian languages, fully offline.

Target Users

Beginner/intermediate learners on CPU-only laptops; hackathon judges (primary audience for demo).

Core Features (must-have)

1. Concept input: topic, level (beginner/intermediate), language, target duration
2. Lesson planner: local LLM via Ollama -> JSON matching `backend/app/schema.py`; schema check + automatic fallback to built-in sample lesson
3. Visual engine: reusable components, >= 3 scene types (Title Card, Diagram w/ labels+arrows, Code Walkthrough)
4. Multilingual narration: local TTS + captions in English (Indian accent) + >= 2 Indian languages, synced to visuals
5. Audio-first timing: scene duration = narration audio length + short pause
6. Interactive in-browser player (play/pause/seek, language switch)
7. MP4 export: 720p H.264, voiceover + burned-in captions and/or SRT
8. Continuous generation: scenes are made while the planner writes (first scene in about a second), long videos in 30 s parts, every episode of a course streams, extra videos can be asked for in the tutor chat (website and Android app)
9. PDF as a source: Markdown for pages with a text layer, OCR for scans and broken fonts, llava for pictures inside the PDF, one continuous progress bar
10. Stays responsive on an 8 GB laptop: priority, headroom gates, idle release, GPU fallbacks (docs/presentation-mode.md)

Nice-to-Have Features

- Support every Indian language with an offline voice (see Languages below)
- Offline translation (IndicTrans2) so the planner writes English once
- Auto-select planner model by free RAM
- Per-language SRT download; script-correct fonts

Platform / Environment

Windows/Linux laptop, CPU-only target (8–16 GB RAM), browser UI, Node + Python, Ollama, ffmpeg. No internet at run time.

Constraints

- No external AI APIs (OpenAI/Gemini/Claude/ElevenLabs/Google Translate/cloud voices)
- Fully offline; Revideo telemetry disabled (`DISABLE_TELEMETRY=true`)
- LLM returns JSON data only, never code
- All models/voices/assets listed in README with valid licenses (no NC licenses; Meta MMS excluded)
- 24 h hackathon; Gate 1 (fixed sample lesson -> MP4) by hour 5

Languages (offline TTS availability, verify per voice before demo)

Languages	Engine	License
hi, te, mr, bn, ml	Piper voices	MIT engine; verify per-voice
ta, kn, gu, en	AI4Bharat Indic-TTS (int8 ONNX repack)	MIT (upstream); repack unverified
pa, or, as	Indic-TTS official zips (optional)	MIT

Success Criteria (judging weights)

Working end-to-end 25%, Teaching quality 20%, Visuals 15%, Languages 15%, Engineering 15%, Speed 10%.

Out of Scope

Cloud anything, native mobile app, STT/Whisper, generative video, user accounts.

Team

Team name	The Last Commit
Members	Aryan Dubey, Priyadarshini Prasad, Niharika K P, Dhiraj Shetty
College	RV University, Bengaluru
Course	B.Tech (Hons)
Branch	Computer Science Engineering
Year of study	2nd year

Ownership

- Teammate: backend (`backend/`) + frontend (`web/`) — merged from origin/main
- This workstream: docs, model/asset setup, benchmarks, orchestration design (frontend `web/` is the teammate's, Remotion-based)

Problem Statement

The Problem

Learners in India need short visual explanations of concepts in their own language, but good ones are online-only, English-only, or need paid cloud AI.

Who Is Affected

Students (beginner/intermediate) on ordinary CPU-only laptops (8–16 GB RAM), often with poor or no internet, whose first language is not English.

Why Existing Solutions Fail

- Depend on cloud LLM/TTS/translation APIs (cost, privacy, need internet)
- English-first; Indic voices are robotic or missing
- Generic video generators do not teach (no structure, diagrams, or code walkthroughs)

Our Solution

Chanakya (UNLOX-TEACHER): a local-first, offline engine. Typed concept -> local LLM lesson plan (validated JSON) -> local multilingual narration (audio sets scene length) -> React/Revideo visual scenes -> 720p MP4 with captions.

Definition of Success

- Judge-given prompt becomes a 30–90 s lesson, offline, live, in English + 2+ Indian languages
- Plays interactively in browser and exports 720p H.264 MP4 with captions
- Audio, captions and visuals stay in sync
- Every model/voice/asset licensed and listed in README

Project Pitch

One-Line Pitch

Type a concept, get a narrated visual lesson in your own language, built entirely on your laptop, offline.

The Problem

Good explainer videos are English-first, cloud-dependent and not made for the learner's concept or language.

Our Solution

A local pipeline: Ollama plans the lesson as validated JSON, local TTS narrates it in English and Indian languages, a React/Revideo engine animates it, and ffmpeg exports a 720p MP4 with captions. Audio sets scene length, so voice, captions and visuals never drift.

Key Features

- Works with Wi-Fi off; no external AI API. A website on the laptop and a native Android app that runs everything on the phone
- **Plays while it is made**: the first scene is watchable in about a second after the planner starts; long videos are planned in 30 second parts and added to the player as they are written; every video of a course streams too
- Ask the tutor chat for "one more video on ..." and it appears in the chat with its first frame, joins the playlist and plays
- 15+ scene types: title, bullets, flow, cycle, compare, diagram, network, quiz, summary, graph (maths curves), code walkthrough
- English + eight Indian languages (Hindi, Telugu, Marathi, Bengali, Malayalam, Tamil, Kannada, Gujarati), with script and back-translation checks
- PDFs: pages with a text layer are read as Markdown (headings, lists, tables), broken fonts are detected and sent to OCR, pictures inside the PDF are described by llava
- Study tools (flashcards, quiz, mind map, notes, glossary), spaced repetition and weak spots, teach-it-back scoring, worksheets and question papers, offline lesson file for a USB stick
- Learnverse: a walkable 3D world the local AI builds for any topic, with quiz gates, points, a shop, a jump button
- Built for an 8 GB laptop: resource limits, GPU fallbacks and idle model release keep a 5-video course from freezing it

Target Audience

Hackathon judges (Unlox / Codestorm), then students and teachers on low-spec laptops.

Tech Stack

Ollama (Qwen2.5 7B, llava, nomic-embed), IndicTrans2 / NLLB, Piper + AI4Bharat Indic voices, faster-whisper, PyMuPDF + pymupdf4llm + Tesseract + Unlimited-OCR, FastAPI, React + Remotion, three.js, ffmpeg, Python; Android: Kotlin + Compose, Gemma (LiteRT-LM), sherpa-onnx (Piper, Whisper), ML Kit.

Why Us / Why Now

Open Indic TTS and small instruct models now run on CPU; privacy and offline use matter for classrooms.

Demo / Screenshots

TBD (add rendered MP4s under `samples/`)

Team

Team name	The Last Commit
Members	Aryan Dubey, Priyadarshini Prasad, Niharika K P, Dhiraj Shetty
College	RV University, Bengaluru
Course	B.Tech (Hons)
Branch	Computer Science Engineering
Year of study	2nd year

Built for the Unlox hackathon.

PLAN.md

Blu Visual Teacher — Project Plan (Codestorm: Unlox 24hr Hackathon)

1. The Challenge, Restated

Build a local-first, offline engine: typed concept → 30–90s narrated, animated lesson in English + 2 Indian languages. Plays in-browser, exports as 720p H.264 MP4.

Hard rules that constrain every decision below: - No external AI APIs (no OpenAI/Gemini/Claude/ElevenLabs/Google Translate/cloud voices) - Fully offline, Wi-Fi off, during actual execution - CPU-only laptop, 8–16GB RAM - LLM outputs JSON data only — never executable code - Every model/voice/asset must be listed in README with license

2. Non-Negotiable Architecture Rule

Audio sets scene length, not the reverse. Pipeline dependency: `JSON plan → per-scene audio generation → scene duration = audio length + pause → visuals rendered to that duration`. Parallelism happens **across scenes** (scene 3 renders while scene 1 composites), never across the audio→visual dependency within a single scene. Violating this breaks caption/narration sync, which is scored under both "Working End-to-End" (25%) and "Teaching Quality" (20%) — the two heaviest rubric items.

3. Stack Decisions (locked)

Stage	Choice	Why	License to verify
Lesson Planner	Local LLM via Ollama (Qwen2.5-7B-Instruct or Phi-3.5-mini, GGUF, Q4_K_M) + GBNF grammar / JSON schema constraint	Structured JSON output, fast on CPU, no per-language model-juggling	Check Qwen2.5/Phi-3.5 license terms
Schema validation	JSON Schema check in-app; hardcoded fallback lesson on failure	Required by organizers (Gate 1 milestone)	N/A
Translation (if needed)	`sarvam-translate:v1` (open-weights, GGUF via llama.cpp)	Confirmed open-weights, offline-capable, Indic-language coverage	Apache 2.0 — confirm on model card
TTS / Narration	Piper TTS (primary)	Fast, small, fully offline, proven Hindi/Indic voices	MIT — confirm per-voice license
STT	**None** — not needed	Narration text is already known (generated by planner); re-transcribing is redundant and adds a failure point	N/A

Stage	Choice	Why	License to verify
Captions	Derived from TTS word/phoneme timestamps (Piper) or estimated via word-count × speaking rate per language	Avoids a second STT pass	N/A
Visual Engine	React + SVG + CSS, 3 scene types: Title Card, Diagram w/ Labels+Arrows, Code Walkthrough	Matches organizer spec exactly; no generative video	N/A
Export	Browser canvas capture (MediaRecorder) or headless Puppeteer + ffmpeg mux	Both offline, no cloud dependency	ffmpeg: LGPL/GPL depending on build

Explicitly rejected: - **Colibri / large MoE models** — built for 744B-class frontier models (~370GB+ weights even quantized); wrong tool for a lightweight planner/TTS pipeline on a hackathon laptop. - **Sarvam Edge** — no confirmed public download path as of this writing; API-key-gated products found in docs are cloud calls, which violate the no-external-API rule. - **Whisper (any variant)** — unnecessary; you generate the narration text, you don't need to transcribe it back. - **Native Android app** — splits engineering time across two deployment targets in a 24h window; rubric doesn't score platform, only the 5-stage pipeline. Browser-only unless core pipeline is done with hours to spare.

4. Risk Register

Risk	Mitigation
TTS quality/speed varies badly by target language	Test Piper against your actual 2+ chosen Indian languages by **hour 2**, not hour 15
Planner LLM produces malformed JSON	Grammar-constrained decoding (GBNF) + schema validator + hardcoded fallback lesson (organizer-mandated)
Sync drift between audio/captions/visuals	Enforce the Section 2 dependency order — no exceptions for "speed" optimization
RAM overflow on 8GB machines	Dynamic quant selection by available RAM (reference: Uktam.ai's Q4_K_S/Q2_K switch pattern)

5. Timeline (organizer milestones, annotated)

- **0–5h (Gate 1):** Hardcoded sample lesson, full pipeline, exported as MP4. This beats a "clever but non-functional" system per organizer pro-tip — do not skip or rush this.
- **5–10h:** Ollama integration, schema validation, fallback logic.
- **10–15h:** Visual engine, player controls, all 3 scene types.
- **15–20h:** TTS integration, script fonts for Indic languages, translation if used, ffmpeg export pipeline.
- **20–24h:** Feature freeze. Offline testing (Wi-Fi actually off). Rehearse the 5-min live demo + judge-assigned topic flow.

6. Open Questions Before You Start Building

- Final 2 target Indian languages (determines whether Piper alone suffices or `sarvam-translate` is needed in the loop)
- Confirm Qwen2.5-7B / Phi-3.5-mini quantized model actually fits comfortably in your team's RAM budget — test, don't assume

Todo (kept so nothing is lost)

Legend: [ ] to do, [~] in progress, [x] done and tested

Website and backend (laptop)

- [x] Progressive scenes: scene 1 watchable ~6 s after the click on web (backend streaming preview + player), built on phone (install pending)
- [x] W1 Profile page (`#/profile`): name and age, age feeds the planner and tutor prompts
- [x] #1 Spaced repetition: SM-2 over flashcards and quiz, SQLite, "due today" on Home, review page, rewards hook
- [x] #2 Weak spots: log wrong quiz answers per concept; planner gets them as focus; "Revise" mode
- [x] #3 Misconception check: planner `common_mistakes` field becomes a compare scene
- [x] #9 Language fidelity: script validity + back-translation check after translation
- [x] #10 Glossary pronunciation: cached TTS clip per term, "hear it" button
- [x] #6 Subtitle-only and audio-only (mp3/wav "podcast") exports per language
- [x] #7 / W13 Offline lesson bundle (.chanakya zip: json, audio, srt, standalone player html); import with no backend, share over USB
- [x] #5 / W5 Question paper generator (MCQ, short, long, marks, answer key) to printable HTML/PDF; worksheet from a lesson
- [x] #4 Teach-back: speak or type an explanation, score by embedding similarity, list missed points (no generative judge)
- [~] W11 Course from a PDF: chapters become episodes, progress per chapter (backend playlists take a PDF; website chapters-as-episodes not built, phone has it)
- [x] #8 Diagram scene from nodes and edges (deterministic layout)
- [x] #11 Resumable jobs: queue in SQLite, resume at the last finished language
- [x] Docs, backend tests for every piece, committed locally (push only when asked)

Android

- [x] Real engines, Figma screens, scene templates, seek bar, full screen, PDF tile, videos x length (built; install done, on-phone test pending)
- [ ] On-phone test of the latest build (ask first, popup)
- [x] Android: parts, streaming episodes, chat video, lag fixes (compiles; v0.15.0)
- [ ] On-phone test of the v0.15.0 build: lag, parts, chat video, auto-advance (ask first, popup)
- [ ] Course from a PDF on the website (chapters as episodes)

Engineering log

Progress log and git history and the documents that follow

Progress Log

Current Version: v0.16.0

Status: In Progress (pre-Gate 1)

[v0.16.0] — 2026-10-29

What's New

- **Flash and Pro** (`quality` on `/api/lessons` and `/api/playlists`, a Flash/Pro switch on the Home page, same model for both). Flash is what it was: scenes appear within seconds and flow into each other while the rest is still being written. Pro shows nothing until the whole video is planned: the video is outlined scene by scene first (`longplan.beat_sheet`), the first minute is written in one call, and each further minute is patched on in a call that sees the whole outline and the full text written so far (`longplan.plan_pro`, `PRO_BLOCK_S = 60`); the fact check gets twice the time. Measured on "teach me what avl trees are" (2 min): Flash 133 s (first scenes after 18 s), Pro 167 s (nothing until the end); Pro covers balance factor, single and double rotation, insertion, deletion and applications, Flash stayed on a "stacked books" comparison
- **Memory of what already exists** (`memory.py`): every finished lesson is summarised (headings, key points, first lines of code); the planner is told which related videos the learner already has and must not repeat their hook, definitions, examples or code. A retry of the same request is not treated as something to avoid. Course episodes get the same through the course context; Pro also sees the other episodes
- **Technical topics are shown, not described**: data structures, trees, DSA, MERN and similar are now recognised as code topics; trees, linked structures, pipelines and architectures are drawn with the `network` template (it was never documented to the model before); the language the learner names ("in C language") is used for the code; a technical lesson of only text scenes is sent back; one everyday comparison per technical video is kept and the rest are removed (`content.strip_analogies`)
- **Reference notes for the fact checker and planner** (`data/kb_extra/`, indexed by `scripts/build_knowledge.py --no-fetch`): data structures and AVL trees (balance factor, four rotation cases, worked examples, C code), MERN, coordinate geometry, Python basics. Hand-written lesson plans live in `data/curated/` (git-ignored)
- **Robustness**: Ollama calls are repeated twice with a wait when it is busy or restarting (`llm.post`); `GET /api/ready` and a start-up log say what is missing and what to run (Ollama, models, knowledge base, disk, voices, video export); `quality` is validated; `build_lesson(plan=...)` builds a hand-written plan through the same voices, templates and renderer
- **Documentation PDF**: `Chanakya-Documentation.pdf` (cover with the team details, contents, README, PRD, problem statement, pitch, plan, progress, issues, design notes and every teammate module note; 137 pages) is built by `tools/build_docs_pdf.py` (Chrome headless). Team: The Last

Commit (Aryan Dubey, Priyadarshini Prasad, Niharika K P, Dhiraj Shetty), RV University, Bengaluru

- **Fact checker** (`facts.py`, from the previous work): arithmetic and canonical lists checked by calculation, statements checked against retrieved passages with a quoted proof, a focused second question before anything is changed, corrections must pass the same check; the report is saved with the lesson and shown on the web page

Known Issues

- Android has no Flash/Pro switch yet (it keeps its own Flash-style generation)
- A bigger local model was measured and rejected: `qwen3.5:9b` is 8.8 GB with the context window, only 72 % fits on the 8 GB card, 17 tokens/s against 43 for the current 7B, and its first answer was invalid JSON. Cloud models in Ollama are refused: the project is offline
- The 7B still gets facts wrong now and then (the fact checker removes what it can prove wrong); code in code scenes is shown, not run or verified (Python snippets are parse-checked)

[v0.15.2] — 2026-10-28

What's New

- **Long videos and courses are coherent**: a "teach me about dsa in java" video with a photo came out as four 30 s mini-lessons glued together (a title card per part, "Let's learn about X." placeholders, narration of 4 to 22 words, the same code and heading repeated, a summary per part). Now a video has at most 3 parts of about 40 s, planned one after the other, and each part is told what the earlier ones taught (`content.taught_summary`). Only the first part has a title card, only the last has the quiz and the summary (`content.clean_part`, `schema.normalize(segment, last, title)`), repeated headings, narration and code are dropped from the joined plan (`content.dedupe_plan`), narration under 18 words is lengthened by the reviewer (floor 22, cap 34), and the director removes empty mid-video title cards instead of turning them into code. A course is at most 3 videos, each told what the earlier videos taught. Measured on the real models (3-video DSA course with the photo): 12, 10 and 11 scenes, narration 27 to 33 words on average, 14 different code scenes that build up (array, access, linked list node, stack class, pitfall, queue), concrete episode titles
- **A photo no longer wrecks a lesson**: llava sometimes looped (a 104-line description); descriptions are deduplicated and cut to 15 lines (`vision.clean_description`, `repeat_penalty 1.2`), and in a course only the lines of the photo that fit an episode are given to it (`vision.relevant`)
- **"Teach me" is not part of the title any more**: `content.clean_topic` (backend, Android `Topic.kt`) takes the subject out of a request: "teach me about dsa in java" is "DSA in Java", "can you make a video on the water cycle please" is "The water cycle", "explain photosynthesis" is "Photosynthesis". Questions and plain topics are left alone, a request with nothing after it is kept whole
- **Expressive voice** (`expression.py`, `TTS_EXPRESSIVE=1`): every scene has a tone (title excited, quiz curious, summary calm, common mistakes serious, code calm, else warm) and every sentence its own delivery (a question slower with a longer pause, an exclamation or a hook quicker and brighter, a very short sentence slowed and stressed, a warning serious). Piper takes per-sentence speed, pitch/rhythm variation and volume, with pauses between sentences; Indic ONNX voices get the

pauses and a small speed change; Parler-TTS gets the tone as a plain-language instruction. Clips are cached per text and tone. Measured: pacing and pauses change (a clip is about 10 % longer than flat), pitch range barely moves (Piper cannot do true emotion; Parler on the GPU can)
- Web and Android: a course is 1 to 3 videos (Auto/1/2/3)
- **Checked on real lessons** (aquatic life, solar system, rainbow, MERN, sky, steel, refrigerator): topics come out clean ("The solar system", "MERN stack basics", "Why the sky is blue", "How is steel made"); fixed what they showed: a title card headed "string" (word copied from the schema), the same heading twice, two scenes opening with the same sentence, scenes still under 18 words after the writer pass. The full suite passes (294), including a test that failed only in the full run (see ISSUE-044)

Known Issues

- Voice expression is not on the phone yet (sherpa-onnx voices are flat), and nobody has listened to the clips: the numbers show pacing and pause changes, not how natural they sound
- The Android changes (parts, topic cleaning, 3-video courses, chat video, lag) compile but are not tested on the phone
- "Common mistakes" appears once per video; one snippet lost a closing brace in a comment
- A question with two parts ("how does the rainbow have 7 colours, why does it come only with rain") is kept whole as the lesson title; a small model may list six of seven rainbow colours (facts are not checked)

[v0.15.1] — 2026-10-28

What's New

- **A photo on the home prompt no longer blocks "Create"**: the button was disabled while llava read the photo (13 s or more) and the page only learned the photo's id when the read finished. Now the upload answers at once with the photo's id (`image_id`), the button is enabled straight away ("Reading, you can create now"), and the lesson, course or homework job reads the photo first if it has not been read yet, showing "Reading your photo" (`vision.ensure`, one reader per photo so llava is never asked twice). Measured on the real models: upload answered in 0.05 s, lesson started 0.03 s later. The read itself is a little faster (image side 1024, shorter answer)

Known Issues

- The first photo read after the model has been idle takes about 13 s (llava has to be loaded, and the language model steps aside on an 8 GB card)

[v0.15.0] — 2026-10-28

What's New

- **First scene sooner, and the same every time**: the planner's system prompt was about 1,700 tokens for every lesson, most of it graph and code templates, examples and rules that only maths and programming topics use. It is now built for the subject (`planner.system_prompt`, `llm.slim_schema_text`): a general topic gets 790 tokens (the reference block is also smaller for short lessons). Measured on the real model, alternating topics: time to the first finished scene 3.2 s on average (2.1 to 4.0 s, the 3-4 s against 6 s you saw) before, 1.2 s (0.9 to 1.3 s) now; prompt read 0.95 s to 0.30 s. Maths and programming topics keep exactly what they had; an unknown subject (a PDF, a photo) keeps everything
- **PDF upload bar is continuous and fast**: reading pages reported nothing until OCR started, indexing reported nothing, and the Markdown conversion made it 10x slower (385 ms a page). Now: vector-graphics analysis only for pages that have rules (38 ms a page, same text, ruled tables still found), a progress step for every page, one band per phase (reading to 30 %, OCR to 80 %, indexing section by section to 97 %, pictures), the bar in the browser is one run from the first byte (sending is the first 12 %, it used to jump back to 2 %), eased and creeping between reports, polled every 300 ms. A test found and fixed a jump backwards for PDFs without scanned pages
- **Android: the same continuous generation as the website**: videos over 90 s are planned in parts of about 30 s (outline first, part 1 streamed token by token, later parts handed to the player whole, in order); every video of a set is listed in the player as soon as its first scene exists, and the player moves on to the next video by itself (waiting for it while it is made); "make one more video on X" in the tutor chat (same recognition rules as the website) adds a video to the lesson's playlist and shows a card with its first frame and a play button; the planner prompt only carries the template docs the topic needs
- **Android: lag fixes**: the creator told the screen about progress on every streamed token (dozens of times a second, each rebuilding the whole app) and re-parsed the whole growing JSON for every token; now progress is published at most every 120 ms or when something visible changes, the JSON is parsed only when a closing brace has arrived, the lesson that is polled while it is made is only assigned when something changed, and the voice engine no longer runs with debug logging on four threads (3 threads, no debug). It builds; not yet measured on the phone
- **Learnverse graphics**: GPU sky with slow aurora curtains and a palette that drifts from sunset to twilight and back, twinkling stars, trees that sway in the wind (instanced vertex shader), colour sparks rising from every station, pulsing crystals and path, a colour-grade pass with vignette (stronger with speed) and film grain on high quality, a livelier avatar (breathing, squash and stretch on jump, dust on landing and while running), glass panels with an inner highlight, an animated sweep on the topic card

Known Issues

- The Android changes compile but are not tested on the phone (lag, parts, chat video, auto-advance)
- Phone: only one extra video is made at a time, and not while another lesson is being made
- The planner still writes about 90 s of narration for a 120 s request

---

[v0.14.4] — 2026-10-28

What's New

- **PDF to Markdown, not OCR, for every page that has a text layer** (`ocr/markdown.py`, `pymupdf4llm`). Measured on a test PDF with known text (title and bullets, two columns, a ruled table, a figure page, Hindi in Noto Sans Devanagari): the text layer is 98.8 to 100 % identical to the original, Markdown 100 %, OCR 68 to 100 % (two columns read across each other 68 %, table 90 %, figure page 88 %, Hindi 88 %); text layer 1 to 15 ms a page, Markdown 54 to 530 ms, OCR 750 to 1330 ms. Markdown also keeps headings (`#`), lists and real table rows, which OCR and plain text lose, so the planner and the search get structured text. OCR remains for scanned pages and for broken text layers
- **Short digital pages are no longer sent to OCR**: a page needed 200 characters of text to count as digital, so a table page, a figure page with a short caption and a Hindi page (all with a perfect text layer) were OCRed and came out worse. The limit is now 40 characters
- **Fonts**: a text layer that cannot be trusted is detected and the page goes to OCR instead: `(cid:12)` marks, private-use boxes, and the old Indian-language fonts (Kruti Dev and the like) whose text layer is Latin gibberish like `ekWuhVj dksf'kdk` where Devanagari should be (mixed-case and symbol-in-word test, and "Latin text in a document marked as an Indian language"). Ligatures (fi), soft hyphens, zero-width characters and words split at a line end are cleaned
- **Pictures inside a PDF are described by llava** (`ocr/figures.py`, `vision.describe_figure`): diagrams, graphs and photos are cut out (page-sized scans, icons and logos repeated on several pages are skipped, at most 40 per PDF, the pages you asked about first), described in the background (the PDF is searchable at once; the pass holds the GPU slot and steps aside for lessons), saved on the page and searched with it as `[Figure on page N: ...]`, so the planner sees them. Verified live with llava:7b on the test PDF (it read the three labels of the diagram; it also invented a fourth, so the prompt now says to mention only what is visible). Upload status reports `figures: found / described / state`
- **Caching and pre-warming**: translations are batched by length (35 % faster on mixed sentences, 4.5 s to 2.9 s for 32) and the cache is written atomically; while the planner is still writing, each finished scene is translated into the cache for the other languages (only when the translator is already on the GPU and memory is free), so the final pass finds it ready; a recording heard before is not transcribed again (cache of 300 clips; cold first question 10.2 s, warm 0.29 s, repeated 0 s); at start-up the embedding models are warmed, and the speech model is loaded once the computer is idle (it is released again after 10 quiet minutes); Whisper goes to the GPU when 2.2 GB is free (was 3)

Known Issues

- Drawn (vector) charts are not pictures inside the PDF, so llava does not see them; only embedded bitmaps are described
- llava can add details that are not in the picture; descriptions are tagged as figures and kept short
- IndicTrans2 still runs without its KV cache (a workaround for the remote model code), which makes long sentences slow
- Pages cached by an older version as "scanned" keep their OCR text (only new uploads use the better triage)

[v0.14.3] — 2026-10-28

What's New

- **Ask the chat for another video**: "generate one more video on black holes", "make another one about volcanoes, 2 minutes", "another lesson on tides" (`videochat.py`; recognised by its words, so it is instant and uses no GPU; questions like "how do I make a video?" or "give me one more example" still go to the tutor). The backend adds the video to the playlist of the lesson being watched (a lesson with no playlist is wrapped into one, as episode 1) and starts building it at once, scene by scene. In the chat a card shows its first frame with a play button as soon as scene 1 exists ("7 of about 7 scenes ready"); the play button opens it in the center player (it starts by itself) and the playlist under the player lists it ("playing, more coming..."). Verified in the browser
- **Every course episode streams**, not only episode 1: episodes 2 and 3 of a test course were watchable 13 s after they started; the player moves on to a streaming episode by itself

Known Issues

- A video asked for in the chat is as long as the model writes it (90 s asked, 51 s made)
- Only one extra video is built at a time (they queue); a course that is still being made cannot take extra videos

---

[v0.14.2] — 2026-10-27

What's New

- **A 5-video course no longer dies on "CUDA error: out of memory"**. Cause: that error is sticky (the GPU cannot be used by torch again in the process) and the cleanup after a failed translation (`_free_gpu`) raised it again, so no fallback ran and all 5 episodes failed with the same text after 384 s. Now: a CUDA error switches every torch model to the CPU for the rest of the process (`config.mark_cuda_broken`), cleanup never raises, the translator starts on the CPU when under 1.8 GB of VRAM is free (the language model holds most of the 8 GB), and a course episode that hits a CUDA error is retried once
- **Long videos stream too**: a 2 minute video is planned in 4 parts of 30 s (was 3 of 40 s). Part 1 is read token by token and its scenes are made while it is written; parts 2, 3 and 4 are added in order as each is finished, so the player starts after the first scenes and the rest follows. Measured on this laptop (cold model): first scenes watchable at 13 s, 5 scenes at 26 s, 15 scenes at 35 s, job done at 63 s
- **Courses**: episode 1 opens and plays while it is still being made (`preview` on the episode, `lesson_id` set early); the episode rail shows "playing, more coming..." and lets you pick it

- **Videos play by themselves again**: the position kept for a language switch (`resume`) was never cleared and was not tied to a lesson, so after one switch every later lesson or episode started at that old position and, if it was paused then, did not play. It now belongs to the lesson it was set in. When the browser has had no click yet (a reload) the video starts muted instead of not starting

- **The other languages are no longer starved by the study tools**: flashcards, quiz and the rest (background, 3 workers) started on the GPU the moment the English video was done and shared the card with the Hindi translation, which then took more than 9 minutes and had not finished. They now wait for the language job too (`jobs.active(..., "langs")`). Same 2 minute video: Hindi in 74 s
- **Root cause of the hangs found and fixed**: PyTorch's GPU allocator reserved 10.3 GB while translating (1 GB in use), beyond the 8 GB card, so Windows spilled into RAM (22 GB committed, 0.2 GB free). Now capped (`PYTORCH_CUDA_ALLOC_CONF`, `TORCH_VRAM_FRACTION=0.35`) and the cache is released after every batch: reserved 0.41 GB, process 4.4 GB in the same test
- **Resource manager** (`resources.py`) so a 5 video course cannot hang the laptop: the backend runs at below-normal priority (the desktop, browser and slides win), the maths libraries use half the cores, background work (extra episodes, study tools) waits while free RAM is under 1.5 GB or free VRAM under 0.8 GB, and speech recognition and the translator are released after 10 quiet minutes. Before, start-up put the translator, Whisper medium and every Piper voice on the 8 GB card next to the 5.5 GB language model, so the card was permanently full (desktop stutter, then CUDA out of memory). Now: Piper on the CPU (`PIPER_CUDA=0`), Whisper loads on the first spoken question (on the GPU only if 3 GB are free, else the CPU), study tools are not queued for every course episode (14 tools x 5 episodes of language-model work), only for episode 1 once the course is done, and 2 study builders instead of 3. `/api/health` reports free RAM and VRAM
- **Measured, the scenario that failed** (5 videos, 2 min each, English and Hindi, intermediate, this laptop): all 5 episodes ready in 496 s (it failed after 384 s before); episode 1 watchable at 25 s and ready at 72 s, then one more roughly every 90 s; lowest free RAM 4.5 GB (it was 0.2 GB), peak VRAM 7.1 of 8.15 GB, PyTorch's share of the card at most 1.97 GB (it was 10.3 GB), backend memory 8.9 GB (it was 22 GB)

Known Issues

- The finished plan is still checked afterwards (director, reviewer), so already-playing scenes can change a little when the final lesson replaces the preview
- Episodes 2 and up of a course are shown when complete (only episode 1 streams); two episodes plus the language model and the translator share 8 GB of VRAM, so on the CPU fallback Hindi takes longer
- Video length: the model writes about 90 s of narration for a 120 s request

---

[v0.14.1] — 2026-10-27

What's New

- **Learnverse builds a world for any topic** (`metaworld.py`, `POST /api/metaworld`): type a topic and the local model writes four stations with questions (data only; cleaned, translated for other languages). A "Building your world" screen with animated blocks and live progress shows while it works (about 25 s for a real topic). The topic box no longer defaults to Java: ready-made worlds (Java, photosynthesis, water cycle, the open lesson) start at once, everything else is built by the AI

- **No E key**: quiz gates open their question by themselves when you reach them, so a learner who knows the answer just answers. "Read the station first" and "Not yet" are offered on the question; stations are optional and tappable ("Read: ...")
- **Jump**: Space bar and a Jump button (also on touch)
- **Points and purchases are kept** in the browser and on the backend (`GET/POST /api/metaverse/save`, newer revision wins), so another address (localhost vs 127.0.0.1, a LAN IP) or cleared browser data no longer loses them. Verified: wiped the browser copy and the 265 points came back from the server
- **Home page**: the "+" (add photo) button sits with the Create button on the bottom row, both 46 px high, centred on the text when the box shrinks; the textarea height rule works again (it was ignored inside the column wrapper)

Known Issues

- Worlds are English first, then translated with the lesson engine; Indic fonts on in-world billboards still fall back to the system font
- The standalone `metaverse/standalone/learnverse.html` and the copy under `Dhiraj/` are not updated

[v0.14.0] — 2026-10-27

What's New

- **Scenes play while they are made** (web and phone). Before, the lesson was planned, narrated and saved whole, so nothing could be watched until the end. Now the planner's JSON is read as it streams (`streaming.py`): each finished scene is cleaned, translated, voiced and published to `lesson.json` (marked `building`), the player starts on scene 1 and the rest is added as it arrives. Measured: scene 1 watchable about 6 s after the click (a whole 60 s lesson took about 18 s), scenes 2-6 within 3 s of it. The player waits at the end of what exists and carries on by itself. Phone: `ScenePreview` does the same with Gemma (`StreamScenes.kt`)
- **Progress per stage** on the website (Plan, Design the visuals, Translate, Narrate, Finish; one row per video for a course), as on the phone
- **Study tools build in parallel** (3 workers, `STUDIO_WORKERS`): flashcards, quiz, mind map, study guide, FAQ, glossary and timeline overlap instead of queueing
- **Profile page** with name and age; the age shapes the planner's and the tutor's wording (`learner.py`)
- **Spaced repetition** (`srs.py` , SQLite): SM-2 over every flashcard and quiz question; Review page, "due today" card, rewards on remembered cards
- **Weak spots and Revise mode**: wrong quiz answers are logged per topic, right ones clear them; a Revise lesson gets the topics as the planner's focus
- **Misconception scene**: the planner's `common_mistakes` become one compare scene before the closing quiz
- **Network scene**: the model lists boxes and arrows, `layout.py` places them (layered, deterministic); drawn by a new Remotion template

- **Language fidelity** (`fidelity.py`): after translation each text is checked for the right script (mixed or Latin text is re-translated by the local model) and, with NLLB, back-translated and compared with the original; each scene carries its report
- **Glossary "Hear it"**, **subtitle-only** (srt, vtt, transcript) and **narration-only** (mp3 or wav) exports
- **Offline lesson file** (`bundle.py`): `.chanakya.zip` with the lesson, audio, captions and a standalone `player.html`; import on another computer from Library
- **Worksheet and question paper** (`paper.py`): MCQ, short, long, fixed marks, student and teacher copies, printable with the Noto fonts
- **Teach it back** (`teachback.py`): explain in your own words (typed or spoken); embedding similarity gives a score and the missed points. Threshold measured on a real lesson (off-topic 0.56 at most, paraphrase 0.75, cut at 0.62)
- **Resumable jobs** (`resume.py`): jobs are kept in SQLite; after a restart, lessons with languages still to make are finished from their saved plan, cut-off lessons and courses are closed cleanly (verified live)
- **Android**: animated scene templates, seek bar, full screen, PDF tile, videos x length, per-stage progress, progressive scenes, Piper voices working (sherpa-onnx 1.13.8)

Known Issues

- The finished plan is checked (director, reviewer) and can lengthen narration, so scenes already shown may change when the final lesson replaces the preview (50 s vs 33 s in one test)
- Scenes preview only for single lessons up to 90 s (longer ones are planned in parts, and course episodes are made whole)
- Paper generation takes about 90 s on the 7B model; teach-back needs English explanations (the embedding model is English)
- Course from a PDF on the website (chapters as episodes) is not built; the phone has it
- The progressive-scene phone build is not tested on the device

Future Scope

- Course from a PDF on the website; MiniLM search, MP4 export on the phone

---

[v0.13.0] — 2026-10-27

What's New

- **Progress page** (`#/progress`): streak heatmap (last 18 weeks), lessons / right answers / tutor questions / days counted for the week, month or year, and XP per day (or per month). New `GET /api/rewards/history` (`rewards.history`, test added)
- **Settings page** (`#/settings`): name and daily goal, spoken answers, opening animation (the intro now respects it), and what is installed on this computer (from `/api/health`). Both pages are in the sidebar
- **Voice ripple and wave** in the tutor chat: the mic spreads a circle with a ring across the panel and shows the live waveform (`VoiceRipple.tsx`), same as the Android app
- **Android app** (`Unlox_Hackethon/`, native Kotlin + Compose, all on the phone): the 8 Figma screens plus study tools, homework, rewards, settings and a profile with name and age; back stack with transitions; the web intro on every launch; book-only launcher icon. Real engines: Gemma (LiteRT-

LM) plans lessons and tutors, Piper voices (sherpa-onnx 1.13.8) narrate, Whisper + Vosk (language-ID approach from NDialRecorder) hear, ML Kit translates and reads photos. Per-stage progress with translating and narrating in parallel

Known Issues

- Android: PDF as a lesson source, MP4 export and the MiniLM search are not built yet; Tamil and Kannada narration needs the phone's own voice installed
- The running backend must be restarted to serve `/api/rewards/history`

[v0.12.0] — 2026-10-27

What's New

- **Voice and tutor detect the language**: speak or type in Hindi, Telugu, Tamil... and the answer comes back in that language, spoken in that voice. Speech: Whisper picks among our 9 languages (`stt._detect`, ~0.1 s on the GPU); typed text: by script (`languages.detect_text_language`). Ask in Hindi while watching a Marathi video and Devanagari is treated as Marathi
- **Latency**: models warm up in the background at start (translation, every voice, speech recognition, the language model); chat translation uses greedy decoding; Whisper no longer re-decodes at higher temperatures (Telugu 10 s -> 0.35 s) and ignores the English vocabulary hint for other languages (that hint was turning Hindi speech into English text). Warm tutor round trip: English ~2.5 s, Tamil 3-5 s, Hindi 5-6 s
- **Languages one by one**: the first language of a lesson is playable as soon as it is ready (a 30 s test: 13 s); the others are made in the background and appear as tabs the moment each is done (`pipeline.finish_languages`, `pending`/`failed` in lesson.json, the page polls). One failing language does not stop the rest
- Video content in the chosen language: every heading, point, label and caption is translated and the voice is that language (checked on a Hindi and a Telugu track)

Known Issues

- Speech recognition now defaults to Whisper **medium** (small is the fallback): Hindi words clearer ("खीजता" for "खींचता" instead of "खीछता"), still 0.5-0.9 s per question, GPU memory 6.5 of 8.15 GB with the language model, translation and voices loaded. Telugu and Tamil words are still rough on the synthetic test voices; a real voice is untested. Scanned-PDF OCR (6.8 GB) next to it was not re-tested
- Hindi tutor answers still take 5-6 s (translation + a longer voice clip)

[v0.11.0] — 2026-10-27

What's New

- **Intro splash on every page load** (`web/src/shell/Intro.tsx`, logo drawing in `logoPaths.ts`, from Niharika's `intro-animation/`); click or Esc skips it; reduced-motion skips it

- **Wordmark in the top-left corner**: the logo mark is gone, the sidebar shows only the name in the splash lettering (`Wordmark.tsx`); icon-only sidebar shows its first letter
- **Voice agent faster**: Whisper now runs on the GPU (cuBLAS/cuDNN 9 DLLs live in torch's `lib` folder, added to the DLL path in `stt.py`, CPU fallback kept); recordings decode in-process (WAV with the standard library, webm/opus with PyAV) instead of starting ffmpeg. A short question: 3.4 s + 1.4 s decode -> 0.25 s

[v0.10.0] — 2026-10-27

What's New

- **Photos**: a plus button in the prompt box attaches up to 4 photos; llava reads them on this computer (`backend/app/vision.py`, `POST /api/images`) and the lesson uses what they show
- **Modes**: Explanation, Study, Homework pills in the settings card; Study adds key terms, a memory trick and a recall quiz, Homework teaches only the concepts needed and never gives answers inside the video
- **Homework answers**: choosing Homework with a PDF or photo also writes a Markdown answer file question by question (`backend/app/homework.py`), viewed in the browser while it grows (`#/homework/<id>`) and downloadable as .md. Nothing guesses whether a file is homework
- **"Auto" video count**: the AI lists the concepts first (`backend/app/concepts.py`, rules plus worked examples, capped at 8) and makes one video per concept
- **Parallel long videos**: parts of a 2-4 minute video are planned two at a time from the outline (`longplan.py`); 2:30 test: plan 120 s -> 66 s, length 211 s -> 156 s
- **Lesson page**: notes are the left column, the sidebar collapses to icons while a lesson is open
- **Mind map redone**: two-sided SVG tree with curved coloured branches (replaces Mermaid); click a box for its notes and an inline "Explain this" answer from the tutor
- Fixes: page no longer scrolls down at every scene (notes and chat scrolled with scrollIntoView); Hindi falls back GPU -> CPU -> Ollama before English, English text is never read with a Hindi voice; backend log at `media/backend.log`; steps strip above the video redesigned

Known Issues

- Photo reading needs llava:7b in Ollama (it shares the GPU with the language model, so the first photo takes a while)
- Code scenes can still appear in non-code topics
- The new Home photo/mode UI, mind map and lesson layout were type-checked but not looked at in a browser (browser tests were stopped because the laptop hung)

Future Scope

- Progressive videos that start playing before later parts exist

---

[v0.9.0] — 2026-10-27

What's New

- **Project renamed Chanakya** (README, PRD, API title, web title and package, Learnverse stays the name of the metaverse mode)
- **Frontend rebuilt in Priya's grey glass language**: tokens and primitives in `web/src/styles/`, design-system parts in `web/src/ds/` (Glyph, DotNumber, Ring, Stat), shell, pages (Home, Library, Rewards, Lesson), Remotion video theme ported to the same palette. Old `app.css`, `screens.css` and screens removed
- **Home**: a big prompt box is the hero; settings (videos 1-8, length 0:30-4:00, level, languages, PDF) sit in a card below. Scrolling shrinks the box into a pinned header, its topic chips slide away behind it (`--p` driven by scroll, `pages/Home.tsx`, `styles/home.css`); the dock height is measured so the chips never overlap the settings card
- **Sidebar = navigation + awards**: streak with week, score and level, daily goal, latest badges, recent lessons, system status
- **Custom scrollbars** that fade out when nothing scrolls (`shell/useScrollbars.ts`, `styles/scroll.css`)
- **Rewards**: streaks (shields, 20 XP to count a day), XP, levels, 13 badges, daily goals; backend `rewards.py` (13 tests), frontend context with XP/level toasts and a badge modal; wired to lessons, quiz, flashcards, tutor questions, study tools
- **Videos up to 4 minutes**: over 1:30 a lesson is planned in parts (`backend/app/longplan.py`: outline, then each ~40 s part knows what came before and what comes next, parts joined into one lesson); API accepts 30-240 s
- **Floating "Creating…" pill** (`shell/CreatingPill.tsx`): leaving the progress screen no longer loses the lesson; a dot globe pill sits bottom right on every page, turns "Created" and pops to 120% when done, and opens it on click. The result is no longer thrown away when you navigate
- **Faster local model**: no constrained decoding (planner 34 s -> 9.8 s), lenient schema repair, one context size, Ollama flash attention + q8 KV cache + 2 parallel slots
- Progress of a lesson being made shows only in the side card, never inside the player; stat pills show number and label together in one pill; Library cards share one size
- Unlimited-OCR (6.7 GB) downloaded and verified on a scanned page (~10-12 s/page)

Known Issues

- A 2:30 test video came out at 3:31 before the scene cap and word floor were tightened; needs one more measurement on the next long build
- Long videos can still pick odd scene types (code scenes in a non-code topic); the director does not catch them yet
- Gemma 4 needs a newer Ollama than 0.23.1; Qwen2.5-7B stays the model (Llama 3.2 3B was slower and weaker)

Future Scope

- Progressive videos: start playing segment 1 while later parts are still being written (`docs/plan-long-videos.md`)
- Metaverse points as a `metaverse_points` reward event

[v0.8.0] — 2026-10-27

What's New

- **Merged everything the team pushed** (origin/main `60f5253`): Dhiraj's PDF upload + OCR (Tesseract / Unlimited-OCR) + hybrid RAG (e5 embeddings, FTS5 + FAISS, bge reranker), offline library, NLLB translation, GPU scheduler, sources shown on lessons; Niharika's `metaverse/` (3D quiz world) and `Niharika/` voice-assistant plan; Dhiraj's `Dhiraj/` snapshot (kept as pushed; not used by the app)
- One pipeline for both: `build_lesson(concept, languages, level, duration_s, documents, series, focus)` keeps level/length, series (courses), director and writer agents and now also grounds the plan in the PDF/library (`plan_lesson` returns plan + sources). Page-range lessons ("chapter 4") work from a PDF with an empty topic. Playlists accept a PDF too
- Frontend: Home uses Dhiraj's `PdfUpload` (progress + OCR status + focus), lesson notes list Sources, "Metaverse quiz world" button on Home and "Quiz in the metaverse" on the lesson screen (lazy-loaded 3D chunk)
- Tutor chat grounded with the same retrieval (`knowledge.retrieve`) instead of the earlier stub (`app/rag.py` removed: it collided with Dhiraj's `app/rag/` package)

- **Voice assistant** (from Niharika's plan, see `docs/voice-assistant.md`): `POST /api/stt` (faster-whisper small, CPU, offline), spoken tutor replies (`speak` flag -> `audio_url`), mic + speaker buttons in the chat panel. Hands-free mode and LTX demo clips are not built (LTX not realistic on 8 GB)

- **Metaverse entrance** (`web/src/components/TunnelTransition.tsx`, `MetaverseStart.tsx`): click the button and the page blurs while a tunnel of blocks assembles block by block around you; you walk down it, accelerating to light speed (~2 s, streaks); the bright light at the far end grows until it floods the screen; then the metaverse appears behind a **Start** page; Start reveals the topic screen. Canvas 2D, no extra dependency, skipped for `prefers-reduced-motion`; the 3D chunk preloads at the click

- **Fast OCR** (`backend/app/ocr/tesseract.py`, `ingest.py`): Tesseract 5.4 installed and the 9 language packs (tessdata_fast) in `models/tessdata`. Workers open the PDF and render their own pages as grayscale pixels (no PNG round trip), one Tesseract per core with all math libraries pinned to one thread (this cut a worker from ~670 MB to ~80 MB and fixed "paging file too small" crashes at 10+ workers), blank pages skipped, no batch barriers, worker count capped by free memory (`OCR_WORKERS` to override), Latin 200 dpi / Indic 300 dpi. The student's focus pages (e.g. "chapter 4") are read first and a section is indexed as soon as its last page is read, so a lesson on that chapter can start while the rest of the book is still being read. Raw OCR: ~7 pages/s on 22 workers (1,000 dense pages in ~140 s). Unlimited-OCR still upgrades pages in the background (priority: contents, focus, worst-confidence)

- `backend/scripts/bench_tesseract.py [pages] [workers] [--ingest]` builds a synthetic scan and times it

- **Study tools** (flashcards, quiz, mind map, study guide, FAQ, glossary, timeline): built by a background queue as soon as a video exists (`studio.py`; click = jump the queue; English raw copy cached, other languages translated from it), progress from the model's token stream. Buttons in the chat panel: ready opens instantly; working buttons get a flowing multi-colour border and a steady percentage

(`useSteadyProgress`: a clock keeps it moving, real progress can only pull it ahead). Mind map = Mermaid text parsed/re-emitted by the backend and drawn on-device. Flashcards redesigned (question front, View answer, 3D flip, slide-away rating, round review)

- **Meta quiz** (`metaquiz.py`, `POST /api/metaquiz`): one metaverse world from every study tool and every video of a course; stations carry extra reading, gates ask up to 5 questions with explanations (metaverse engine extended: `Zone.more`, `info.extra`, `Quiz.why`)
- **PDF notes in 30-page windows** (`ocr/digest.py`): written while OCR runs; 100-page scan: first notes at ~22 s, all at ~32 s; whole-PDF lessons plan from the notes; upload cap 1,000 pages; pass-1 embeddings moved to the idle GPU (OCR+index 1,002 pages: 341 s -> 171 s)
- **Metaverse entrance made smooth**: the lag was the 3D world initialising on the main thread during the animation (up to 0.9-1.6 s freeze); now state is isolated (`MetaverseLayer`), the world starts behind a still white frame and the light clears when it has painted and frames are calm, no CSS blur / backdrop-filter, precomputed tunnel geometry, adaptive canvas resolution, chunk preloaded on hover/idle

Known Issues

- Studio background builds queue behind each other (one local model): a video's 7 tools take about 2 minutes; opening a lesson queues its tools, and a click goes first
- `transformers` in `backend/venv` is 4.46 (Dhiraj pins 4.57.1 for Unlimited-OCR); scanned-PDF OCR with Unlimited-OCR needs its own ~6.7 GB model and that pin, digital PDFs work without
- Whisper small is weak on Telugu/Tamil/Kannada speech; English and Hindi are the supported voice-input languages for the demo
- Mic needs localhost or HTTPS

[v0.7.0] — 2026-10-27

What's New

- **Fixed the repeated "bulb" scenes.** Root cause: `normalize()` demoted any scene with too few points to a plain `title` card, so a graph lesson became 4 identical lightbulb cards. Now points are rebuilt from the narration and the originally wanted template is remembered
- **Visual director agent** (`backend/app/director.py`): finds weak scenes (title mid-lesson, same type twice in a row, bullets on a maths/code topic) and redesigns each with a focused call: graphs for maths, code for programming, diagrams/flows for processes. Visual fields only; narration kept
- **Coordinate geometry**: graph scenes can plot points without a curve, draw segments between points (`link` mark), show quadrants I-IV, auto-fit the plane to the points
- **Playlists / courses** (`backend/app/playlist.py`): curriculum agent outlines 5-8 episodes, each built as a series-aware lesson (links to the previous episode, teases the next, no repeats); playlist file saved after every episode so playback starts on episode 1 while the rest are made; episodes build 2 at a time (voice of one overlaps planning of the next, episode 2 waits until episode 1 is past planning); one failed episode does not sink the course; `POST /api/playlists`, `GET /api/playlists[/{id}]`, `POST /api/playlists/{id}/export` (renders every episode and joins them into one MP4). Playlists and exports get their own job workers

- **Course UI**: Format switch on Home (One video / Course of 5-8), episode rail under the video, auto-advance when a video ends (waits if the next is still being made), deep links `#c-<id>[-<episode>]`, "whole course MP4" button
- **Multimodal visual QA** (`backend/scripts/visual_qa.py`): renders-frames-to-vision-model check. moondream (1.7B) gave unusable answers; llava:7b works: it described the old lesson's scenes 1-4 as the same "question mark and light bulb" graphic and flagged them

Known Issues

- Planner still sometimes puts a quiz early; title/summary/quiz ordering is not enforced
- llava:7b QA needs the planner model unloaded first (8 GB VRAM) and is a script, not part of the pipeline

---

[v0.6.0] — 2026-10-27

What's New

- **Real translation working**: IndicTrans2 (en->hi/te/ta/mr/...) running locally from `models/indictrans2` (set `INDICTRANS2_MODEL` to that folder, `HF_HUB_OFFLINE=1`)
- **Voices verified by round-trip** (speak a known sentence, transcribe with Whisper): hi, ta, kn, en good; te, ml, gu correct sounds; mr was garbled on Piper -> now Indic-TTS (0.77 match); bn unverified (Whisper medium cannot read Bengali). `TTS_PREFER_INDIC=bn,mr`
- **Multi-agent pipeline** (all local, all data-only): Planner (Qwen2.5-7B, schema JSON) -> Reviewer (ast check + repair) -> Writer (per-scene free-text narration when the lesson is too short) -> Translator (IndicTrans2) -> Narrators (one thread per language) -> Tutor (chat). Lesson size is set from the slowest chosen language (`WORDS_PER_SEC`) so every track stays inside the requested length
- Per-language TTS locks: all languages record at once
- Planner model benchmark (see below)

Planner model benchmark (RTX 5060 8 GB, 3 prompts, JSON-schema planner)

Model	avg s/lesson	valid	notes
qwen2.5:7b-instruct-q4_K_M	34	3/3	picks graph for maths, code for programming; writes short lessons (writer agent fixes)
qwen2.5:3b-instruct-q4_K_M	24-33	2/2 run	missed graph scenes on the circle prompt
qwen3:4b	128	3/3	right length but thinking tokens make it 4x slower; removed
qwen3.5:9b, llama3.2:3b	n/a	n/a	not measured (C: disk and RAM pressure)

Known Issues

- C: drive was nearly full (0.1 GB): HF cache and Ollama models live there. New downloads go to D: (`models/`)
- Indic Parler-TTS not installed (3.7 GB, not needed). Bengali voice quality unverified

- Whisper-medium check is a proxy: it cannot judge accent or naturalness; a human ear is still needed

---

[v0.5.0] — 2026-10-27

What's New

- **Lesson screen = notes | video | tutor chat.** Left: scene-by-scene notes from the lesson (follow playback, click to jump, code/equations included, "My notes" box, copy/download .md). Right: tutor chat that knows the whole lesson and the scene being watched (`POST /api/lessons/{id}/chat`; English plan -> answer -> translated to the active language)
- **Progress is inside the video area** (no separate screen) and continuous: planner now streams tokens for real progress, per-scene voice progress, plus a smoothing hook so the bar never sits still or jumps backwards
- **English is an ordinary option** (no longer forced); at least one language required; lesson stores the English plan for chat
- **Reviewer agent** (`backend/app/review.py`): `ast.parse` check of Python snippets + one small LLM repair call; notes returned in `lesson.review` and shown in the UI
- Removed the separate Creating screen

Known Issues / findings

- Root cause of "Hindi speaks English with a Hindi accent": `TRANSLATE_BACKEND=none` in a local `.env` -> English text read by a Hindi voice. UI now warns when a non-English track was not translated
- Ollama (Qwen) translation is poor (Telugu leaked Chinese characters); IndicTrans2 needed. Hugging Face downloads stall on the Xet path: use `HF_HUB_DISABLE_XET=1`. Network ~0.5-1.3 MB/s
- Indic Parler-TTS (3.7 GB) not downloaded: not needed now that ta/kn/gu have CPU Indic-TTS voices and bn has a Piper voice

---

[v0.4.0] — 2026-10-27

What's New

- Merged teammate's frontend (status panel, history, remote backend)
- **Screen-by-screen UI** (`web/src/screens/`): Home (topic box, PDF upload area, level, length 30/60/90 s, languages, recent lessons, system status) -> Creating (live progress, steps) -> Lesson (player)
- **Player screen**: custom controls: seek bar with scene ticks, tap or drag to seek, hover preview (time + scene), time readout, play/pause, +-5 s, previous/next scene, volume, captions toggle, settings (language, speed 0.75-1.5x, captions, download quality 720p/1080p), full screen; double-tap left/right third of the video = -/+5 s; keyboard (space, arrows, M, C, F); language switch keeps your position; scene list tap-to-jump; MP4 + SRT download
- `POST /api/lessons` now takes `level`, `duration_s` (30-90) and `documents` (ids); prompt adapts to level and length

- `backend/app/rag.py`: single hook `retrieve_context(concept, document_ids) -> str` for the RAG/OCR service; UI uploads to `POST /api/documents` (multipart `file` -> `{id, name, pages?}`) and shows "document service not connected yet" until it exists
- Hash routing fixed (Back/forward and pasted `#id` links open the lesson)

Known Issues

- Document upload has no backend yet (friend's RAG/OCR service); `documents` ids are accepted but ignored
- Dev-only React warning from Remotion's AbsoluteFill in the Player (pre-existing, harmless)
- Chrome extension was not reachable; UI verified with headless Chrome (screenshots in `output/ui/`)

Ideas evaluated

- **LTX-2.5 (Lightricks) generative video**: not adopted for the core pipeline. 22B params (int8 21.5 GB + text encoder 15.4 GB, NVFP4 18.7 GB) vs this laptop's 8 GB VRAM / 15 GB RAM; far beyond the 8-16 GB CPU target; cannot render exact Indic text or follow our audio-sets-length rule; gated + community licence. Possible only as an optional, pre-generated b-roll step on a GPU machine

---

[v0.3.0] — 2026-10-27

What's New

- **Graph scenes** (`web/src/remotion/graph.tsx`): axes, grid, ticks, curves that draw themselves (line, parabola, cubic, sine, cosine, exp, abs, sqrt, reciprocal, circle, ellipse), circle marks (centre, radius, diameter, chord, tangent), points, formula card, legend. LLM supplies enum + numbers only; nothing is evaluated
- **Code scenes** (`web/src/remotion/code.tsx`): dark editor, offline syntax colouring (Python, Rust, JS/JSX, Java, C, Go, SQL), line highlight following the narration, per-line notes, output console. Snippets are display text, never run
- Planner routes by subject (maths -> >=2 graph scenes, programming -> >=2 code scenes) with worked examples; `normalize()` fills missing labels/highlights; keyword-routed offline samples for `LLM_BACKEND=mock`
- Export height option: `POST /api/lessons/{id}/export {"lang","height":720|1080}`, default 720. Measured 1080p 38.5 s vs 720p 21.3 s for a 40 s lesson, so 720 is default; the UI can offer 1080p as a boost
- CPU voice engine `indic` (AI4Bharat Indic-TTS ONNX: en, ta, kn, gu; RTF ~0.4) ahead of the GPU-only Parler fallback; Bengali now uses its Piper voice
- GPU+CPU: Ollama layer offload (`OLLAMA_NUM_GPU`, auto = as many as fit; 100% on RTX 5060), ONNX CUDA provider when onnxruntime-gpu is present, torch CUDA installed, `/api/health` shows `gpu.ollama_loaded[].gpu_percent`; `DEVICE=cpu` forces CPU everywhere
- Backend venv, web deps, headless Chrome, Piper voices and Noto fonts installed locally

Known Issues

- IndicTrans2 and Indic Parler-TTS are gated on HuggingFace: account must accept each model's terms before download (translation to Indic languages untested until then)
- 7B model sometimes writes slightly invalid snippets (e.g. missing closing tag); no syntax check yet (idea: `ast.parse` for Python)
- Indic voices (ta/kn/gu ONNX repack) and Indian-accent English not yet judged by ear

---

[v0.2.0] — 2026-10-27

What's New

- Merged teammate's `origin/main` (5127b43): FastAPI backend (Ollama planner, IndicTrans2, Piper/Parler TTS, captions, jobs), Remotion `web/` frontend, 8 templates
- Removed our Revideo scaffold and draft `schema/` (superseded by `backend/app/schema.py` and Remotion); scaffold kept outside repo
- License note: project is hackathon-only, not published; license columns kept in README for the submission rule

Known Issues

- Teammate's design uses Indic Parler-TTS on GPU for ta/kn/bn/gu; spec says CPU-only. Needs CPU path (our Indic-TTS ONNX voices) or accept GPU risk
- Renders at 1920x1080; spec asks 720p (RENDER setting to check)

[v0.1.0] — 2026-10-27

What's New

- Project docs (prd, problem_statement, pitch, issues, progress, README)
- Draft shared lesson contract: `schema/lesson.schema.json` + `schema/sample-lesson.json` (en + hi)
- `frontend/`: Revideo 0.11.0 starter scaffolded, deps installed
- Local assets downloaded (git-ignored, under `models/` and `fonts/`):
- Piper voices: hi, te, mr, bn, ml, en_GB-alan
- AI4Bharat Indic-TTS int8 ONNX: en, ta, kn, gu
- Noto Sans fonts (OFL): Devanagari, Telugu, Bengali, Malayalam, Tamil, Kannada, Gujarati, Gurmukhi, Oriya
- Ollama: qwen2.5:7b-instruct-q4_K_M, qwen2.5:3b-instruct-q4_K_M (plus pre-existing qwen3.5:9b)
- Python venv `.venv` with piper-tts, onnxruntime, huggingface_hub (`hf` CLI)

References reviewed

- `inspiration/` (TheoremExplainAgent, MIT): ideas only, no code reuse (needs cloud LLMs via litellm, LLM writes Manim code, English-only). Borrowed: multi-stage plan (outline -> per-scene detail -> narration), audio-duration-drives-scene pattern, learning-objective prompting. `inspiration/` is git-

ignored.

Known Issues

- No voice has been run/timed yet; ta/kn/gu/en repack quality and Indian-accent English unverified
- IndicTrans2 is gated on HuggingFace; needs `hf auth login` + terms acceptance (user action)
- Backend not started here (teammate owns it)

Future Scope

- Revideo spike: sample lesson -> MP4 with Hindi text + audio, offline
- Scene components: TitleCard, Diagram, CodeWalkthrough driven by lesson JSON
- Player with language switch; captions (SRT + burned-in)
- Voice smoke test + CPU timing per language
- Optional: pa/or/as via official Indic-TTS zips; IndicTrans2 translation

---

Git History

Commit	Date	Message
d2d2546	2026-10-29	v0.16.0: Flash and Pro modes, memory of existing lessons, fact checker, technical topics shown with code and network scenes, reference notes, retries and readiness checks, full README
d9ed0c9	2026-10-28	v0.15.2: coherent long videos and courses (max 3 parts/videos, no repeats), topic cleaning, expressive narration, non-blocking photo, fixes found by live lessons
342b893	2026-10-28	v0.15.0: prompt tightened, PDF upload progress, Android continuous generation and lag fixes, Learnverse graphics, docs
2dacea8	2026-10-28	v0.14.4: PDF Markdown pipeline, font checks, llava for PDF pictures, caching and pre-warming
e7f578e	2026-10-28	v0.14.3: ask the chat for another video, every course episode streams
ea3b050	2026-10-27	v0.14.2: 5 video course no longer hangs the laptop, GPU errors fall back to CPU, long videos and episode 1 stream, autoplay fixed
fd03bb2	2026-10-27	v0.14.1: Learnverse AI worlds with a building screen, gates open by themselves, jump, saved points and purchases, home plus button aligned
543e8b5	2026-10-27	v0.14.0: scenes play while they are made, per-stage progress, parallel study tools, profile, spaced repetition, weak spots, exports, offline lesson file, papers, teach-back, resumable jobs
2c9fb16	2026-10-27	progress.md: git history for v0.13.0
cddcb2a	2026-10-27	Android app (native Kotlin + Compose, fully on-device): Figma screens, Gemma tutor and lesson planner, Piper voices, Whisper and Vosk speech, ML Kit translation and OCR, study tools, homework, rewards, profile with name and age
5f07b0c	2026-10-27	v0.13.0 web: Progress and Settings pages, rewards history endpoint, voice ripple and waveform in the tutor chat, opening animation switch

Commit	Date	Message
2f926c5	2026-10-27	Chanakya v0.9.0: grey-glass frontend, rewards, scroll-shrinking prompt home, videos up to 4 min, floating Creating pill
1cb5b0d	2026-10-27	Project setup: docs, lesson schema, Revideo frontend scaffold, fonts
3d9d818 .. 123513d	2026-10-27	Orchestration doc, graph/code scenes, merge of teammate frontend, screen-by-screen UI
d7527c7	2026-10-27	Notes panel, tutor chat, progress
4b0466c	2026-10-27	IndicTrans2, voices, writer/reviewer agents, pacing
86a5fed	2026-10-27	Director agent, point-plotting graphs, playlists backend + UI, vision QA script
9ae15b7	2026-10-27	Merge origin/main (PDF/OCR/RAG, metaverse); voice, study tools, Meta quiz, fast OCR, tunnel entrance

Issues & Fixes Log

> AI: Read this file before debugging. If the same issue occurred before, the fix is already here.

[ISSUE-001] — 2026-10-27

Title: Piper voice downloads truncated / missing `.onnx.json` **Symptom:** After parallel `curl -O` of several voices, `.onnx` files were smaller than catalog size and most `.onnx.json` were absent **Root Cause:** Backgrounded `curl` with two `-O` URLs per job in a shell loop; jobs raced/were cut off **Fix:** Download via a Python script that checks each file against `size_bytes` from `voices.json` (`urllib`), re-fetching mismatches **Files Changed:** `models/piper/*` **Status:** Resolved

[ISSUE-002] — 2026-10-27

Title: Windows Python cannot open `/c/Users/...` path from Git Bash **Symptom:** `FileNotFoundError` reading scratchpad JSON **Root Cause:** POSIX-style path passed to native Windows Python **Fix:** Use a Windows path (`C:\Users\...`) in Python code **Files Changed:** none **Status:** Resolved

[ISSUE-003] — 2026-10-27

Title: `npm init @revideo@latest` is interactive-only **Symptom:** Stalls at prompts when stdin is piped all at once; creates `my-revideo-project` **Root Cause:** `prompts` library needs newlines delivered over time; no CLI flags exist **Fix:** Pipe empty lines with delays: `( for i in 1..8; do sleep 4; echo; done ) | npx @revideo/create@latest`, then rename folder to `frontend` **Files Changed:** `frontend/` **Status:** Resolved

[ISSUE-004] — 2026-10-27

Title: PLAN.md license claim for sarvam-translate wrong **Symptom:** PLAN.md says Apache 2.0 **Root Cause:** Unverified; HF license tag is GPL-3.0 and the model is a Gemma-3-4B fine-tune **Fix:** Do not use sarvam-translate; use IndicTrans2 distilled 200M (MIT) **Files Changed:** none (PLAN.md left as-is by request) **Status:** Resolved

[ISSUE-005] — 2026-10-27

Title: Two license problems found in the stack **Symptom:** README claimed Piper = MIT and listed Qwen2.5-3B as usable **Root Cause:** `pip install piper-tts` (1.8.0) is the GPL-3.0-or-later successor (OHF-Voice/piper1-gpl, uses espeak-ng); Qwen2.5-3B ships under the Qwen Research License **Fix:** README licenses table corrected; planner fallback switched to qwen3:4b (verify Apache-2.0); 3B model not used. GPL decision for our repo is open (see progress.md) **Files Changed:** README.md, .env.example **Status:** Ongoing (repo license choice pending)

[ISSUE-006] — 2026-10-27

Title: Backend crashed on start after editing planner prompt **Symptom:** `ValueError: Invalid format specifier '"circle","a":0,...'` importing `app.planner` **Root Cause:** `SYSTEM_PROMPT` is an f-string; raw JSON braces in the added examples were parsed as format fields **Fix:** Double the braces (`{{ }}`) in prompt text that contains JSON **Files Changed:** backend/app/planner.py **Status:** Resolved

[ISSUE-007] — 2026-10-27

Title: Stale processes: duplicate model downloads and self-killing restart **Symptom:** `.part` file locked (WinError 32); a PowerShell restart command exited 255; old uvicorn kept serving mock lessons **Root Cause:** Two `download_models.py` ran at once; a kill command filtered on `uvicorn app.main` also matched its own command line **Fix:** Kill by listening port (Get-NetTCPConnection -LocalPort 8000) and by process name; run downloads once **Files Changed:** none **Status:** Resolved

[ISSUE-008] — 2026-10-27

Title: Code line cut mid-statement on screen **Symptom:** React snippet ended `getElementById('root'` **Root Cause:** `normalize()` truncated lines to 58 columns **Fix:** Cap raised to 72; prompt asks for lines under 45 characters and to split long statements **Files Changed:** backend/app/schema.py, backend/app/planner.py **Status:** Resolved (monitor)

[ISSUE-009] — 2026-10-27

Title: Hindi/Telugu tracks spoke English **Symptom:** Hindi voice reading English text **Root Cause:** `TRANSLATE_BACKEND=none` in backend/.env (set for English-only testing); IndicTrans2 not downloaded **Fix:** Translation switched on; UI warns when a non-English track is untranslated; IndicTrans2 download completed with `HF_HUB_DISABLE_XET=1` **Files Changed:** backend/.env (local), web/src/screens/LessonScreen.tsx **Status:** Resolved once IndicTrans2 is installed

[ISSUE-010] — 2026-10-27

Title: `hf download` stalls at 0 bytes for gated models **Symptom:** `.incomplete` blobs stay 0 bytes forever **Root Cause:** Xet download path stalls on this network **Fix:** `HF_HUB_DISABLE_XET=1` and a retrying `snapshot_download` script **Files Changed:** none **Status:** Resolved

[ISSUE-011] — 2026-10-27

Title: Merge: `app/rag.py` stub shadowed by Dhiraj's `app/rag/` package **Symptom:** After merging origin/main, `from .rag import retrieve_context` would fail (package wins over module of the same name) **Root Cause:** Both sides used the name `rag` (my one-file hook for the friend's service, Dhiraj's real package) **Fix:** Removed `rag.py`; pipeline/planner/chat use `knowledge.retrieve()` and `knowledge.pages_passages()`; uploads are `POST /api/uploads` (sha) instead of the placeholder

`/api/documents`; frontend uses `PdfUpload`, the placeholder `UploadArea` and `uploadDocument` were deleted **Files Changed:** backend/app/{rag.py (deleted), pipeline.py, planner.py, chat.py, main.py}, web/src/{App.tsx, api.ts, screens/Home.tsx, components/UploadArea.tsx (deleted)} **Status:** Resolved

[ISSUE-012] — 2026-10-27

Title: `/api/stt` returned 500: `open() got an unexpected keyword argument 'metadata_errors'` **Symptom:** faster-whisper 1.2.1 crashed decoding any clip **Root Cause:** it calls PyAV with an argument that PyAV 19 no longer has (faster-whisper expects av 14-15) **Fix:** `stt.py` decodes with ffmpeg (already required for export) to 16 kHz mono float32 and passes the array to Whisper, so PyAV is not used **Files Changed:** backend/app/stt.py **Status:** Resolved

[ISSUE-013] — 2026-10-27

Title: PDF ingest failed: `models/multilingual-e5-small not found` **Symptom:** upload job errors in 4 s on a fresh machine **Root Cause:** Dhiraj's RAG expects the e5 embedder and bge reranker under `models/` (not downloaded by `download_models.py`) **Fix:** downloaded `intfloat/multilingual-e5-small` and `BAAI/bge-reranker-v2-m3` into `models/` with `HF_HUB_DISABLE_XET=1` **Files Changed:** none (models are gitignored) **Status:** Resolved on this machine; teammates need the same download

[ISSUE-014] — 2026-10-27

Title: Metaverse entrance laggy at the white flash and at the Start page **Symptom:** frame freezes right when the tunnel ends and again as the Start button appears **Root Cause:** the 3D world initialised on the main thread in the middle of the animation (measured 0.9-1.6 s long tasks), App re-rendered the whole tree at that moment, and the Start page used `backdrop-filter: blur` over a live WebGL canvas **Fix:** `MetaverseLayer` owns all entrance state (no App re-render); the world is mounted only once the screen is a still white frame and the light clears after it has painted and frames are calm; blur/backdrop-filter removed (opacity/transform only); tunnel geometry and colours precomputed, no per-edge strokes, adaptive canvas scale; chunk preloaded on hover and when idle **Files Changed:** web/src/components/{TunnelTransition,MetaverseLayer,MetaverseStart}.tsx, web/src/App.tsx, web/src/screens.css **Status:** Resolved (tunnel phase has no frame over 50 ms; remaining start-up work is hidden behind the still frame)

[ISSUE-015] — 2026-10-27

Title: OCR with many workers crashed ("paging file is too small"), and ingest ran at half speed **Symptom:** more than ~10 Tesseract workers died with BrokenProcessPool; a full upload took 2x the raw OCR time **Root Cause:** each worker imported numpy/OpenBLAS, which reserved ~670 MB per worker; e5 embedding ran on the CPU while every core was busy with Tesseract **Fix:** pin OpenBLAS/MKL/OMP to one thread in workers (80 MB per worker), cap workers by free memory, embed on the idle GPU during pass 1 **Files Changed:** backend/app/ocr/tesseract.py, ingest.py, rag/embed.py **Status:** Resolved (5.9 pages/s end to end)

[ISSUE-016] — 2026-10-27

Title: Study-tool progress border drew 0% and the number stalled then jumped **Symptom:** the border arc never showed progress; the percentage sat at 15% then jumped to 30% **Root Cause:** `@property --p` was registered with `inherits: false`, so the pseudo-element never saw it; backend progress arrives in bursts (silence while the model reads the lesson, then tokens) **Fix:** `inherits: true`; border is now a constant flowing gradient; the displayed percentage comes from `useSteadyProgress` (a clock that always moves; real progress can only pull it ahead, and it glides) **Files Changed:** web/src/studio/{studio.css,useSteadyProgress.ts,StudioButton.tsx} **Status:** Resolved (measured 6% to 60% in 12 s, biggest step 1%)

[ISSUE-017] — 2026-10-27

Title: Lesson lost when the user left the progress screen; blank pages after rewriting a file **Symptom:** navigating to another page while a video was being made dropped the result and there was no way back; separately every page went blank with "does not provide an export named 'Stat'" **Root Cause:** navigation bumped a run counter that made the finished job's result be ignored; Vite had cached `Stat.tsx` as empty because it read the file while a shell heredoc was rewriting it **Fix:** a `creation` record in `App.tsx` outlives navigation, shows the floating `CreatingPill` anywhere except on its own progress screen, and only opens the lesson itself if the user is still watching; touching the file made Vite re-read it (check with `curl http://127.0.0.1:5173/src/<file>`) **Files Changed:** web/src/App.tsx, web/src/shell/CreatingPill.tsx, web/src/styles/home.css, web/src/ds/Stat.tsx **Status:** Resolved

[ISSUE-018] — 2026-10-27

Title: Schema repair dropped real scenes; long plans ran 40% over length **Symptom:** "Lesson plan has no usable scenes" from the director; a 150 s request produced 211 s **Root Cause:** the lenient scene validator also dropped valid `ScenePlan` objects built in code; the writer agent floored every scene at 22 words and parts allowed 5 scenes each **Fix:** the validator keeps `ScenePlan` instances (`tests/test_schema_lenient.py`); the floor is 14 words and a part has at most 4 scenes **Files Changed:** backend/app/schema.py, review.py, planner.py **Status:** Resolved (length to be re-measured)

[ISSUE-019] — 2026-10-27

Title: Android: every Piper voice produced silence **Symptom:** lessons made on the phone had empty narration files (44-byte .wav), no sound in the player; the voice test said "no sound" **Root Cause:** sherpa-onnx 2.0.x removed its built-in espeak phonemizer ("--vits-data-dir is deprecated and ignored"), so text never became phonemes. The first dependency setup had picked 2.0.1 **Fix:** use sherpa-onnx 1.13.8 (the build the NDialRecorder uses) and drop the separate onnxruntime-android (1.13.8 ships its own libonnxruntime.so); added a Settings > Test the voice row and a trace file (`files/trace.log`) so native failures leave a trail **Files Changed:** Unlox_Hackethon/app/build.gradle.kts, app/libs, data/Engines.kt, data/Trace.kt, ui/screens/More.kt **Status:** Resolved

[ISSUE-020] — 2026-10-27

Title: Android: app exits at the Telugu voice (and would at Marathi and Bengali) **Symptom:** the app vanished mid-lesson with no Java crash; `dumpsys activity exit-info` said EXIT_SELF status 255 **Root Cause:** the tokens.txt files made for the three raw Piper voices had Windows (CRLF) line endings; the native token reader calls exit(-1) on a malformed line. Marathi and Bengali also have multi-character tokens it rejects **Fix:** write tokens.txt with LF endings and drop the 5 multi-character tokens; a guard (`Trace.guard/dead`) skips a native step that killed the app on an earlier run, reset on every new install **Files Changed:** models/tts (not in git), data/Trace.kt, data/Engines.kt **Status:** Resolved

[ISSUE-021] — 2026-10-27

Title: adb push to /sdcard silently copied nothing from Git Bash **Symptom:** `adb push dir /sdcard/...` ran for minutes, no files appeared, then std::bad_alloc **Root Cause:** MSYS path conversion rewrote `/sdcard/...` to `C:/Program Files/Git/sdcard/...` **Fix:** `export MSYS_NO_PATHCONV=1` before any adb call with a device path; push one folder at a time **Files Changed:** none (workflow) **Status:** Resolved

[ISSUE-022] — 2026-10-27

Title: Android progress dial stuck and jumped **Symptom:** the Generating dial sat at 10% while Gemma planned, jumped to 45%, and showed 99% after a failure **Root Cause:** the pipeline only reported at stage boundaries; the dial showed that directly **Fix:** every stage has its own real percentage (the plan stage streams Gemma's output); translating and narrating run in parallel; the dial uses the website's steady-progress idea (a clock that may run only 4% ahead of real work) **Files Changed:** data/Creator.kt, data/Engines.kt, ui/screens/CreateAndGenerating.kt, ui/UnloxApp.kt **Status:** Resolved

[ISSUE-023] — 2026-10-27

Title: The player waited for the whole lesson (web and phone) **Symptom:** nothing could be watched until planning, translation and narration of every scene had finished (25-45 s); the "make the first scene, then keep adding" idea in `docs/plan-long-videos.md` had never been built, only languages one by one **Root Cause:** the plan was parsed only when complete, and `lesson.json` was written once at the end **Fix:** read the planner's JSON as it streams (`streaming.extract_scenes`), make and publish each finished scene (`ScenePreview`), poll the lesson while `building`, wait at the end of what exists, then swap in the finished lesson. Same on the phone (`StreamScenes.kt`). Scene 1 now plays after about 6 s **Files Changed:** backend/app/streaming.py, pipeline.py, planner.py, llm.py, jobs.py, web/src/App.tsx, pages/LessonPage.tsx, Android data/StreamScenes.kt, data/Creator.kt, ui/UnloxApp.kt **Status:** Resolved (phone build awaits an on-device test)

[ISSUE-024] — 2026-10-27

Title: Study tools built one after another **Symptom:** flashcards, quiz, mind map and the rest appeared one at a time **Root Cause:** one worker thread drained the queue **Fix:** `STUDIO_WORKERS` (default 3) builder threads; the English version of a tool is written once under a per-key lock **Files Changed:** backend/app/studio.py, backend/tests/test_studio_queue.py **Status:** Resolved

---

[ISSUE-025] — 2026-10-27

Title: Teach-back scored off-topic text 60/100 **Symptom:** a sentence about football covered several lesson points **Root Cause:** nomic-embed-text with the search prefix gives unrelated texts cosine 0.5-0.67, above the first threshold **Fix:** no prefix, threshold 0.62, chosen from measurements on a real lesson (off-topic max 0.56, paraphrase 0.75) **Files Changed:** backend/app/teachback.py **Status:** Resolved

---

[ISSUE-026] — 2026-10-27

Title: PDF tests fail in the full suite, pass alone **Symptom:** test_ingest, test_tesseract, test_triage and test_routes_rag raise MemoryError when the whole suite runs **Root Cause:** the laptop had about 2 GB RAM free (the backend python process holds 4.8 GB), not a code fault **Fix:** none needed; run those files alone (27 passed) or stop the backend first **Files Changed:** none **Status:** Ongoing (environment)

[ISSUE-027] — 2026-10-27

Title: Learnverse purchases and points gone the next time **Symptom:** after buying a ride or hat the shop looked empty on the next visit and the points were back to the start **Root Cause:** everything lived only in the browser's `localStorage`, which belongs to one address; opening the app from another address (localhost vs 127.0.0.1 vs a network IP, another dev port) or clearing site data starts from nothing **Fix:** keep a second copy on the backend (`media/metaverse_save.json`, newer `rev` wins, a stale tab cannot overwrite); load it when the world starts **Files Changed:** metaverse/src/engine.ts, web/src/components/MetaverseLayer.tsx, backend/app/metaworld.py, backend/app/main.py **Status:** Resolved (verified by clearing localStorage: points restored from the server)

[ISSUE-028] — 2026-10-27

Title: "+" button and Create button in different corners of the home prompt **Symptom:** the add-photo "+" floated at the top right of the box while Create lesson sat at the bottom right; collapsed header was two rows tall **Root Cause:** `align-items: flex-start` plus a margin-top on the "+", `align-self: flex-end` on Create; and `.dock-box textarea { flex: 1 }` inside the new column wrapper `.dock-field` made flex-basis 0, so the `height` rule was ignored **Fix:** one `.dock-actions` cluster (both 46 px, bottom aligned), `flex: none` on the textarea, a wrap rule under 560 px **Files Changed:** web/src/pages/Home.tsx, web/src/styles/home.css **Status:** Resolved

[ISSUE-029] — 2026-10-27

Title: 5-video course failed on every episode with "CUDA error: out of memory" **Symptom:** the course (Pac-Man, intermediate, 120 s, English + Hindi) ran 384 s and ended with status error: all five episodes failed with the same AcceleratorError, nothing to play **Root Cause:** the 8 GB card was full (language model + translator + voices). A CUDA out-of-memory raised as AcceleratorError is sticky: torch cannot use the GPU again in that process. `translate._translate_with_fallbacks` caught it, then called `_free_gpu()` which called `torch.cuda.empty_cache()`, which raised the same error and escaped, so the CPU and Ollama fallbacks never ran; every later episode hit the same dead GPU **Fix:** `config.mark_cuda_broken()` (torch models use the CPU from then on), `_free_gpu` swallows everything, the translator only starts on the GPU when at least 1.8 GB is free (`TRANSLATE_MIN_FREE_GB`), a CUDA-failed episode is retried once. Tests in `tests/test_gpu_resilience.py` **Files Changed:** backend/app/config.py, translate.py, playlist.py, tests/test_gpu_resilience.py **Status:** Resolved (restart the backend after a CUDA error to get the GPU back)

[ISSUE-030] — 2026-10-27

Title: Long videos and courses showed nothing until everything was done **Symptom:** a 2 minute video (planned in parts) and every course episode were saved only when complete; the earlier "first scene in 6 s" worked only for single lessons of 90 s or less **Root Cause:** `_build` started the scene preview only for `series is None and duration_s <= LONG_MIN` ; part planning (`plan_long`) and course episodes never fed it **Fix:** parts of 30 s; the preview streams part 1 and receives parts 2.. whole and in order (`ScenePreview.add_scenes`); episode 1 of a course is built with the preview and the course page opens it as soon as its `lesson_id` exists **Files Changed:** backend/app/longplan.py, pipeline.py, streaming.py, playlist.py, web/src/App.tsx, api.ts, components/EpisodeRail.tsx **Status:** Resolved (episodes 2+ of a course are not streamed)

[ISSUE-032]

Title: Hindi of a new video stuck for minutes behind the study tools **Symptom:** the English video was ready, Hindi stayed pending for more than 9 minutes (GPU at 84%, 7.8 of 8 GB used) while flashcards, quiz and other study tools appeared **Root Cause:** `studio._next` held background tools back only while a `lesson` or `playlist` job was active, not while the `langs` job (the other languages) was; the tools and the translation shared the card **Fix:** background study tools also wait while a `langs` job is active **Files Changed:** backend/app/studio.py **Status:** Resolved (measured: Hindi 74 s for 15 scenes)

[ISSUE-031] — 2026-10-27

Title: Generated videos did not start playing **Symptom:** a finished lesson or episode opened paused, or at a point in the middle **Root Cause:** `LessonPage` kept `resume` (position and "was playing") in a ref after a language switch and never cleared it or tied it to a lesson, so every later lesson reused it **Fix:** `resume` carries

the lesson id and is used only for that lesson; muted start when the page has had no user activation **Files Changed:** web/src/pages/LessonPage.tsx **Status:** Resolved

---

[ISSUE-033] — 2026-10-27

Title: Generating a 5 video course made the computer hang **Symptom:** the desktop stuttered and froze while a course was made, then episodes failed with CUDA out of memory **Root Cause:** measured: while translating, PyTorch's GPU allocator reserved 10.3 GB (only 1 GB in use) in the backend process, more than the whole 8 GB card, so Windows pushed the rest into system RAM (the backend's committed memory reached 22 GB, free RAM 0.2 GB) and froze the desktop. Also: the 8 GB card was full all the time: start-up warm-up loaded the translator (about 2 GB), Whisper medium (about 1.5 GB) and every Piper voice with CUDA next to the language model (5.5 GB); Windows then spills into system memory, which freezes the display. On top of that each finished episode queued 14 study tools (2 languages x 7), three at once, against the same model, and torch and OpenMP used every core **Fix:** `PYTORCH_CUDA_ALLOC_CONF=garbage_collection_threshold:0.6,max_split_size_mb:128`, a cap on torch's share of the card (`TORCH_VRAM_FRACTION`, 0.35), the allocator cache handed back after every translation batch (reserved memory 10.3 GB to 0.41 GB, process memory 14 GB to 4.4 GB in the same test), plus `resources.py` (below-normal priority, headroom gate, idle release), Piper on the CPU, Whisper lazy and VRAM-aware, no automatic study tools for course episodes 2+, 2 study builders, thread caps. See docs/presentation-mode.md **Files Changed:** backend/app/resources.py, config.py, stt.py, tts.py, translate.py, studio.py, playlist.py, pipeline.py, main.py **Status:** Resolved (see the measured numbers in progress.md v0.14.2)

[ISSUE-034] — 2026-10-28

Title: Digital PDF pages with little text were OCRed, and fonts that map to gibberish were trusted **Symptom:** a table page, a figure page with a short caption and a Hindi page, all with a perfect text layer, were treated as scanned and read by OCR (88 to 90 % accurate, columns mixed up); a PDF in an old Indian-language font was indexed as Latin gibberish **Root Cause:** `triage.classify_page` needed 200 characters to call a page digital and only checked the share of letters, which gibberish like `ekWuhVj dksf'kdk` passes **Fix:** 40 character limit and `markdown.looks_garbled` (cid marks, private-use characters, mixed-case or symbols inside words, Latin text in a document marked as an Indian language) **Files Changed:** backend/app/ocr/triage.py, markdown.py **Status:** Resolved

---

[ISSUE-035] — 2026-10-28

Title: Pictures inside an uploaded PDF were ignored **Symptom:** a lesson from a PDF could not use its diagrams; llava was only called for photos uploaded on their own **Root Cause:** ingestion read text (text layer or OCR) only **Fix:** `ocr/figures.py` finds embedded pictures, `ingest._describe_figures` describes them with llava in the background and indexes the description with the page; page records are updated through `update_page` (one lock, fresh copy) so the OCR upgrade and the figure pass cannot overwrite each other **Files Changed:** backend/app/ocr/figures.py, ingest.py, types.py, vision.py **Status:** Resolved (vector drawings are not covered)

[ISSUE-036] — 2026-10-28

Title: First scene came after 3 s sometimes and 6 s other times **Symptom:** the first video of a session started fast, the next one noticeably later **Root Cause:** every call sent the same 1,700-token system prompt that covers maths graphs and code scenes, plus up to 4,000 characters of reference text; the read time of that prompt (0.7 to 0.95 s, more when the model's prompt cache missed) and the length of the answer's first scene varied between topics **Fix:** a prompt built for the subject (790 tokens for a general topic), smaller reference block for short lessons. Measured 3.2 s (2.1 to 4.0) to 1.2 s (0.9 to 1.3) **Files Changed:** backend/app/planner.py, llm.py, tests/test_planner_prompt.py **Status:** Resolved

---

[ISSUE-037] — 2026-10-28

Title: PDF upload bar was slow, stuck and went backwards **Symptom:** the bar sat at 2 % while pages were read, jumped, went back to 2 % when the upload ended and reading began, and stopped during indexing **Root Cause:** no progress reports in the page-reading and indexing loops; Markdown conversion analysed vector graphics on every page (385 ms); the browser showed upload and reading progress as two separate bars from zero; the bar had no easing **Fix:** per-page and per-section reports in bands, graphics analysis only for pages with rules, one continuous eased bar. A no-scanned-pages PDF reported OCR at 80 % and then indexing from 30 %: fixed (test) **Files Changed:** backend/app/ocr/ingest.py, markdown.py, web/src/components/PdfUpload.tsx, api.ts, tests/test_upload_progress.py **Status:** Resolved

---

[ISSUE-038] — 2026-10-28

Title: The Android app lagged while a lesson was being made **Symptom:** stutter on every screen during generation **Root Cause:** `Creator.stage()` published a new state for every streamed token and the root composable rebuilt the whole app each time; `ScenePreview.feed` parsed the whole JSON for every token; the lesson polled while building was reassigned every 1.2 s even when unchanged; the voice engine ran with debug logging **Fix:** publish at most every 120 ms (or on a visible change), parse only after a closing brace, assign only on change, TTS debug off and 3 threads **Files Changed:** data/Creator.kt, StreamScenes.kt, Engines.kt, ui/UnloxApp.kt **Status:** Resolved in code; to be measured on the phone

---

[ISSUE-039] — 2026-10-28

Title: Attaching a photo on the home page made the user wait before they could create **Symptom:** the Create button stayed disabled while "Reading..." showed on the photo, for as long as llava took **Root Cause:** the page needed the finished vision job to know the photo's id, and Home disabled Create while any photo was reading **Fix:** `POST /api/images` returns `image_id` immediately; Create is enabled for a photo being read; `vision.context/topics` read an unread photo themselves (under a per-photo lock) inside the lesson, playlist or homework job **Files Changed:** backend/app/vision.py, main.py, pipeline.py, web/src/photos.ts, App.tsx, pages/Home.tsx, tests/test_photo_nonblocking.py **Status:** Resolved

---

[ISSUE-040] — 2026-10-28

Title: A long video or a course read like several unrelated mini-lessons with repeats and one-line captions **Symptom:** "teach me about dsa in java" with a photo: four 30 s parts each with its own "Let's learn about X." title card, narration of 4 to 22 words, the same heading and code block repeated, "Summary" and "Common mistakes" per part, empty title cards turned into code scenes **Root Cause:** every part was planned as a whole lesson, in parallel, without seeing the others; `schema.normalize` added a title card per part; nothing checked the joined plan for repeats or short narration **Fix:** at most 3 parts (40 s) planned one after the other, each told what was taught (`content.taught_summary`); first part opens, last part closes (`clean_part`, `normalize(segment, last, title)`); `dedupe_plan`, `quality_issue` (repeated headings/code, narration under 18 words), reviewer lengthens short captions; director drops empty mid-video title cards; courses capped at 3 videos with `taught` handed on **Files Changed:** backend/app/content.py (new), schema.py, planner.py, longplan.py, director.py, review.py, playlist.py, concepts.py, pipeline.py, main.py, tests/test_content_quality.py, test_longplan.py, test_homework_concepts.py; Android Creator.kt, CreateAndGenerating.kt; web Home.tsx **Status:** Resolved (verified on the real models; not yet on the phone)

[ISSUE-041] — 2026-10-28

Title: llava described a photo in a 104-line loop and the planner used all of it **Symptom:** the lesson was full of the photo's repeated lines and went off topic **Root Cause:** no repeat penalty or length cap on the vision model, no cleanup of its text, the whole text given to every episode **Fix:** `vision.clean_description` (dedupe lines, at most 15 lines / 900 characters, keeps the Topic line), `num_predict 380`, `repeat_penalty 1.2`, prompt says never repeat, `vision.relevant` picks the lines that fit an episode, old cached descriptions are cleaned on read **Files Changed:** backend/app/vision.py, pipeline.py, playlist.py **Status:** Resolved

---

[ISSUE-042] — 2026-10-28

Title: Titles and captions began with "Teach me ..." **Symptom:** typing "teach me about dsa in java" gave a lesson called "Teach me about dsa in java" **Root Cause:** the whole typed sentence was used as the topic **Fix:** `content.clean_topic` strips the request words (and fixes the case of DSA, Java, SQL and similar) in `create_lesson`, `create_playlist`, `pipeline.build_lesson`, `playlist.start`, and on Android in `Creator.make` (`Topic.kt`). Bare "Teach me" is kept whole; a result starting with a lower-case word is capitalised **Files Changed:** backend/app/content.py, main.py, pipeline.py, playlist.py, tests/test_content_quality.py; Android data/Topic.kt, Creator.kt **Status:** Resolved

---

[ISSUE-043] — 2026-10-28

Title: Found by generating real lessons: a heading "string", the same heading twice, scenes opening with the same sentence, short scenes left short **Symptom:** "teach me about the solar system": title card headed "string" and narrated "Let's explore string ..."; "how does the rainbow have 7 colors...": the heading "Sunlight

& Raindrops" twice and two scenes starting "Just like when sunlight passes through a glass prism"; MERN: a scene of 11 words after the writer pass **Root Cause:** the model copied the word "string" from the JSON schema; the heading check only dropped a repeat if the template or the words matched; narration similarity needed 70 %; the writer pass accepted a rewrite that was still under 18 words **Fix:** `schema.SCHEMA_WORDS` (heading falls back to the lesson title) and `content.schema_echo` (a retry with feedback); an identical heading is always dropped; scenes whose first six words are the same are dropped; `review` runs a second `only_short` rewrite for scenes still under 18 words **Files Changed:** backend/app/schema.py, content.py, planner.py, review.py, tests/test_content_quality.py **Status:** Resolved

---

[ISSUE-044] — 2026-10-28

Title: test_figure_captions_are_stored_on_the_page_and_found_by_search failed only in the full suite **Symptom:** `seen` was empty after `wait_for_upgrades(10)` **Root Cause:** two tests in test_knowledge_bridge.py ingested a PDF without `notes=False`, which started the real study-notes writer (it holds the GPU slot, the shared Ollama model); the figures thread waited behind it **Fix:** `notes=False` in those two tests; `wait_for_upgrades` is asserted, so a timeout fails clearly **Files Changed:** backend/tests/test_knowledge_bridge.py, tests/test_pdf_markdown_figures.py **Status:** Resolved

---

[ISSUE-045] — 2026-10-29

Title: "teach me what avl trees are": repeated, shallow video built on a "book of stories" comparison **Symptom:** a 2-episode course with six scenes each: "stories" instead of nodes, no definition of balance factor or rotations, no tree drawn, no code although the request said C, the same hook twice, narration of 18 to 34 words **Root Cause:** AVL trees were not in the knowledge base, so there was no source; DSA words were not recognised as a code topic; the `network` (boxes and arrows) template was never described to the model, so a tree could not be drawn; the prompt asked for an everyday example in every scene; nothing told the planner what the learner already had **Fix:** subjects (`planner.CODE_WORDS`, `NET_WORDS`), the network template documented only for topics that can use it, `language_of`, a retry when a technical lesson has no code/network/graph scene or leans on comparisons (`content.analogy_overload`), `content.strip_analogies`, `memory.py`, reference notes in `data/kb_extra/` (AVL, BSTs, heaps, sorting, graphs, MERN), Flash/Pro modes **Files Changed:** backend/app/planner.py, content.py, memory.py (new), longplan.py, pipeline.py, llm.py, selfcheck.py (new), main.py, playlist.py, tests/test_modes_memory.py; web Home.tsx, api.ts, App.tsx; data/kb_extra/* **Status:** Resolved for Flash and Pro (checked on the real model); the 7B can still pick a weak example

---

[ISSUE-046] — 2026-10-29

Title: A larger local model (qwen3.5:9b) is slower, not better, on the 8 GB card **Symptom:** `ollama ps` showed 72 % of the model on the GPU (6.3 of 8.8 GB, the rest on the CPU); 17 tokens/s against 43 for qwen2.5:7b (100 % on the GPU, 5.5 GB); the AVL plan came back as invalid JSON **Root Cause:** a 9.7B Q4 model plus a 6144-token context does not fit in 8 GB next to the translation model and voices **Fix:** nothing

adopted; Flash and Pro use the same model. A model is only worth adding if `ollama ps` shows 100 % on the GPU. Models whose name ends in `-cloud` or `:cloud` run on a remote server and are not allowed (offline rule) **Files Changed:** none (benchmark: scratch script) **Status:** Closed (decision)

[ISSUE-047] — 2026-10-29

Title: test_studio_queue tests fail in the full suite only while the machine is loaded **Symptom:** `test_prebuild_queues_everything_once` and `test_urgent_request_jumps_ahead_of_background` failed twice in a full run made right after a GPU benchmark; they pass alone and in three other full runs (343 passed) **Root Cause:** timing: they wait on background threads **Fix:** none yet; rerun when the laptop is idle **Files Changed:** none **Status:** Ongoing (flaky under load)

Design notes

Orchestration and the documents that follow

Orchestration: streaming pipeline (speed-first)

Idea: JSON writer and frame builder run **at the same time** and see each other's output. Implementation: one shared **lesson doc** (blackboard) + an **event stream**; workers subscribe and react. Not two chatting LLMs.

Why not two LLM agents

- CPU-only: two LLM calls share the same cores, so no real speedup, only contention.
- Frame building has fixed templates (title / diagram / code). Filling a template from JSON is deterministic code: instant, zero tokens, and keeps "LLM returns data, never code".
- So: **1 LLM writer** (+ optional tiny outliner) and **deterministic workers** for everything else.

Workers and events

Worker	Type	Listens to	Emits
Outliner (optional)	LLM, ~100-150 tokens: title + `[{id,type,name}]`	`request`	`outline`
Skeleton builder	code	`outline`	`scene_skeleton` (blank scene named, e.g. "s2 Diagram: Loop flow")
Scene writer	LLM, streamed JSON, incremental parse	`outline`, earlier `scene_ready` (context: headings, narration)	`scene_ready` per completed scene object
Frame filler	code	`scene_ready`, `audio_ready`	`scene_filled`: content into the skeleton, then duration
Translate+ TTS	IndicTrans2 / Piper / Indic-TTS	`scene_ready`	`audio_ready` per scene+language (duration_s)
Validator	code	`scene_ready`	`scene_invalid` -> swap in fallback scene, keep going
Exporter	Revideo render + ffmpeg	all scenes `audio_ready`	`done` (mp4, srt)

Shared context: every worker reads the lesson doc (heading names, scene order, ids). The writer sees the outline and previous scenes so narration doesn't repeat; the frame filler sees names and types as soon as they exist.

Timeline (what the learner sees)

1. t~0: request -> outline (seconds) -> blank named scenes appear in the player
2. scene 1 JSON completes -> its card fills in; its TTS starts while scene 2 is still being written
3. each `audio_ready` fixes that scene's length (audio + pause) -> captions timed
4. last scene ready -> render. Revideo can split a render across workers. Time-to-first-visual drops to seconds; total time ~ LLM time + last scene TTS + render, rather than the sum of all stages.

Rules

- Audio sets scene length. A scene is "playable" only after its `audio_ready`; until then it shows a skeleton.
- Any schema failure on one scene replaces just that scene with a fallback; whole-lesson fallback only if the outline fails.
- Pin CPU threads: LLM gets most cores, TTS 2, so they do not thrash.

Knobs to benchmark (do not assume)

- Single streamed call vs outline + per-scene calls (per-scene repeats prompt prefill; Ollama KV cache may hide it if the prefix is stable).
- `OLLAMA_NUM_PARALLEL` 1 vs 2 for outline + writer.
- 3B vs 7B planner: tokens/s and JSON validity on CPU-only.
- Piper vs Indic-TTS real-time factor per language.

Decision

Start with: single streamed LLM call + incremental JSON parse + per-scene TTS overlap + deterministic frame filler. Add the outliner only if time-to-first-visual matters in the demo.

Tools to adopt (scout report; licenses via `gh api`, re-verify before submission)

Need	Pick	License
Constrained JSON	Ollama `format` = JSON schema	MIT
Streaming scene parse	json_repair / partial-json-parser (Py), partial-json (JS, verify license)	MIT
Diagram layout	dagre	MIT
Code highlight	shiki (per-line tokens)	MIT
TTS runtime (optional)	sherpa-onnx; test Indic voices first	Apache-2.0
Word timing	ctc-forced-aligner (check Indic CTC model) or Piper phoneme durations	BSD-2
Fallback LLM serving	llama.cpp (GBNF, speculative decoding)	MIT
Excluded: aeneas (AGPL), MMS-TTS (NC), elkjs (EPL-2.0, unconfirmed), Qwen2.5-3B (research license).		

Measured on this laptop (2026-10-27, Ollama 0.23.1, schema-constrained `format`, ~110 tokens out)

Model	Device	Speed	JSON valid
qwen2.5:7b q4_K_M	RTX 5060 GPU	62.7 tok/s	yes
qwen2.5:7b q4_K_M	CPU only	failed to load: only 3.7 GB RAM free of 15.4	-
qwen2.5:3b q4_K_M	GPU	104.6 tok/s	yes
qwen2.5:3b q4_K_M	CPU only (Core Ultra 9 275HX)	11.2 tok/s	yes
A ~700-token lesson takes ~60 s at 11 tok/s on CPU (3B), ~11 s on GPU. Streaming scene-by-scene hides most of the CPU wait. Close Chrome etc. before CPU runs; on an 8 GB judge laptop use <=3B.			

Plan: 2.5-minute videos that build themselves while you watch

Status: draft for review. Nothing here is implemented. Written against commit `7587bbb`.

The goal

A lesson should be able to run **2:30**, but it should start playing in under 30 seconds and never feel like it is being made. Making 2.5 minutes in one go does not work: the 7B model loses the thread, one long answer is slow and fragile, nothing can be shown until the end, and a single failure loses everything. So the video is made in **segments of about 30 seconds**. While segment 1 plays, segment 2 is already being made, and so on. The player shows **2:30** from the first second.

The idea that makes this cheap

Playback in this app is not a video file. The browser draws the lesson from JSON (scenes, timings, audio clips) with Remotion. So "rendering while it plays" costs nothing: the only slow parts are **writing** the scenes (LLM), **translating** them and **recording the voice**. Adding more video is just appending more scenes to the JSON the player already reads. The MP4 is only made when someone presses Download.

1. Making the next part before it is needed

Storyboard first (about 6 s)

A storyboard agent writes the whole 2:30 outline as small JSON: 5 chapters, each with a goal, a role (hook, idea, example, practice, recap), a word budget, the visual types it should use (graph, code, flow...) and the key terms it introduces. This gives: - the **2:30 timer** (word budgets / speaking speed of the language), and chapter ticks on the seek bar from the start; - coherence: every segment knows what came before and what comes next; - the quiz and summary placed in the last chapter only, which fixes the "quiz appears early" problem.

Segment pipeline

Each segment goes through the same agents we already have, but with a small prompt: `plan (3–4 scenes) -> director -> reviewer/writer -> translate -> voice -> append`. The planner is told the storyboard row, a running summary of what was already said, the last scene of the previous segment and the next chapter title (the series mechanism built for courses already does this).

Staying ahead of playback

Measured today: a whole 6-7 scene lesson takes about 34 s to plan. A 3-4 scene segment is roughly half the tokens, so about 12-15 s of planning plus a few seconds of translation, with the voices recording on the CPU while the next segment is being planned. The target is **generation at least 1.5x faster than playback**.

How we get there: - **Short first segment (about 15-20 s):** a hook and the first idea, so playback starts about 25-30 s after the click. - Plan segment k+1 as soon as segment k has finished planning (do not wait for its voice). Planning uses the GPU, voices use the CPU. - Adaptive size: 20 s, then 30 s segments. - **Behind-schedule mode:** if the buffer gets thin, skip the optional agents (writer, director) for the next segment and use the cheap path; catch up later. - Start playing only when the buffer is safe: `ready seconds >= expected time to make the next segment x 1.5`, measured on this laptop. - A **benchmark gate** in milestone 1: if the sustained rate is under 1.3x real time, shrink prompts, use a faster reviewer, or raise the initial buffer before building any UI.

2. Joining the parts without a seam

One continuous timeline

Segments are not separate videos. The composition is one timeline of scenes; a segment is just a batch of scenes added at the end. The audio timeline is **narration-first**: voice clips exist (with exact durations) before their scenes are added, so the timeline never changes retroactively and the seek bar never jitters.

Morphing, not cutting

Transitions work per scene, so a segment boundary looks like any other scene change: - templates receive `previousScene` and `nextScene`, and animate **in from the previous scene's layout** (heading text slides into the new heading; bullet items re-flow to their new positions, the same technique as "FLIP" animations); - **persistent stage elements** survive between scenes that use the same component: a graph keeps its axes and adds the new curve, a code scene keeps its editor frame and appends lines, a diagram keeps its centre node. These are the reusable rendered components you asked for; - 10-12 frame overlap with an audio crossfade, and a short chapter card (about 0.6 s) at chapter starts that also hides small timing gaps; - the director agent sees the whole video's visual history, so no three identical templates in a row even across a boundary.

3. Nothing laggy

- **Never rebuild the player.** Scenes live in an external store; the player's `durationInFrames` grows in place, the instance keeps its identity (no remount, no key change). Past scenes are memoised.
- **Pre-mount the next scene** about 1 s before it starts (Remotion `premountFor`), so its first frame is already painted.
- **Append at scene boundaries, in idle time** (`requestIdleCallback`), O(1) work per arrival. Text-fitting measurements run in a worker.
- **Audio is decoded ahead** as soon as a segment lands (decode once, keep in memory) so a clip never waits on the network or decoder.
- **Compositor-only motion** (transform and opacity), the same rules that fixed the metaverse entrance.
- **Acceptance test with the profiler already built:** no frame over 50 ms while a segment is appended, no audio gap over 20 ms at a boundary, while a fake producer delivers segments at 1.2x, 1.0x and 0.8x playback speed.

4. What the viewer sees

- Time label: **0:00 / 2:30** from the start. It is the storyboard estimate and is corrected in steps of 5 s as real voice lengths arrive (never a visible jump near the end).
- The seek bar has three zones, like a streaming service: played, **ready**, and **planned** (striped, with the chapter titles on hover). Seeking into the planned zone goes to the end of the ready zone with a quiet "next part is being prepared" tooltip.
- If playback ever catches up with the buffer, it does **not** freeze: the current scene holds its last frame calmly and a small chapter card ("Next: Common mistakes") shows with a gentle pulse, and it resumes by itself when the next scene arrives.
- Other languages: the active language is built first, the others follow; their tabs show a building state like the study-tool buttons.

5. Data and API

- `lesson.json` gains `target_duration_s`, `storyboard`, `segments: [{id, status, scene_range, start_s, duration_s}]` and a `revision`. Tracks are append-only: `languages[lang].scenes` and `duration_s` only grow.
- `POST /api/lessons {duration_s: 150}` returns the stub (storyboard + segment 1) as soon as it exists; lessons of 90 s or less keep the current one-shot path.
- `GET /api/lessons/{id}?since=<revision>` returns only the new scenes; the page follows an event stream (SSE) or polls every second.
- Everything is saved after each segment, so a refresh or a closed tab loses nothing, and generation continues on the server.

6. Export

When the last segment lands the full timeline exists, so the MP4 is the usual Remotion render. To make Download instant, segment MP4s are rendered in the background as segments finish (the render slot is free while the model is not), then joined with ffmpeg with no re-encode, the same way the course export works.

Milestones

#	Deliverable	Done when
M 1	Storyboard agent, segment pipeline, append-only lesson JSON, delta API, benchmark	2:30 lesson built; first segment ready within 30 s; sustained generation rate at least 1.3x real time on this laptop
M 2	Live scene store, growing player, 2:30 label, three-zone seek bar, hold-and-resume	No frame over 50 ms when appending, at fake producer rates 0.8x to 1.2x
M 3	Morph transitions, persistent graph/code/diagram stage, premount, audio crossfade	Boundaries cannot be told from scene changes in a frame-by-frame check
M 4	Whole-video director, running-summary memory, role-based chapters	Rust, Python loops, circle: no repeated points, quiz only at the end, vision QA finds no duplicate-looking scenes

#	Deliverable	Done when
M 5	Background segment export, docs, cleanup	Download available within seconds of the last segment

Risks

Risk	Mitigation
Model slower than playback	short first segment, behind-schedule mode, benchmark gate, larger start buffer
GPU contention between planning, translation, study-tool queue	the study-tool queue already yields to lesson jobs; translation and voices already run off the planner's critical path
Story drifts over 5 segments	storyboard, running summary, series-style context, last-scene hand-off
A segment fails	retry once, then a simpler scene set from the storyboard row; the video never stops
Seeking and resuming at odd moments	the ready zone is the only seekable zone; time label is monotonic
Long audio memory	decode per segment, release segments far behind the playhead

Decisions I need from you

1. Is 2:30 the fixed target, or should the length picker offer 1:00 / 2:30 / 4:00 (the same mechanism scales)?
2. May the first segment be shorter (about 15-20 s) to start sooner?
3. Should courses (5-8 videos) also use this, or keep one-shot videos for courses?

Presenting without the laptop hanging

The laptop has 8 GB of graphics memory and 16 GB of RAM. The language model alone takes 5.5 GB of the card, and Windows draws the screen on the same card, so anything else that fills it makes the whole desktop stutter. The backend now protects itself (below-normal priority, half the cores, background work waits for free memory, idle models are released), but the apps around it matter too.

Before the demo (2 minutes)

1. Close what you do not need: Figma, Android Studio, extra Chrome windows and tabs, the Vite dev server. In a terminal in `Unlox_Hackethon`, run `gradlew --stop` (the Gradle and Kotlin daemons keep 1 to 3 GB).
2. Run the built app, not the dev server: build once with `cd web && npx vite build`, then open `http://localhost:8000` (the backend serves `web/dist`). Do not run `npm run dev` during the demo.
3. Start the backend and Ollama only (`backend/run.ps1`). Check `http://localhost:8000/api/health`: `resources` should show about 6 GB or more of free RAM and 6 GB or more of free VRAM before you start.
4. Warm it with one short lesson before you present (30 s, English): the language model is then loaded and the first real lesson starts at once.

During the demo

- Ask for what you will show, not more: a course of 3 videos of 1 minute is ready in a few minutes; 5 videos of 2 minutes takes about ten.
- Episode 1 plays while it is made; episode 2 and later appear as they finish. Start talking about the first one while the rest builds.
- Study tools (flashcards, quiz, mind map) are written when you open them; the first one takes about half a minute.
- Hindi and the other languages arrive after the English version. The page adds them by itself.

If something is slow

- `/api/health` shows `resources`. Under 1.5 GB of free RAM or 0.8 GB of VRAM, background work pauses on its own; closing a browser window or Figma frees it again.
- After a CUDA error the backend uses the CPU for the translator until it is restarted (slower but it does not fail). Restart the backend to get the GPU back.
- Lower the load further with environment variables before starting the backend: `STUDIO_WORKERS=1`, `PLAYLIST_WORKERS=1`, `STT_DEVICE=cpu`.

Voice assistant: what is built, what is not

Niharika's plan is in `Niharika/` (7 documents, no code). This page records how much of it was feasible on the demo laptop (RTX 5060 8 GB, CPU-capable) and what was built on top of the tutor chat that already existed.

Verdict per milestone

Plan milestone	Verdict	Status
M1 grounded text chat	Feasible, and it already existed: `POST /api/lessons/{id}/chat` knows the lesson, the scene on screen and (now) the uploaded PDF / library passages	Done
M2 talkback (spoken answers)	Feasible with the same Piper / Indic-TTS voices as the lessons, one clip per reply	Done (whole reply, not sentence streaming)
M3 push-to-talk	Feasible: faster-whisper `small`, int8, on the CPU, so the GPU stays free for the LLM	Done (tap to start, tap to send)
M4 hands-free + barge-in	Possible later (needs a voice-activity detector in the browser)	Not built
M5 demo clips: template animation	Possible (planner already returns scene data)	Not built
M5 demo clips: LTX video model	Not realistic on 8 GB VRAM while the LLM is resident	Not built, not planned

How it works

```
mic (browser, tap) -> POST /api/stt?lang=hi&lesson_id=...  (raw webm/wav body)
                      faster-whisper small, CPU, int8, lesson title/terms as a vocabulary hint
                   -> {text, lang, duration_s}            (empty text = "I did not catch that")
text -> POST /api/lessons/{id}/chat {speak: true}
        tutor answers in English -> IndicTrans2/NLLB -> TTS (lesson voice for that language)
     -> {reply, scene, audio_url}  -> played in the browser; text is shown too
```

- Everything is local. Audio is held in memory, written to a temp file for decoding, and deleted.
- Languages: Whisper is good for English and Hindi; Tamil, Telugu etc. are weaker (Niharika's `03-speech-to-text.md` suggests AI4Bharat IndicConformer as a later per-language backend behind `stt.py`). The UI shows what was heard so a wrong transcript is visible.
- Microphone needs `http://localhost` or HTTPS (browser rule). On a second device over the LAN the mic is blocked.
- Setup: `pip install faster-whisper`, then `python scripts/download_models.py --stt` (about 460 MB into `models/whisper-small`). Without the model the mic button still shows and returns a clear error; typing keeps working.
- Health: `/api/health` -> `stt`.

Metaverse

`metaverse/` (Niharika) is the quiz world. The web app opens it from Home ("Metaverse quiz world") and from a lesson ("Quiz in the metaverse"); the lesson's quiz scenes become gates in the world (`metaverse/src/lessonToWorld.ts`). Reusing the voice tutor inside the metaverse stations is a possible next step.

Offline RAG + Dual-Engine OCR for Chanakya — Design

Date: 2026-10-27 Status: approved in conversation, pending written-spec review

1. Goal

Ground every generated lesson in real source material, fully offline:

- A student types a concept, optionally uploads a PDF (100 to several thousand pages, digital or scanned).
- The lesson planner retrieves relevant passages (from the uploaded PDF first, then an offline library) and builds the lesson only from them. The final video carries knowledge from the PDF and cites pages.
- Works with Wi-Fi off. No external AI APIs. The LLM returns data (JSON), never code.

Non-goals: live internet search at demo time; redistributing copyrighted texts; training any model.

2. Hardware budget (this laptop)

RTX 5060 Laptop 8 GB VRAM, 31.5 GB RAM, 24 CPU threads, 189 GB free disk, Windows 11. Consequence: at most one large GPU model is resident at a time (see §8 GPU scheduler).

Runtime environment: a dedicated Python 3.12 venv (`backend/.venv`), torch built for CUDA 12.8+ (required by Blackwell / RTX 50-series). Python 3.14 is not used.

3. Offline library (built once, while online)

Downloaded today by resumable scripts; each source is normalised to JSONL records `{doc_id, source, title, lang, url, license, text}` and logged in `data/library/manifest.json` (source, date, count, sha256).

Source	Acquisition	Est. clean text
English Wikipedia — Vital Articles Level 5 (~50k)	HF dataset `wikimedia/wikipedia` (en), filter by the Level-5 title list, discard rest	~2.5 GB
Simple English Wikipedia (all)	same dataset, `simple` config	~0.8 GB
Indic Wikipedias: hi, te, ta, ml, mr, kn, bn, gu	same dataset, per-language configs	~4 GB

Source	Acquisition	Est. clean text
NCERT textbooks, classes 6–12, English + Hindi	official PDFs, ingested through the same PDF pipeline as uploads (§4)	~1.5 GB
OpenStax (CC-BY)	official PDFs, same PDF pipeline	~0.5 GB
archive.org books, public-domain / CC only	advancedsearch API filtered by `licenseurl` (publicdomain, creativecommons) and subject (science, mathematics, history, technology), items that have a `_djvu.txt` text derivative	capped ~5 GB

Quality filters: drop documents < 200 words, list/disambiguation pages, pages whose non-letter ratio > 40%, archive.org items whose dictionary-hit rate < 70% (bad scans).

Licensing: every chunk keeps `license` + `url`. NCERT is copyrighted (free to read); it is indexed locally for this prototype only and never redistributed.

4. PDF ingestion with dual-engine OCR

Applies to student uploads and to NCERT/OpenStax library PDFs.

4.1 Page triage (PyMuPDF, ~1 ms/page)

For each page compute: extractable text length, image-area ratio, and font/glyph sanity.

- **digital**: ≥ 200 extractable chars and text is sane → use the text layer directly. No OCR.
- **scanned**: otherwise → OCR (both engines, §4.2).
- Outline: `doc.get_toc()`; if empty, the TOC is recovered after OCR of the first ~15 pages (heading detection on OCR markdown).

4.2 Two-pass OCR, combined

Pass 1 — Tesseract (CPU, fast, everything). All scanned pages are rendered at 300 DPI and run through Tesseract 5 (LSTM) in a process pool (`cores − 4` workers), with the language set chosen from the requested lesson language plus English (e.g. `eng+hin`, `eng+tel`; script auto-detected by Tesseract OSD when unknown). Output per page: text + mean word confidence. Each page is chunked and indexed as soon as it finishes, so the whole document becomes searchable within minutes.

Pass 2 — Unlimited-OCR (GPU, high quality, priority order). `baidu/Unlimited-OCR` (3.3B MoE, ~0.6B active, BF16 6.7 GB, MIT) via transformers `infer_multi`, `image_size=1024`, batches of ~8 pages per call (stays under the 32k-token context), pages rendered at 200 DPI. Output is cleaned with the model card's `remove_det` post-processing (keeps headings, markdown tables, LaTeX equations). Priority queue: 1. TOC/front-matter pages; 2. pages in the range relevant to the student's request (from TOC match or explicit page numbers), and pages whose Tesseract chunks rank in the top retrieval results for the concept; 3. pages with the lowest Tesseract confidence; 4. everything else, in background, while the GPU is otherwise idle.

Per-page arbiter (combines the two). When Unlimited-OCR finishes a page, pick the final text:

- Reject Unlimited-OCR output if it shows a degenerate repetition loop (any 30-token n-gram repeated ≥ 3×), or its word count is < 0.5× or > 2× Tesseract's on a page where Tesseract confidence ≥ 80.
- Otherwise compute: script validity (share of characters in the expected Unicode block), lexicon hit rate (against the library vocabulary for that language), and character-level agreement (normalised edit similarity) between the two outputs.
- Default winner is Unlimited-OCR (better layout, tables, equations). Tesseract wins when Unlimited-OCR was rejected, or when its script validity / lexicon hit rate is ≥ 10 points worse than Tesseract's.
- Agreement < 0.6 marks the page `low_confidence`; such pages are down-weighted ×0.8 in retrieval.
- The winning text replaces the page's chunks in the index (delete-by-page, re-insert).
- Every decision is logged in the page record (`engine`, scores) for debugging and the demo.

Caching. Per-page results are stored under `data/uploads/<sha256>/pages/<n>.json` (`{text, engine, tess_conf, agreement, status}`). Re-uploading the same file costs nothing; an interrupted job resumes from the last finished page.

4.3 Limits and failures

- Max 2 GB, 5,000 pages per file. Encrypted/password PDFs are rejected with a clear message.
- Unlimited-OCR CUDA OOM → halve batch, retry; at batch 1 still failing → keep Tesseract text.
- A page failing both engines is marked `failed` and skipped; the job continues.
- Progress is reported through the existing job system: `"OCR 340 / 1,200 pages (Tesseract done, Unlimited-OCR 85)"`.

5. Chunking

- Structure-aware: split on chapter → section → paragraph (TOC + markdown headings), then pack to ~350 tokens with 50-token overlap (token counts from the e5 tokenizer).
- Tables and display equations are atomic; never split.
- Chunk record: `chunk_id, doc_id, title, section_path, page_start, page_end, lang, source, license, engine, low_confidence, text`.

6. Indexes

Two kinds of collection, same code:

- `library` — the §3 corpus (~6–8 M chunks), built once, read-only.
- `uploads/<sha256>` — one per uploaded PDF, built incrementally during ingestion.

Each collection = SQLite file (chunk table + **FTS5** BM25 index) + **FAISS** index of **multilingual-e5-small** embeddings (384-d). Library uses IVF-PQ (≈1–2 GB on disk); uploads use a flat inner-product index (small, exact, supports delete/re-insert by page).

Embedding: library embedded once on GPU (est. 2–3 h). Uploads embedded on CPU (e5-small, 24 threads, ~5k chunks per 1,000 pages ≈ 30 s) so the GPU stays free for OCR.

7. Retrieval service

Module `backend/app/rag/search.py` + endpoint `POST /api/search` `{query, scope: "library" | "upload:<sha>" | "both", k=8}` → `[{chunk_id, text, title, page_start, page_end, source, score}]`.

1. Query normalisation: if the query is in an Indic language, IndicTrans2 produces an English version; both are searched.
2. Candidates: FTS5 top-50 + FAISS top-50 per scope, merged by reciprocal rank fusion (k=60).
3. Rerank with **bge-reranker-v2-m3**; keep top-k.
4. Scope weighting for `both`: upload passages first; library contributes at most 2 passages (background / analogy only).
5. Targets: < 400 ms library, < 150 ms single upload (warm).

8. Lesson agents

Modes: - **Concept + PDF** → search within the upload (scope `both`). - **Chapter / page range** ("explain chapter 4", "pages 120–135") → resolve the range via TOC, take its chunks in reading order; if longer than the planner budget, map-reduce: Qwen condenses each section into short notes, then plans from the notes. - **PDF only** → topic = the dominant chapter in the TOC; overview lesson. - **Concept only** → scope `library`.

Pipeline: 1. **Retriever** (§7) → passages labelled [P1]…[P8]. 2. **Planner** (Qwen2.5-7B via Ollama, JSON schema): instructed to teach only what the passages support; `ScenePlan` gains `sources: list[str]` (passage labels). 3. **Fact-check**: each narration sentence is scored against its cited passages with the reranker; score below threshold → regenerate that scene once, then flag it. 4. Existing pipeline unchanged: translate → narration first → scene timing → visuals → MP4. 5. Summary scene shows a source line (e.g. "From: Biology Class 11, pp. 214–219"); the player lists all sources with page numbers. 6. If no passage clears the relevance threshold, the lesson says the PDF does not cover the concept and falls back to the library; it never invents content to fill the gap.

GPU scheduler (`backend/app/gpu.py`): a single lock that owns "the big slot". Unlimited-OCR (6.7 GB) and Qwen (4.7 GB, Ollama `keep_alive=0`) never overlap; OCR pass 2 yields the GPU whenever a lesson needs planning, then resumes. The reranker (~1.1 GB fp16) and TTS share the GPU otherwise.

9. Code layout

```
backend/app/
  gpu.py                # big-slot GPU scheduler
  rag/
    __init__.py
    chunker.py          # structure-aware chunking
    store.py            # SQLite+FTS5 and FAISS collection
    embed.py            # e5-small (CPU/GPU) wrapper
    rerank.py           # bge-reranker-v2-m3 wrapper
    search.py           # hybrid search + RRF + scope weighting
  ocr/
    __init__.py
    triage.py           # digital vs scanned page detection, TOC
    tesseract.py        # pass 1 (process pool)
    unlimited.py        # pass 2 (Unlimited-OCR)
    arbiter.py          # per-page combination
    ingest.py           # orchestration, caching, progress, incremental indexing
backend/scripts/
  bench_ocr.py          # throughput + quality benchmark (plan step 1)
  fetch_library.py      # §3 downloads
  build_library.py      # chunk + embed + index the library
backend/tests/          # pytest
data/                   # gitignored: library/, uploads/, models/
```

10. Testing

- Unit: triage thresholds, chunk boundaries (tables/equations atomic), arbiter decisions on crafted inputs (repetition loop, wrong script, agreement), RRF merge, scope weighting.
- Integration: ingest a 20-page mixed PDF (digital + scanned) end to end; search returns the expected page.
- Retrieval benchmark: 50 question → expected-page pairs across an English NCERT book, a Hindi NCERT book and a scanned archive.org book; report recall@8 per engine (Tesseract-only, Unlimited-only, combined).
- Benchmark first: pages/min for each OCR engine on this laptop and character accuracy on Hindi/Telugu pages. Decision gates: if Unlimited-OCR < 4 pages/min under Windows transformers, evaluate WSL2 + vLLM; if its Indic accuracy is below Tesseract's, the arbiter prefers Tesseract for that script.

11. Risks

Risk	Mitigation
Unlimited-OCR too slow on Windows	Tesseract pass makes every page searchable anyway; priority queue OCRs relevant pages first; WSL2 + vLLM option
Unlimited-OCR custom code incompatible with the 5060/torch build	pin versions from the model card; Tesseract-only mode still meets the requirement
8 GB VRAM contention	single big-slot scheduler; CPU embeddings for uploads
Library build time	run overnight; Simple-English + NCERT subset is buildable in hours as a fallback

Risk	Mitigation
Copyright	only PD/CC archive.org items; NCERT local-only; licences stored per chunk

Offline RAG + Dual-Engine OCR Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Let a student upload a PDF of up to thousands of pages (digital or scanned). It is OCR'd by Tesseract and Unlimited-OCR, combined per page, chunked and indexed. A large offline library is also indexed. Both feed the existing `knowledge.retrieve()` so the lesson is built from them.

Architecture: New packages `backend/app/ocr/` (triage, Tesseract pass, Unlimited-OCR pass, arbiter, ingestion) and `backend/app/rag/` (chunker, SQLite-FTS5 + FAISS collections, e5 embedder, bge reranker, hybrid search). These packages own their routes through an `APIRouter` in `app/routes_rag.py`. `knowledge.retrieve()` gains an optional `upload` argument and delegates to `rag.search` when new collections exist; the existing `data/kb` index keeps working unchanged.

Tech Stack: - Python 3.12 (`backend/venv`) - torch (CUDA 12.8 wheels), transformers==4.57.1, PyMuPDF, pytesseract + Tesseract 5 - faiss-cpu, sentence-transformers, rapidfuzz, datasets, pytest

Spec: `docs/superpowers/specs/2026-10-27-rag-ocr-design.md`

Global Constraints

- Runs with Wi-Fi off. All models and data live under `models/` and `data/`. Set `HF_HUB_OFFLINE=1` at runtime.
- Python 3.12 venv at `backend/venv`. Do not use Python 3.14.
- Unlimited-OCR requirements are pinned from its model card: torch 2.10 (cu128 build), torchvision 0.25, transformers==4.57.1, einops==0.8.2, addict==2.4.0, easydict==1.13, pymupdf==1.27.2.2.
- Upload limits: max 2 GB, max 5,000 pages. Encrypted PDFs are rejected.
- Chunks are ~350 tokens with 50-token overlap. Tables and display equations are never split.
- Embedder `intfloat/multilingual-e5-small` with prefixes `query:` / `passage:`. Reranker `BAAI/bge-reranker-v2-m3`.
- RRF k=60. Retrieval candidates: 50 keyword + 50 vector. Final k=8. In scope `both`, the library contributes ≤ 2 passages. Low-confidence pages are weighted ×0.8.
- Arbiter thresholds:
- reject Unlimited-OCR output if a 30-token n-gram repeats ≥ 3 times;
- reject it if its word count is < 0.5× or > 2× Tesseract's when Tesseract confidence ≥ 80;
- Tesseract wins when it is ≥ 0.10 better on script validity or on lexicon hit rate;
- agreement < 0.6 marks the page low_confidence.
- Only one big GPU model at a time: Unlimited-OCR (6.7 GB) and the Ollama LLM never overlap.
- Files owned by session `dhiraj-shetty-26`: `planner.py`, `pipeline.py`, `main.py`, `translate.py`, `web/`, `backend/.env`. Do not edit them. Task 12 hands the integration lines to that session.

- `knowledge.retrieve(concept)` stays backward compatible and still returns `list[Passage]`. `Passage(title, section, text, url, source, score)` is unchanged.

Review Focus

1. **A PDF whose pages are mostly images but carry a thin invisible text layer** (typical of scanned books with bad embedded OCR). Expect the page to be treated as scanned, not trusted as digital. → Task 2 test `test_garbage_text_layer_is_scanned`.
2. **A student re-uploads the same PDF, or the server restarts mid-OCR.** Expect no repeated work: OCR resumes from the cache. → Task 9 test `test_resume_uses_cache`.
3. **A lesson is requested while OCR pass 2 is holding the GPU.** Expect OCR to yield within one batch and the lesson not to run out of GPU memory. → Task 5 test `test_ocr_yields_to_urgent`.
4. **A Hindi or Telugu PDF searched with an English concept, or the reverse.** Expect cross-lingual hits through the multilingual embeddings. → Task 7 test `test_cross_script_query_still_hits_vector`.
5. **A concept the PDF does not cover.** Expect `[]` from the upload scope and library passages instead, never padding with irrelevant upload chunks. → Task 8 test `test_irrelevant_upload_returns_library_only`.

Spec deviation (deliberate): §7 step 1 (translating Indic queries with IndicTrans2 before search) is replaced by multilingual-e5 cross-lingual search plus FTS on the original script. `translate.py` belongs to the other session and is changing to NLLB; this avoids depending on it. Revisit only if the Task 13 benchmark shows poor cross-lingual recall.

Task 1: Environment, models, Tesseract, and the OCR benchmark (decision gate)

Files: - Create: `backend/requirements-rag.txt`, `backend/scripts/setup_rag.ps1`, `backend/scripts/bench_ocr.py`, `backend/tests/__init__.py`, `backend/tests/conftest.py`, `backend/pytest.ini` - Modify: `.gitignore` (add `models/tessdata/` is already covered by `models/`; no change if so)

Interfaces: - Produces: `app.ocr.paths.TESSDATA_DIR = config.MODELS_DIR / "tessdata"`, `app.ocr.paths.UNLIMITED_DIR = config.MODELS_DIR / "Unlimited-OCR"`, `app.rag.paths.LIBRARY_DIR = config.ROOT / "data" / "library"`, `app.rag.paths.UPLOADS_DIR = config.ROOT / "data" / "uploads"`. Each paths module only defines constants. - pytest markers: `gpu` (needs CUDA + Unlimited-OCR weights), `slow`. Default run: `-m "not gpu and not slow"`.

- [ ] **Step 1:** Write `requirements-rag.txt` with the Unlimited-OCR pins above plus `pytesseract`, `faiss-cpu`, `sentence-transformers`, `rapidfuzz`, `datasets`, `pyarrow`, `python-multipart`, `pytest`. Torch comes from `--index-url https://download.pytorch.org/whl/cu128`. Use torch 2.10 if that index has a build for it; otherwise use the newest torch there and record the version in the file.
- [ ] **Step 2:** Write `setup_rag.ps1`. It should:
- install the torch wheel, then the requirements file;
- `winget install UB-Mannheim.TesseractOCR`;
- download `eng hin tel tam mal mar kan ben guj` `.traineddata` from `tesseract-ocr/tessdata_best` into `models/tessdata`, along with `osd`;

- run `huggingface-cli download baidu/Unlimited-OCR --local-dir models/Unlimited-OCR --exclude "model.safetensors" "assets/*" "wheel/*" "*.pdf"`, which keeps only the indexed `model-00001-of-000001.safetensors`;
- download `intfloat/multilingual-e5-small` and `BAAI/bge-reranker-v2-m3` into `models/`.
- [ ] **Step 3:** Run it. Expected: `python -c "import torch;print(torch.cuda.is_available(), torch.cuda.get_device_name())"` prints `True NVIDIA GeForce RTX 5060 Laptop GPU`, and `tesseract --list-langs --tessdata-dir models/tessdata` lists all 9 languages plus osd.
- [ ] **Step 4:** Write `bench_ocr.py PDF [--pages N] [--lang hi]`. It renders N pages and runs Tesseract (pool, 300 DPI) and Unlimited-OCR (single-page `infer`, gundam config, 200 DPI). It prints pages/min for each engine and the GPU memory peak. With `--truth file.txt`, it also prints character accuracy (rapidfuzz ratio). Read `models/Unlimited-OCR/modeling_unlimitedocr.py` to find what `infer` / `infer_multi` return, and record whether `infer_multi` output can be split per page (look for a page separator token). Write the findings as a comment block at the top of `bench_ocr.py`.
- [ ] **Step 5:** Run it on 20 pages each of an English scanned book and a Hindi NCERT scan. **Decision gate:**
- **Unlimited-OCR < 4 pages/min:** note "WSL2+vLLM follow-up" in the comment block and continue (Tesseract still covers every page).
- **Its Hindi accuracy < Tesseract's:** set `UNLIMITED_SCRIPTS_DISABLED={"hi",...}` in `app/ocr/paths.py`.
- **`infer_multi` splits per page:** set `UNLIMITED_BATCH = 8`; otherwise set `UNLIMITED_BATCH = 1`.
- [ ] **Step 6:** Commit: `git add backend/requirements-rag.txt backend/scripts/setup_rag.ps1 backend/scripts/bench_ocr.py backend/tests backend/pytest.ini backend/app/ocr/paths.py backend/app/rag/paths.py backend/app/ocr/__init__.py backend/app/rag/__init__.py` then `git commit -m "rag/ocr: environment, model setup and OCR benchmark"`.

Task 2: Page triage and TOC

Files: Create `backend/app/ocr/types.py`, `backend/app/ocr/triage.py`, `backend/tests/test_triage.py`, `backend/tests/pdfs.py` (test PDF factory).

Interfaces: - Produces in `types.py`: - `EngineResult(engine: str, text: str, confidence: float | None)`, where Tesseract confidence is on a 0–100 scale - `TocEntry(level: int, title: str, page: int)` (1-based page) - `PageRecord(page: int, kind: Literal["digital","scanned"], text: str, engine: str, tess_conf: float|None, agreement: float|None, low_confidence: bool, status: Literal["pending","tesseract","final","failed"])` with `to_json()` / `from_json()` - Produces in `triage.py`: - `classify_page(page: fitz.Page) -> Literal["digital","scanned"]` - `read_toc(doc: fitz.Document) -> list[TocEntry]` - `render_png(page: fitz.Page, dpi: int) -> bytes` - Produces in `tests/pdfs.py`: `make_pdf(tmp_path, pages: list[str | Literal["image"] | tuple[Literal["image"], str]]) -> Path` - a `str` makes a text page; - `"image"` makes a page that is a rendered picture of lorem text; - `("image", junk)` makes a picture page with an invisible text layer holding `junk`.

- [ ] **Step 1: Write failing tests**

```
def test_text_page_is_digital(tmp_path): assert kinds(make_pdf(tmp_path, ["Photosynthesis " *
40])) == ["digital"]
def test_image_page_is_scanned(tmp_path): assert kinds(make_pdf(tmp_path, ["image"])) ==
["scanned"]
def test_garbage_text_layer_is_scanned(tmp_path):
    assert kinds(make_pdf(tmp_path, [("image", "~#@!%^ ¬¬ ¦¦ " * 30)])) == ["scanned"]
def test_toc_is_one_based(tmp_path):
    p = make_pdf(tmp_path, ["a "*200, "b "*200]); set_toc(p, [[1, "Ch 1", 1], [1, "Ch 2", 2]])
    assert read_toc(fitz.open(p)) == [TocEntry(1, "Ch 1", 1), TocEntry(1, "Ch 2", 2)]
```

- [ ] **Step 2:** Run `venv/Scripts/python -m pytest tests/test_triage.py -v`. Expected: FAIL (ImportError).
- [ ] **Step 3:** Implement. A page is digital iff its stripped text has ≥ 200 characters **and** letters make up ≥ 0.6 of its non-space characters (`str.isalpha`, which counts Indic letters). The image-area ratio only breaks ties: an image covering > 0.8 of the page with < 600 characters of text is scanned.
- [ ] **Step 4:** Run the tests again. Expected: 4 passed.
- [ ] **Step 5:** Commit: `git commit -m "ocr: page triage and TOC"`.

Task 3: Tesseract pass

Files: Create `backend/app/ocr/tesseract.py`, `backend/tests/test_tesseract.py`.

Interfaces: - Consumes: `EngineResult`, `paths.TESSDATA_DIR`. - Produces: - `tess_langs(lang: str | None) -> str`: en→`eng`, hi→`eng+hin`, te→`eng+tel`, ta→`eng+tam`, ml→`eng+mal`, mr→`eng+mar`, kn→`eng+kan`, bn→`eng+ben`, gu→`eng+guj`, None→`eng` - `ocr_image(png: bytes, langs: str) -> EngineResult` with engine `"tesseract"`; confidence is the mean of word confidences > -1 - `ocr_many(items: list[tuple[int, bytes]], langs: str, workers: int) -> Iterator[tuple[int, EngineResult]]`, yielding in completion order

- [ ] **Step 1: Failing tests**:
- `test_tess_langs_maps_codes`: assert the table above.
- `test_ocr_reads_rendered_text`: render "The mitochondria is the powerhouse" as an image page and expect `"mitochondria" in result.text.lower()` and `result.confidence > 60`.
- `test_ocr_many_returns_every_page`: 4 pages with workers=2, and the set of returned indices == {0,1,2,3}.
- [ ] **Step 2:** Run. Expected: FAIL.
- [ ] **Step 3:** Implement with `pytesseract.image_to_data(..., config=f'--tessdata-dir "{TESSDATA_DIR}" --oem 1 --psm 3', output_type=DICT)`. Use a `ProcessPoolExecutor` with an initializer that sets `pytesseract.pytesseract.tesseract_cmd` from `shutil.which("tesseract")`, falling back to `C:/Program Files/Tesseract-OCR/tesseract.exe`. Default workers = `max(1, os.cpu_count() - 4)`.
- [ ] **Step 4:** Run. Expected: 3 passed.
- [ ] **Step 5:** Commit: `git commit -m "ocr: Tesseract pass"`.

Task 4: Unlimited-OCR pass

Files: Create `backend/app/ocr/unlimited.py`, `backend/tests/test_unlimited.py`.

Interfaces: - Consumes: `EngineResult`, `paths.UNLIMITED_DIR`, `paths.UNLIMITED_BATCH`. - Produces: - `remove_det(raw: str) -> str`: copied verbatim from the model card's OmniDocBench post-processing - `class UnlimitedOCR`: - `.load() -> None` - `.unload() -> None`, which deletes the model and calls `torch.cuda.empty_cache()` - `.loaded: bool` - `.ocr(pngs: list[bytes]) -> list[EngineResult]` (engine `"unlimited"`, confidence `None`); `len(pngs) <= UNLIMITED_BATCH` - On CUDA OOM: `ocr` raises `OcrOOM`, after emptying the cache.

- [ ] **Step 1: Failing tests:**
- `test_remove_det_strips_markers_and_images`: input `"<|det|>title [1,2,3,4] <|/det|>Heading\n<|det|>image [..]<|/det|>x\n<|det|>text [..]<|/det|>Body line"` gives `"Heading\n\nBody line"`.
- `@pytest.mark.gpu test_ocr_reads_rendered_text`: same check as in Task 3.
- [ ] **Step 2:** Run `pytest tests/test_unlimited.py -v -m "not gpu"`. Expected: FAIL.
- [ ] **Step 3:** Implement. Load with `AutoModel.from_pretrained(UNLIMITED_DIR, trust_remote_code=True, use_safetensors=True, torch_dtype=torch.bfloat16).eval().cuda()`. Single page uses `infer(prompt='<image>document parsing.', base_size=1024, image_size=640, crop_mode=True, max_length=8192, no_repeat_ngram_size=35, ngram_window=128)`. Use the output-handling approach recorded in Task 1's findings, writing to a temp directory. Pass the raw text through `remove_det`.
- [ ] **Step 4:** Run with and without `-m gpu`. Expected: both pass. The GPU test needs Task 1's setup.
- [ ] **Step 5:** Commit: `git commit -m "ocr: Unlimited-OCR pass"`.

Task 5: GPU big-slot scheduler

Files: Create `backend/app/gpu.py`, `backend/tests/test_gpu.py`.

Interfaces: - Produces: - `class BigSlot`: - `hold(owner: str, urgent: bool = False)`, a context manager that blocks until the slot is free; an urgent waiter goes ahead of non-urgent ones - `should_yield(owner: str) -> bool`, True while an urgent waiter is queued - a module singleton `big_slot = BigSlot()` - Usage contract: - The OCR upgrader checks `should_yield` between batches. When it's True, it unloads Unlimited-OCR, leaves `hold`, and re-enters `hold("ocr")`. - The planner side wraps its LLM call in `with big_slot.hold("planner", urgent=True)` (done in Task 12).

- [ ] **Step 1: Failing tests:**
- `test_hold_is_exclusive`: two threads both increment a counter inside `hold`; the observed max concurrency is 1.
- `test_ocr_yields_to_urgent`: thread A holds `"ocr"` and loops `while not should_yield("ocr")`. Thread B requests `hold("planner", urgent=True)`. A exits; B acquires within 1 s; the event order is `[ocr-in, ocr-out, planner-in]`.
- [ ] **Step 2:** Run. Expected: FAIL.
- [ ] **Step 3:** Implement with a `threading.Condition` and a deque of waiters, with urgent waiters inserted at the left.
- [ ] **Step 4:** Run. Expected: 2 passed.
- [ ] **Step 5:** Commit: `git commit -m "gpu: big-slot scheduler"`.

Task 6: Arbiter

Files: Create `backend/app/ocr/arbiter.py`, `backend/tests/test_arbiter.py`.

Interfaces: - Consumes: `EngineResult`. - Produces: - `has_repetition_loop(text: str, n: int = 30, times: int = 3) -> bool`, computed over whitespace tokens - `script_validity(text: str, lang: str) -> float`: the share of alphabetic characters inside the script range for `lang`. Ranges: en Latin `\u0041–\u024F`, hi/mr Devanagari `\u0900–\u097F`, bn `\u0980–\u09FF`, gu `\u0A80–\u0AFF`, ta `\u0B80–\u0BFF`, te `\u0C00–\u0C7F`, kn `\u0C80–\u0CFF`, ml `\u0D00–\u0D7F`. For non-English languages, Latin also counts as valid. - `lexicon_hit_rate(text: str, lexicon: set[str] | None) -> float | None` - `agreement(a: str, b: str) -> float`: `rapidfuzz.fuzz.ratio` of whitespace-normalised, lowercased text, divided by 100 - `Decision(text: str, engine: str, agreement: float | None, low_confidence: bool, reason: str)` - `choose(tess: EngineResult, uni: EngineResult | None, lang: str, lexicon: set[str] | None = None) -> Decision`

- [ ] **Step 1: Failing tests** (one assertion each):
- `test_no_unlimited_keeps_tesseract`: `choose(t, None, "en").engine == "tesseract"`
- `test_repetition_loop_rejected`: Unlimited text is `"a b c ... " * 10` with 40 distinct tokens repeated, giving `engine == "tesseract"` and `"repetition" in reason`
- `test_length_mismatch_rejected_when_tess_confident`: Tesseract conf 85 with 300 words vs Unlimited 100 words gives `"tesseract"`
- `test_length_mismatch_ignored_when_tess_unsure`: the same texts with Tesseract conf 50 gives `"unlimited"`
- `test_wrong_script_loses`: lang "hi", Tesseract text in Devanagari, Unlimited text in Latin transliteration gives `"tesseract"`
- `test_default_prefers_unlimited`: similar texts in the same script give `"unlimited"`
- `test_low_agreement_flags_page`: two unrelated valid texts give `low_confidence is True`
- [ ] **Step 2:** Run. Expected: FAIL.
- [ ] **Step 3:** Implement the rules in the order given in Global Constraints. `reason` names the rule that decided.
- [ ] **Step 4:** Run. Expected: 7 passed.
- [ ] **Step 5:** Commit: `git commit -m "ocr: per-page arbiter combining Tesseract and Unlimited-OCR"`.

Task 7: Chunker, embedder, collection store

Files: Create `backend/app/rag/chunker.py`, `backend/app/rag/embed.py`, `backend/app/rag/store.py`, and tests `test_chunker.py` and `test_store.py`. Also create `backend/tests/fakes.py` (a `FakeEmbedder` that hashes word tokens into a 384-dimension unit vector, so texts sharing words are similar).

Interfaces: - Consumes: `TocEntry`. - Produces: - `Chunk` (dataclass): `chunk_id: str, doc_id: str, title: str, section: str, section_idx: int, page_start: int, page_end: int, lang: str, source: str, license: str, url: str, engine: str, low_confidence: bool, text: str` - `sections_from_toc(n_pages: int, toc: list[TocEntry]) -> list[tuple[int, str, int, int]]`, returning `(section_idx, title, first_page, last_page)`. Uses the top-level TOC entries; without a TOC, it uses 10-page windows titled `"Pages a–b"`. - `chunk_section(doc_meta: dict, section: tuple, pages: list[tuple[int, str, bool]], count_tokens: Callable[[str], int], max_tokens=350, overlap=50) -> list[Chunk]`. Pages are `(page_no, text, low_confidence)`. `chunk_id` is `f"{doc_id}:{section_idx}:{i}"`. A chunk is low_confidence if any of its pages is. - `class Embedder(model_dir: Path, device: str = "cpu")`: - `.dim = 384` - `.passages(texts: list[str]) -> np.ndarray` - `.query(text: str) -> np.ndarray` - Rows are L2-normalised float32, with `passage:` / `query:` prefixes. `.count_tokens(text) ->`

`int`. - `class Collection(path: Path, embedder)`: - `.add(chunks: list[Chunk])` - `.delete_section(doc_id: str, section_idx: int)` - `.keyword(query: str, k: int) -> list[tuple[str, float]]` - `.vector(qvec: np.ndarray, k: int) -> list[tuple[str, float]]` - `.get(ids: list[str]) -> list[Chunk]` - `.count() -> int` - `.meta: dict`, read from and written to `path/meta.json`; it includes `embed_model` - Storage: `path/chunks.sqlite`, with table `chunks` (all Chunk fields, integer `rowid`) and FTS5 `chunks_fts(text, title, content='chunks', content_rowid='rowid', tokenize='unicode61 remove_diacritics 0')`. Vectors go in `path/vectors.faiss`: an `IndexIDMap2(IndexFlatIP)` keyed by rowid for uploads, or an `IndexIVFPQ` written by Task 11 for the library. `Collection` detects the type on open. `delete_section` raises on IVF-PQ (library collections are read-only). - FTS queries: build `OR` of quoted tokens (`"tok1" OR "tok2"`) so user text never breaks FTS syntax. Score = `-bm25()`.

- [ ] **Step 1: Failing tests** (`test_chunker.py`):
- `test_chunks_respect_max_tokens_and_overlap`: 2,000 words gives every chunk ≤ 350 tokens, and consecutive chunks share ≥ 40 tokens.
- `test_markdown_table_is_atomic`: a 12-row `|` table sits inside exactly one chunk.
- `test_display_equation_is_atomic`: a `$$ ... $$` block spanning 5 lines is never split.
- `test_sections_fallback_windows`: 25 pages with no TOC gives `[(0,"Pages 1–10",1,10),(1,"Pages 11–20",11,20),(2,"Pages 21–25",21,25)]`.
- `test_low_confidence_propagates`.
- [ ] **Step 2: Failing tests** (`test_store.py`, using FakeEmbedder):
- `test_keyword_finds_exact_term`
- `test_vector_finds_shared_words`
- `test_delete_section_removes_from_both_indexes`
- `test_fts_query_with_quotes_and_symbols_does_not_raise`, with query `'what is "AC/DC"? (part-2)'`
- `test_cross_script_query_still_hits_vector`: store a Devanagari chunk; with a real e5 embedder (`@pytest.mark.slow`), the English query "photosynthesis in plants" returns it in vector top-3.
- [ ] **Step 3:** Run. Expected: FAIL.
- [ ] **Step 4:** Implement. The chunker splits text into blocks (paragraphs; tables and `$$`/`\[ \]` blocks kept whole), then packs blocks greedily up to max_tokens. Overlap carries trailing blocks, or a trailing sentence slice, of ≥ 50 tokens into the next chunk. A single block larger than max_tokens is split on sentences unless it's a table or equation, which stays whole.
- [ ] **Step 5:** Run `pytest tests/test_chunker.py tests/test_store.py -v` and also `-m slow`. Expected: all pass.
- [ ] **Step 6:** Commit: `git commit -m "rag: chunker, e5 embedder, SQLite FTS5 + FAISS collections"`.

Task 8: Reranker and hybrid search

Files: Create `backend/app/rag/rerank.py`, `backend/app/rag/search.py`, `backend/tests/test_search.py`.

Interfaces: - Consumes: `Collection`, `Embedder`, `Chunk`, `paths.LIBRARY_DIR`, `paths.UPLOADS_DIR`. - Produces: - `class Reranker(model_dir: Path, device: str = "cuda" if available else "cpu")` with `.score(query: str, texts: list[str]) -> list[float]`, where scores are sigmoid outputs in 0..1 - `rrf(rankings: list[list[str]], k: int = 60) -> list[tuple[str, float]]` - `Hit(chunk: Chunk, score: float, scope: str)` - `open_collection(scope: str) -> Collection | None`, cached. Scopes are

`"library"` or `"upload:<sha>"`; returns None if the collection doesn't exist on disk. - `search(query: str, scope: str, k: int = 8, *, embedder=None, reranker=None) -> list[Hit]`. Scope is `"library"`, `"upload:<sha>"` or `"both:<sha>"`. - `MIN_RELEVANCE = 0.2` (reranker score); hits below it are dropped.

- [ ] **Step 1: Failing tests** (FakeEmbedder plus `FakeReranker` = word-overlap fraction):
- `test_rrf_orders_by_combined_rank`: `rrf([["a","b"],["b","c"]])[0][0] == "b"`
- `test_upload_first_library_max_two`: in scope both, the upload has 10 relevant chunks and the library 10; the result has ≤ 2 library hits, and every upload hit ranks above every library hit.
- `test_low_confidence_downweighted`: identical texts, the low-confidence one ranks second.
- `test_irrelevant_upload_returns_library_only`: the upload is about cricket and the query is "photosynthesis"; all hits have scope `"library"`.
- `test_missing_collection_returns_empty`.
- [ ] **Step 2:** Run. Expected: FAIL.
- [ ] **Step 3:** Implement. Per collection, take keyword top-50 and vector top-50, merge with `rrf`, fetch the chunks, and rerank. Final score = reranker score × (0.8 if low_confidence). Apply MIN_RELEVANCE. For `both`, take the upload hits first, then fill with library hits, at most 2 and only while the total is < k.
- [ ] **Step 4:** Run. Expected: 5 passed.
- [ ] **Step 5:** Commit: `git commit -m "rag: reranker and hybrid search with scope weighting"`.

Task 9: Upload ingestion (both passes, incremental indexing, cache, progress)

Files: Create `backend/app/ocr/ingest.py`, `backend/tests/test_ingest.py`.

Interfaces: - Consumes everything in Tasks 2–8 and `gpu.big_slot`. - Produces: - `class UploadError(ValueError)` - `validate_pdf(path: Path) -> int`, which returns the page count. It raises `UploadError` with these exact messages: - `"PDF is larger than 2 GB"` - `"PDF has more than 5,000 pages"` - `"PDF is password-protected"` - `"Not a readable PDF"` - `file_sha(path: Path) -> str`, sha256 hex, first 16 characters - `resolve_focus(toc: list[TocEntry], n_pages: int, focus: str | None) -> list[int]`. It understands `"pages 120-135"`, `"page 7"` and `"chapter 4"` (the 4th level-1 TOC entry's range), and otherwise matches TOC titles by rapidfuzz `partial_ratio ≥ 80`. Returns 1-based pages; `[]` if nothing matches. - `priority_order(n_pages: int, toc_pages: list[int], focus_pages: list[int], tess_conf: dict[int, float]) -> list[int]`: TOC/front pages first, then focus pages, then by ascending Tesseract confidence, then the rest in page order. No duplicates. - `ingest_pdf(path: Path, lang: str | None, focus: str | None, progress: Callable[[str, float], None], *, unlimited=None, embedder=None) -> dict`. It runs pass 1 synchronously and returns `{"sha", "title", "pages", "digital", "scanned", "toc": [...], "focus_pages": [...]}`. It then starts the pass-2 upgrader thread, unless `unlimited is False` (used in tests). - `upload_status(sha: str) -> dict`: `{"sha", "pages", "tesseract_done", "unlimited_done", "failed", "low_confidence", "state": "ocr"|"upgrading"|"ready"}` - On disk under `UPLOADS_DIR/<sha>/`: `source.pdf`, `info.json`, `pages/<n>.json` (`PageRecord.to_json`), and `index/` (the Collection).

- [ ] **Step 1: Failing tests** (6-page `make_pdf` with 3 text pages and 3 image pages; FakeEmbedder; `unlimited=False` or a `FakeUnlimited` that returns fixed text):
- `test_ingest_indexes_digital_and_scanned_pages`: after `ingest_pdf`, `search("…word from scanned page…", f"upload:{sha}")` hits that page.
- `test_progress_reports_page_counts`: a progress stage contains `"OCR 3 / 3"`.

- `test_upgrade_replaces_section_chunks`: with FakeUnlimited, after the upgrader finishes, `PageRecord.engine == "unlimited"` for the scanned pages, and the old Tesseract-only text is gone from the index.
- `test_resume_uses_cache`: run ingest twice; the second run calls Tesseract 0 times (patched counter).
- `test_encrypted_pdf_rejected`, `test_too_many_pages_rejected` (patch the limit to 5).
- `test_resolve_focus_variants`: `"pages 2-3"` → `[2, 3]`; `"chapter 2"` → the 2nd TOC range; a title fuzzy match.
- `test_priority_order`: `priority_order(6, [1], [4, 5], {2: 30, 3: 90, 6: 50}) == [1, 4, 5, 2, 6, 3]`.
- `test_failed_page_does_not_stop_job`: Tesseract raises on page 2; the job finishes, page 2 is `failed`, and the others are indexed.
- [ ] **Step 2:** Run. Expected: FAIL.
- [ ] **Step 3:** Implement:
- **Pass 1:** digital pages take their text layer. Scanned pages go through `ocr_many`, and each finished page is written to the cache and its section re-chunked into the collection. Progress format: `f"OCR {done} / {scanned} pages"`.
- **Pass 2 (daemon thread):** acquire `big_slot.hold("ocr")`, load Unlimited-OCR, and walk `priority_order` in batches of `UNLIMITED_BATCH`. For each batch, apply `arbiter.choose`, update the cache, and `delete_section` + re-add the affected sections. Check `should_yield` between batches. On `OcrOOM`, halve the batch; at batch size 1, keep the Tesseract text. Skip scripts in `UNLIMITED_SCRIPTS_DISABLED`.
- The lexicon for the arbiter comes from `data/library/lexicon_<lang>.txt` if present (Task 11 writes it), otherwise None.
- [ ] **Step 4:** Run. Expected: all pass.
- [ ] **Step 5:** Run a GPU smoke test with `-m gpu`: ingest a real 30-page scanned PDF end to end and report times for both passes.
- [ ] **Step 6:** Commit: `git commit -m "ocr: upload ingestion with two-pass OCR and incremental indexing"`.

Task 10: HTTP routes (own router)

Files: Create `backend/app/routes_rag.py`, `backend/tests/test_routes_rag.py`.

Interfaces: - Consumes: `jobs.submit`, `ingest_pdf`, `upload_status`, `search`. - Produces an `APIRouter` named `router`: - `POST /api/uploads`: multipart `file` plus form fields `lang?` and `focus?`. It streams the file to `UPLOADS_DIR/tmp-<uuid>.pdf`, stopping at 2 GB with a 413. It runs `validate_pdf` (400 with the UploadError message), moves the file to `<sha>/source.pdf`, and returns `jobs.submit("upload", ingest_pdf, path, lang, focus)`. - `GET /api/uploads/{sha}` → `upload_status`. Returns 404 if unknown; sha must match `^[0-9a-f]{16}$`, otherwise 400. - `POST /api/search` with `{query: str, scope: str = "library", k: int = 8}` → a list of hit dicts (the Chunk fields plus `score` and `scope`).

- [ ] **Step 1: Failing tests** (FastAPI `TestClient` on an app that includes only `router`; ingest patched to a fake):
- `test_upload_returns_job`
- `test_upload_rejects_non_pdf_with_400`
- `test_status_404_and_400`
- `test_search_returns_hits`

- [ ] **Step 2:** Run. Expected: FAIL.
- [ ] **Step 3:** Implement.
- [ ] **Step 4:** Run. Expected: 4 passed.
- [ ] **Step 5:** Commit: `git commit -m "rag: upload and search routes"`.

Task 11: Offline library: fetch and build

Files: Create `backend/scripts/fetch_library.py`, `backend/scripts/build_library.py`, `backend/tests/test_library.py`.

Interfaces: - Consumes: `make_pdf`-style ingestion (`ingest_pdf(..., unlimited=False)` is not reused; NCERT and OpenStax use `triage` + `tesseract` directly into JSONL), `chunk_section`, `Embedder(device="cuda")`, `Collection`. - Produces: - Under `data/library/raw/<source>.jsonl`: records `{doc_id, source, title, lang, url, license, text}`, plus `manifest.json`. - `data/library/` as a Collection with IVF-PQ (`nlist=4096`, `m=48`, `nbits=8`, trained on a 200k random sample), plus `lexicon_<lang>.txt` (the top 200k tokens per language). - `keep_record(text: str, title: str) -> bool`, which applies the quality filters from spec §3. - Commands: - `fetch_library.py --source {vital,simple,indic,ncert,openstax,archive} [--limit N]`, resumable (it skips doc_ids already in the JSONL) - `build_library.py [--sources ...]`

- [ ] **Step 1: Failing tests:**
- `test_keep_record_filters`: < 200 words → False; a title ending in `(disambiguation)` → False; 50% non-letters → False; normal text → True.
- `test_build_small_library` (`slow`): three tiny JSONL files with FakeEmbedder and IndexFlatIP below 10k chunks (IVF-PQ needs ≥ 39×nlist points, so small builds fall back to flat). `search("…", "library")` returns a hit.
- [ ] **Step 2:** Run. Expected: FAIL.
- [ ] **Step 3:** Implement the fetchers:
- **vital:** collect titles from `https://en.wikipedia.org/wiki/Wikipedia:Vital_articles/Level/5/*` (all subpages, link text in list items), then stream `datasets.load_dataset("wikimedia/wikipedia", "20231101.en", streaming=True)` and keep matching titles.
- **simple, indic:** the same dataset with configs `20231101.simple` and `20231101.{hi,te,ta,ml,mr,kn,bn,gu}`, downloaded in full.
- **ncert:** book zip URLs from `https://ncert.nic.in/textbook.php`, classes 6–12, English and Hindi medium.
- **openstax:** PDF links from `https://openstax.org/subjects`.
- **archive:** the `https://archive.org/advancedsearch.php` query `mediatype:texts AND (licenseurl:*publicdomain* OR licenseurl:*creativecommons*) AND subject:(science OR mathematics OR history OR technology) AND format:"DjVuTXT"`, fetching `<id>_djvu.txt` and stopping at 5 GB total.
- Each fetcher records license and url.
- [ ] **Step 4:** Run the tests. Expected: pass.
- [ ] **Step 5:** Run every fetcher **in the background, today, online** (the long step). Then run `build_library.py` (GPU embedding, est. 2–3 h). Expected: `manifest.json` lists all 6 sources, and `Collection(LIBRARY_DIR).count()` is in the millions.
- [ ] **Step 6:** Commit the scripts and tests (not the data): `git commit -m "rag: offline library fetch and build"`.

Task 12: Integration with the lesson pipeline (gated on the other session)

Precondition: session `dhiraj-shetty-26` has committed its planner, pipeline and main.py work. Coordinate by message before editing.

Files: - Modify: `backend/app/knowledge.py` (retrieve, status) - Create: `backend/app/rag/factcheck.py`, `backend/tests/test_knowledge_bridge.py`, `backend/tests/test_factcheck.py` - Hand-off (other session edits): the `main.py` include_router line; `planner.py` (passes `upload`, the `sources` field, the big_slot hold, fact-check call); the web upload UI.

Interfaces: - Produces: - `knowledge.retrieve(concept: str, upload: str | None = None) -> list[Passage]`. - If `open_collection("library")` or an upload collection exists, it calls `rag.search(concept, "both:<sha>" if upload else "library", k=config.RAG_TOP_K)` and maps each Hit to Passage: - `title` = chunk title - `section` = `f"{chunk.section}, pp. {page_start}-{page_end}"` (or the section alone when the source is Wikipedia) - `url` = chunk url, or `f"upload:{sha}#page={page_start}"` - `source` = chunk source - `score` = reranker score - Otherwise it falls back to the existing `KnowledgeBase`. - It never raises. - `knowledge.status()` adds `"library": {"chunks": n} | None` and `"uploads": n`. - `factcheck.unsupported(sentences: list[str], passages: list[str], reranker, threshold: float = 0.3) -> list[int]`: the indices of sentences whose best reranker score against any passage is below the threshold. - `knowledge.pages_passages(upload: str, focus: str) -> list[Passage]`, which serves the chapter/page-range and PDF-only modes (spec §8). It calls `resolve_focus` against the upload's TOC (for an empty focus, it picks the longest level-1 chapter) and returns that range's chunks in reading order as Passages. Condensing them (map-reduce) stays in the planner.

- [ ] **Step 1: Failing tests:**
- `test_retrieve_uses_new_search_when_library_exists`, with Passage fields mapped as above
- `test_retrieve_falls_back_to_kb`
- `test_retrieve_never_raises` (search patched to raise)
- `test_unsupported_flags_unrelated_sentence`
- `test_pages_passages_reading_order`: `"pages 2-3"` returns chunks with non-decreasing page_start, all within 2..3
- [ ] **Step 2:** Run. Expected: FAIL.
- [ ] **Step 3:** Implement.
- [ ] **Step 4:** Run the full suite: `venv/Scripts/python -m pytest -m "not gpu and not slow"`. Expected: all pass.
- [ ] **Step 5:** Commit. Then send `dhiraj-shetty-26` the hand-off list:
- `app.include_router(routes_rag.router)` in main.py
- `knowledge.retrieve(concept, upload=req.upload)`
- `with gpu.big_slot.hold("planner", urgent=True):` around the Ollama call
- `factcheck.unsupported(...)` for scene regeneration
- the summary-scene source line from `Passage.section`
- retuning `RAG_MIN_SCORE` for reranker scores (suggest 0.2)

Task 13: Retrieval benchmark

Files: Create `backend/scripts/bench_retrieval.py`, `backend/data_bench/questions.jsonl` (committed: 50 `{pdf, question, expected_pages}` rows across an English NCERT book, a Hindi NCERT book and a scanned archive.org book).

- [ ] **Step 1:** Write the 50 questions by reading the three PDFs.
- [ ] **Step 2:** Implement `bench_retrieval.py --mode {tesseract,unlimited,combined}`. It re-indexes each PDF using only that engine's page texts (from the cache) and prints recall@8 and mean latency per mode.
- [ ] **Step 3:** Run all three modes. Expected: combined ≥ max(tesseract, unlimited), and warm latency < 150 ms per upload. Record the numbers in the spec's §10 as a results table.
- [ ] **Step 4:** Commit: `git commit -m "rag: retrieval benchmark and results"`.

> Older detailed notes and per-version changelog (kept for reference; the main README.md is the current overview).

Chanakya

Type a concept and get a 30–90 s narrated, animated lesson in English plus Indian languages. It plays in the browser and exports to MP4. Everything runs offline on one laptop.

```
concept → retrieve facts from offline knowledge base (embeddings, ~1 ms search)
        → Ollama LLM → lesson JSON (schema-checked) → IndicTrans2 → Piper / Indic Parler-TTS
                                                                    | audio length = scene
length
browser player ← Remotion scene templates ← timed lesson JSON ←──────┘ captions timed to
audio
MP4 export     ← Remotion renderer (headless Chrome + bundled ffmpeg)
```

- **Data, never code:** the LLM only picks one of 10 templates (title, bullets, flow, cycle, compare, diagram, quiz, summary, **graph**, **code**) and fills in its text fields.
- **Narration first:** each scene's voice is synthesized before anything is animated, and the scene lasts exactly as long as that audio plus 0.5 s. Item reveals and captions are timed against the same audio.

Knowledge base and retrieval (RAG)

Lessons are grounded in an offline knowledge base, so facts come from a source and not only from the 7B model's memory.

- **Dataset:** about 300 school topics in `data/topics.txt`, covering biology, chemistry, physics, earth and space, maths, Indian geography, history and civics, economics and computers. They are fetched from **Simple English Wikipedia**, which is written for learners, with English Wikipedia as a fallback. The current build has 308 articles split into 3,500 passages.
- **Your own material:** drop `.txt` or `.md` files (notes, syllabus text, worksheets) into `data/kb_extra/`, then rebuild.
- **Indexing:** `backend/scripts/build_knowledge.py` splits articles by section into ~700-character passages and drops references and links sections. It embeds them with `nomic-embed-text` through Ollama and saves a normalised matrix in `data/kb/`. Rebuild offline with `--no-fetch`.
- **Search:** one matrix-vector product (cosine similarity), plus a boost when the question names the article. At most 2 passages per article for variety, and a score cut-off (0.68) so unrelated topics add nothing. Takes ~35–60 ms.
- **Use:** the top passages (up to ~4,000 characters) go into the planner prompt as reference material. Each lesson lists its **Sources**.
- **Inspect it:** `GET /api/knowledge/search?q=how do vaccines work`.

Wikipedia text is CC BY-SA 4.0. The articles are downloaded during setup, not stored in this repo, and each lesson links to the articles it used.

Lessons from your own PDF

Attach a textbook or notes PDF (up to 2 GB / 1,000 pages) in the UI, or `POST /api/uploads`. Digital pages are read directly. Scanned pages go through **Tesseract** (about 6 pages/second on 22 cores: 1,000 pages in under 3 minutes, including embeddings and the search index) and then, when the GPU is idle, a higher-accuracy **Unlimited-OCR** pass that always gives way to lesson generation.

- **Concept + PDF:** "photosynthesis" searches the PDF first, then the knowledge base.
- **Chapter or pages:** leave the concept empty and type `chapter 4` or `pages 120-135` as the focus, or type "explain chapter 4" as the concept. The lesson teaches that part of the book in reading order, condensed to fit the prompt.
- The recap scene shows a citation such as `Source: Science 8, Chapter 2, pp. 41-44`.
- Uploads run on their own job queue, so a long OCR run never delays a lesson.

Design: `docs/superpowers/specs/2026-10-27-rag-ocr-design.md`. Code: `backend/app/ocr/`, `backend/app/rag/`, `backend/app/routes_rag.py`.

Quick start (Windows, PowerShell)

```
git clone https://github.com/signingoff-dubey/UNLOX-TEACHER.git
cd UNLOX-TEACHER
.\setup.ps1 -Gpu     # once, online. Leave out -Gpu for a CPU-only setup (Piper voices + Ollama
translation)
.\start.ps1          # opens http://127.0.0.1:8000; add -Lan to accept other devices
```

If PowerShell blocks the scripts, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once. You also need **Ollama** (https://ollama.com); `setup.ps1` pulls the model if it's installed.

What the UI offers: - Type a concept or click an example, pick languages, and get a narrated lesson in under a minute. - Language tabs to switch the same lesson between English, Hindi, Telugu and others. - A scene list that jumps the player to any scene and highlights the one playing. - MP4 export and `.srt` captions download per language. - **Recent lessons:** replay any earlier lesson. Generate backups before the demo. - **System status:** shows whether the planner, translator, voices and video export are ready. - **Backend address:** the UI can use a backend on another laptop (e.g. `http://192.168.1.20:8000`). - Build-time breakdown, and the lesson JSON for judges who ask how it works.

Layout

Path	What
`backend/app/planner.py`	Retrieval-grounded prompt, Ollama JSON-schema output, validation, retry, fallback
`backend/app/knowledge.py`	Offline vector search over the knowledge base
`backend/scripts/build_knowledge.py`	Fetches, chunks and embeds the dataset
`data/topics.txt`, `data/kb_extra/`	Dataset topic list and your own documents

Path	What
`backend/app/schema.py`	Lesson schema + `normalize()` that repairs small-model mistakes
`backend/app/translate.py`	IndicTrans2 (or Ollama) translation, cached on disk
`backend/app/tts.py`	Piper (CPU) → Indic Parler-TTS (GPU) → silent fallback chain, cached
`backend/app/pipeline.py`	Orchestration; builds per-language timed tracks + `.srt`
`backend/app/exporter.py`	Runs `web/render.mjs` to make the MP4
`backend/app/main.py`	FastAPI app, job queue, serves media/fonts/built frontend
`web/src/remotion/`	Video: templates, icons, captions, fonts
`web/src/App.tsx`, `web/src/components/`	Student UI: input, player, scenes, history, status
`web/src/api.ts`	API client, job polling, configurable backend address
`setup.ps1`, `start.ps1`	One-command setup and start

Setup on the machine that runs the backend (the one with the models)

The backend renders MP4s with Node, so copy the **whole project folder** (backend + web) to that machine.

1. **Python 3.12** (3.14 is too new for these packages): `powershell cd backend py -V:3.12 -m venv venv .\venv\Scripts\pip install -r requirements.txt # GPU extras (RTX 50-series needs CUDA 12.8 wheels): .\venv\Scripts\pip install torch --index-url https://download.pytorch.org/whl/cu128 .\venv\Scripts\pip install -r requirements-ml.txt`
2. **Config:** `copy .env.example .env`, then point it at the models already on that machine: - `INDICTRANS2_MODEL` / `PARLER_MODEL`: a Hugging Face id (read from the HF cache) or a local folder path - `PIPER_DIR`: the folder with `*.onnx` + `*.onnx.json` voices. Files must be named as in `backend/app/languages.py`, e.g. `hi_IN-priyamvada-medium.onnx`. - `OLLAMA_MODEL`: must match a name in `ollama list`
3. **Anything missing** (voices, fonts, models) while still online: `.\venv\Scripts\python scripts\download_models.py [--ml]`
4. **Frontend + renderer** (Node 20+): `powershell cd ..\web npm install npm run bundle # pre-builds the video bundle + downloads headless Chrome once npm run build # backend then serves the UI at http://127.0.0.1:8000`
5. **Run:** `ollama serve`, then `cd backend; .\run.ps1` (add `-Lan` to accept other devices). Check `http://127.0.0.1:8000/api/health`: it reports what loaded (Ollama, translator, voices, renderer).
6. **Before the demo:** set `HF_HUB_OFFLINE=1`, turn Wi-Fi off, and generate one lesson end to end.

Development: run the backend, then `cd web; npm run dev` (UI on :5173, proxied to :8000). `npm run studio` previews every template with no backend. `LLM_BACKEND=mock` runs the whole pipeline without Ollama.

Fallbacks (the demo keeps going)

Failure	What happens	Shown in the response as
LLM returns bad JSON	One retry with the error message, then a minimal 2-scene lesson	—
Ollama not running	Job fails with a clear "start Ollama" message	`job.error`
Translation model won't load	English text is kept for that language	`track.translator = "untranslated"`
No Piper voice for a language (kn, ta, bn, gu)	Indic Parler-TTS is used	`track.tts_engines`
No voice model at all	Silent audio with the right timing	`track.tts_engines = ["silent"]`

API

All long work runs as a job: POST returns a job, then poll `GET /api/jobs/{id}` until `status` is `done` or `error`.

Method	Path	Body	Returns
GET	`/api/health`	—	What is installed and loaded
GET	`/api/languages`	—	`[{code, name, native, piper}]`
POST	`/api/lessons`	`{"concept": "Photosynthesis", "languages": ["hi","te"]}`	Job; `result` = Lesson
GET	`/api/jobs/{id}`	—	`{id, status, stage, progress 0..1, result, error}`
GET	`/api/lessons/{id}`	—	Lesson
GET	`/api/lessons`	—	Recent lessons (history)
GET	`/api/knowledge/search?q=…&k=5`	—	Retrieved passages with scores
POST	`/api/lessons/{id}/export`	`{"lang": "hi"}`	Job; `result` = `{"url": "/media/.../lesson_hi.mp4"}`
GET	`/media/...`	—	Audio, MP4s, `captions_<lang>.srt`
GET	`/fonts/...`	—	Noto fonts used by the player and renderer

English is always included. The language codes are `en hi te ml mr kn ta bn gu`.

Lesson shape (`web/src/types.ts` mirrors it):

```
{
  "id": "e66517564b", "concept": "Photosynthesis",
  "timings": {"plan_s": 12.1, "en_s": 2.0, "hi_s": 3.5},
  "videos": {"en": "/media/lessons/e66517564b/lesson_en.mp4"},
  "languages": {
    "hi": {
      "lang": "hi", "name": "Hindi", "native": "हिन्दी", "title": "प्रकाश संश्लेषण",
      "duration_s": 49.6, "tts_engines": ["piper"], "translator": "indictrans2",
      "scenes": [{
        "template": "diagram", "heading": "…", "narration": "…", "items": ["…"], "icon": "leaf",
        "left_title": "", "right_title": "", "left": [], "right": [], "answer_index": 0,
        "audio_url": "/media/audio/<hash>.wav", "speech_s": 7.07, "duration_s": 7.57, "start_s":
4.85,
        "captions": [{"start": 0.0, "end": 2.4, "text": "…"}]
      }]
    }
  }
}
```


To render a track in any React frontend, pass it to `LessonVideo` (`web/src/remotion/Lesson.tsx`) with Remotion's `<Player>`: `inputProps={{ track, assetBase: "" }}`. `assetBase` is the backend origin if the UI is served from somewhere else.

Verified on the RTX 5060 laptop (mock planner, Piper voices)

- A 6-scene, 40 s English lesson with Hindi and Telugu tracks built through the API (narration: ~3.5 s for Hindi, ~4.2 s for Telugu).
- MP4 export: 1920×1080 H.264 + AAC, 40.2 s, matching the narration. The first export took 87 s, including the one-time bundle and Chrome download.
- Devanagari and Telugu render correctly in the video, including conjuncts, using the bundled Noto fonts.
- The backend serves the built UI, assets, lesson history and deep links (`/#<lessonId>`).
- **Full offline pipeline (Dhiraj's machine):** Ollama `qwen2.5:7b-instruct-q4_K_M` with RAG, NLLB-200 on the GPU, and Piper.
- "Why do we have seasons?" in English, Hindi and Telugu was built in **32 s**: plan 22 s, Hindi 4.5 s, Telugu 3.9 s.
- It drew on 5 knowledge-base sources (top match "Season", 0.80) and produced a 6-scene lesson of 47 s in English and 73 s in Hindi.
- The Hindi MP4 export took 60 s. The server starts in ~14 s with models preloaded.
- **Translation comparison:** NLLB-200 gives fluent Hindi and Telugu. Qwen-7B mixed scripts in Hindi and produced mostly wrong Telugu, and is now only a fallback. IndicTrans2 (Kabir's machine) is also supported and handles hi/te/ta/mr well.
- **Known limits:**
- Very short labels can be mistranslated because they lack context ("Light bends" became "lightweight bends").
- IndicTrans2 is gated on Hugging Face and its toolkit needs a C compiler on Windows.

Models, Voices & Assets Used (licenses)

Component	Name	License	Source
Planner LLM	Qwen2.5-7B-Instruct, Qwen3-4B (fallback)	7B: Apache-2.0 (verified via `ollama show --license`). 3B: Qwen Research License (verified) — NOT used. Fallback: qwen3:4b (confirm Apache-2.0 via `ollama show --license` once pulled)	ollama.com/library/qwen2.5
TTS engine	piper-tts 1.8.0 (OHF-Voice/piper1-gpl; uses espeak-ng)	**GPL-3.0-or-later** (verified via pip metadata). Old rhasspy/piper is MIT but archived	github.com/OHF-Voice/piper1-gpl
Piper voices	hi_IN-pratham, te_IN-venkatesh, mr_IN-google, bn_BD-google, ml_IN-meera, en_GB-alan (medium)	Per-voice, see each `.onnx.json` / model card — TO VERIFY	huggingface.co/rhasspy/piper-voices
Indic TTS	AI4Bharat Indic-TTS, int8 ONNX repack (en, ta, kn, gu)	MIT (upstream); repack by RaunakSaha, unofficial	huggingface.co/RaunakSaha/echobharat-models
Translation (optional)	IndicTrans2 distilled 200M	MIT (gated download)	huggingface.co/ai4bharat/indictrans2-en-indic-dist-200M
Fonts	Noto Sans (Indic scripts)	SIL OFL 1.1	github.com/google/fonts
Video engine	Revideo	MIT	github.com/midrender/revideo
Encoder	ffmpeg	LGPL/GPL by build	ffmpeg.org
Excluded: Meta MMS-TTS (CC-BY-NC), sarvam-translate (GPL-3.0 tag + Gemma base).			

Hackathon docs

See `prd.md`, `problem_statement.md`, `pitch.md`, `progress.md`, `issues.md`, `docs/orchestration.md`.

Submission Checklist

- [ ] Repo with README, setup, licenses

- [ ] MP4 on judge topic (live), MP4 on our topic
- [ ] 5-min offline demo rehearsed
- [ ] Verified with network disabled

Graph and code lessons

- Maths topics (circle, parabola, sine wave, lines...) get `graph` scenes: curves are chosen from a fixed list (`line, parabola, cubic, sine, cosine, exp, abs, sqrt, reciprocal, circle, ellipse`) with numeric parameters; circles can mark centre, radius, diameter, chord, tangent.
- Programming topics (Python, Rust, React/JSX, Java, Go, SQL...) get `code` scenes: syntax-coloured snippet, a highlight that follows the narration, one note per line, optional program output. Snippets are only displayed, never executed.
- Test offline without a model: `LLM_BACKEND=mock` with a concept containing "circle" or "python".
- Export size: `POST /api/lessons/{id}/export` takes `"height": 720` (default) or `1080`.
- GPU/CPU: `OLLAMA_NUM_GPU=auto|N|0`, `DEVICE=auto|cuda|cpu`; see `/api/health` -> `gpu`.

Screens and document upload

- Home: topic, optional PDF (with a focus like "chapter 4"), level, length, languages. Creating: progress. Lesson: player, scenes, downloads, sources.

Models and voices (as run on the demo laptop)

Role	Model	Where
Planner / reviewer / writer / tutor	Qwen2.5-7B-Instruct Q4_K_M via Ollama (GPU layers auto)	Ollama
Translation	AI4Bharat IndicTrans2 dist-200M (MIT, gated)	`models/indictrans2` (`INDICTRANS2_MODEL`)
Voices hi, te, ml, en	Piper	`models/piper`
Voices ta, kn, gu, bn, mr (+ en)	AI4Bharat Indic-TTS ONNX (CPU)	`models/indic-tts/<lang>`
Check only	faster-whisper medium (round-trip test of voices)	HF cache

Download gated models with `HF_HUB_DISABLE_XET=1` set (the Xet path stalls on some networks).

Study tools, Meta quiz and the metaverse entrance (Kabir)

- **Study tools** (`backend/app/studio.py`, `web/src/studio/`): flashcards, quiz, mind map, study guide, FAQ, glossary and timeline, made from the lesson (and its PDF passages). They are written **in the background** as soon as a video exists (low priority, so lesson generation is never slowed; a tool you click jumps the queue). The buttons at the top of the chat panel show the state: ready tools open instantly; a tool being written has a border of flowing colour and a steady percentage, and opens by

itself when done. API: `GET /api/lessons/{id}/studio?lang=` (status), `POST` the same path (jump the queue), `GET .../studio/{kind}` (result). Output is JSON only; the browser draws it. Size and difficulty options, Markdown export, Anki CSV export for flashcards.

- **Mind map** is Mermaid text written by the model, parsed and re-written by the backend (nothing but a clean mind-map tree can reach the browser) and rendered on-device with Mermaid (bundled, offline).
- **Meta quiz** button: builds one metaverse world from **everything** the AI wrote for the video (or for every episode of a course): the lesson, quiz, flashcards, glossary, FAQ, study guide, timeline and mind map. Four stations to read (with extra reading from every tool) and quiz gates that ask several questions each, with explanations. `POST /api/metaquiz`.
- **PDF notes** are written 30 pages at a time as the OCR reads them (`backend/app/ocr/digest.py`, `GET /api/uploads/{sha}/notes`): a 100-page scan has its first notes after about 22 s and all of them after about 32 s. A lesson about a whole PDF is planned from these notes, not from 1,000 pages. Uploads are capped at 1,000 pages.
- **Metaverse entrance** (`web/src/components/TunnelTransition.tsx`, `MetaverseLayer.tsx`): a tunnel of blocks builds around you, light speed, a bright light, then a Start page. All motion is compositor-only (no blur filters); the 3D world starts up behind a still white frame so nothing animated waits on the main thread.

PDF lessons, RAG and the metaverse (merged from Dhiraj and Niharika)

- **PDF lessons:** `POST /api/uploads` (PDF up to 2 GB) -> job; poll `GET /api/uploads/{sha}`. Then `POST /api/lessons` with `upload=<sha>` (and optional `focus` such as "chapter 4"); an empty concept teaches that part of the PDF. Playlists take `documents: [sha]`. OCR (Tesseract + Unlimited-OCR) and hybrid search (e5 embeddings, FTS5 + FAISS, bge reranker) live in `backend/app/ocr` and `backend/app/rag`; the offline library is built with `backend/scripts/fetch_library.py` / `build_library.py`.
- **Metaverse quiz world (Niharika):** `metaverse/` is a walkable 3D world with quiz gates, points and a shop. The web app opens it from the Home screen and from a lesson ("Quiz in the metaverse"); a lesson becomes a world through `metaverse/src/lessonToWorld.ts`.
- **Voice assistant (Niharika's plan in `Niharika/`):** see `docs/voice-assistant.md` for what is built.
- `Dhiraj/` is a full snapshot of Dhiraj's own build, kept as pushed; the running app is the root `backend/` and `web/`.
- Backend tests: `cd backend && pytest` (needs `requirements-rag.txt`).

Home screen, rewards and long videos (v0.9.0)

- The prompt box on Home shrinks into a header as you scroll; settings (1-3 videos, 0:30-4:00 each, level, languages, PDF) are in the card below it
- The left column holds your awards: streak, score, level, daily goal, badges
- Videos over 1:30 are planned in parts and joined (`backend/app/longplan.py`); the API takes `duration_s` 30-240
- While a video is being made you can leave its page: a floating "Creating…" pill (bottom right) leads back and says "Created" when it is done

Photos, modes and homework (v0.10.0)

- Plus button in the prompt box: attach photos (read locally by llava)
- Explanation / Study / Homework pills; Homework also writes an answers file (`#/homework/<id>`, download as .md)
- Videos: Auto lets the AI pick one video per concept

Android app (v0.13.0)

`Unlox_Hackethon/` is the same product as a native Kotlin + Jetpack Compose app that runs fully on the phone (tested on a OnePlus 13R, 16 GB). No server, no internet at run time.

Part	Engine
Tutor, lesson planner, study tools, homework solver	Gemma 4 E2B through LiteRT-LM
Narration	Piper voices through sherpa-onnx 1.13.8 (en, hi, te, mr, bn, gu, ml); the phone's own voice for Tamil and Kannada
Speech to text	Whisper names the language, then Vosk (en, hi, te, gu) or Whisper base (ta, kn, ml, mr, bn). The approach is borrowed from the NDialRecorder
Translation	ML Kit language packs (download once), Gemma as the fallback
Homework photos	ML Kit text recognition (Latin and Devanagari)

Screens follow the Figma file "NOTES - Kabir Edition" (onboarding, home, create, generating, player, tutor chat, progress, library) plus study tools, homework, rewards, settings and a profile with name and age. The opening animation and the ripple and wave of the voice screen are the website's. Models are not in git (about 4 GB): copy them to `Android/data/com.kabir.unlox_hackethon/files/models/` (`llm/`, `stt/`, `tts/`, `embeddings/`); Settings > On this phone shows what is found. Build: `cd Unlox_Hackethon && gradlew :app:assembleDebug`. A debug APK is attached to the GitHub release.

What is new in v0.14.0

- **Scenes play while they are made**: the player opens on scene 1 about 6 s after you ask, the rest arrives behind it.
- **Review** (spaced repetition), **Revise** mode (your weak spots), **Profile** (name and age), **Teach it back**.
- **Take it away**: subtitles only, narration only (mp3), the whole lesson as one `.chanakya.zip` with its own player (copy it to a USB stick, import it from Library), worksheets and question papers to print, "Hear it" for glossary terms.
- **Safer translation** (script and back-translation checks), **network diagrams**, **misconception scenes**, **jobs that resume after a crash**.
- New pages: `#/review`, `#/profile`. New API: `/api/review/*`, `/api/weak*`, `/api/profile`, `/api/papers*`, `/api/lessons/{id}/{subtitles,podcast,say,bundle,teachback}`, `/api/bundle/import`, `/api/jobs`.
- Environment: `STUDIO_WORKERS` (3), `BACKCHECK` (1: translate narration back to English and compare).

- New backend files: `streaming.py`, `srs.py`, `learner.py`, `fidelity.py`, `exports.py`, `bundle.py`, `paper.py`, `teachback.py`, `layout.py`, `resume.py`.

What is new in v0.14.1

- **Learnverse builds a world for any topic you type** with the local model, behind a "Building your world" screen. Quiz gates open by themselves when you reach them, Space (or the Jump button) jumps, and points and purchases are also kept on the backend.
- New API: `POST /api/metaworld`, `GET/POST /api/metaverse/save`. New backend file: `metaworld.py`.
- The "+" button on the home prompt now sits beside Create lesson.

What is new in v0.14.2

- A GPU out-of-memory no longer fails a whole course: torch models fall back to the CPU, the failed episode is retried. Environment: `TRANSLATE_MIN_FREE_GB` (1.8).
- Long videos are made in parts of 30 s and play while they are made; episode 1 of a course plays while it is made.
- Videos start playing by themselves again after a language switch or a new episode.
- Resource limits (all optional): `MIN_FREE_RAM_GB` (1.5), `MIN_FREE_VRAM_GB` (0.8), `MODEL_IDLE_S` (600), `PIPER_CUDA` (0), `PRELOAD_STT` (0), `STT_MIN_FREE_GB` (3), `STUDIO_WORKERS` (2), `PLAYLIST_WORKERS` (2). See `docs/presentation-mode.md`.
- `TORCH_VRAM_FRACTION` (0.35): the share of the graphics card PyTorch may hold.

What is new in v0.14.3

- Say "generate one more video on ..." in the tutor chat: the video is built scene by scene, shows up in the chat with its first frame and a play button, and is added to the playlist under the player. Every episode of a course now plays while it is made.

What is new in v0.14.4

- **PDFs**: pages with a text layer are read as Markdown (headings, lists, tables kept) instead of OCR; broken fonts are detected and sent to OCR; pictures inside the PDF are described by llava (`PDF_FIGURES=1`, `PDF_MAX_FIGURES=40`) and searched with their page. New dependency: `pymupdf4llm==1.27.2.2` (`pip install -r backend/requirements-rag.txt`).
- **Speed**: translation batches by length, other-language translations pre-cached while a lesson is being planned, speech recognition results cached, embedding and speech models pre-warmed when idle.

What is new in v0.15.0

- **Faster, steadier first scene**: the planner prompt is built for the subject (first scene about 1.2 s warm, was about 3.2 s with a wide spread).
- **PDF upload**: one continuous progress bar, much faster Markdown reading.

- **Android**: long videos in 30 s parts, every video of a set opens as soon as its first scene exists and plays on to the next, "make one more video on ..." in the tutor chat, and lag fixes.
- **Learnverse**: new sky, wind, sparks, colour grade, livelier avatar.

What is new in v0.15.2

- **Coherent long videos and courses**: at most 3 parts per video and 3 videos per course, planned one after the other, each told what was already taught; one title card, one summary, no repeated headings, code or narration, captions of 22 to 34 words.
- **"Teach me" is gone from titles**: the subject is taken out of the request ("teach me about dsa in java" is "DSA in Java"), on the website and the phone.
- **A photo no longer blocks Create** and its description is cleaned (no loops, only the lines that fit each episode).
- **Expressive narration** (`TTS_EXPRESSIVE=1`, default on; `0` for the old flat voice): per-scene tone and per-sentence pacing, pauses and emphasis. New backend files: `content.py`, `expression.py`.

What is new in v0.16.0

- **Flash and Pro** on the Home page: Flash shows scenes within seconds and keeps writing; Pro plans the whole video first (outline, first minute, then each further minute patched on) and shows it when it is complete. Same model for both. API: `"quality": "flash" | "pro"`.
- **The planner remembers**: it is told which related videos the learner already has and does not repeat them (`memory.py`).
- **Technical topics are shown**: code scenes in the language you name, trees and structures drawn as boxes and arrows, one everyday comparison at most.
- **Reference notes** in `data/kb_extra/` (data structures and AVL trees, MERN, coordinate geometry, Python) feed the planner and the fact checker; rebuild with `python scripts/build_knowledge.py --no-fetch`.
- **Robustness**: retries when Ollama is busy, `GET /api/ready` (what is missing and the command to fix it), a start-up self-check in the log.
- New backend files: `memory.py`, `selfcheck.py`, `facts.py`.

Teammate modules

Blender 3D lessons (Dhiraj) and the documents that follow

3D lessons on any topic, offline (headless Blender)

A separate tool, outside the Visual Teacher repo. A student's question becomes a narrated 3D video, with no internet:

1. **Plan:** the local model (Ollama) returns JSON data only, never code. `planner.py` validates and clamps it: - If a hand-built scene fits exactly (seasons, solar_system, atom), it writes the script for that scene. - Otherwise it designs a **custom scene** from building blocks: objects, colours, finishes, positions, labels, and per-shot actions and camera moves. A layout pass pushes overlapping objects apart.
2. **Translate** narration and labels with the backend's NLLB.
3. **Narrate** with the backend's Piper voices. Each shot lasts as long as its voice.
4. **Labels** are drawn by Windows text shaping, so Hindi, Telugu and Tamil conjuncts are correct. They float above objects and face the camera.
5. **Render** in Blender (EEVEE, headless). It writes `lesson.mp4` (H.264 with AAC narration) and `lesson.srt` captions.

Use

```
.\make3d.ps1 -Doctor                                          # check this PC first
.\make3d.ps1 -Concept "explain the water cycle" -Lang te
.\make3d.ps1 -Concept "how does a lever work" -Quality good
.\make3d.ps1 -Template atom -Element oxygen -Lang hi
.\make3d.ps1 -Plan out\<run>\plan.json -Lang ta               # same scene in another language
```

Each run writes `out\<time>-<topic>-<lang>\` with `lesson.mp4`, `lesson.srt`, `plan.json` (editable; re-render with `-Plan`), `job.json` and `render.log`. `python tools_frames.py out\<run> sheet.png` makes one preview frame per shot.

Building blocks (custom scenes)

Shapes	sphere, box, bar, cylinder, cone, torus, ring, plane, arrow (any direction), spring, wave surface
Science	planet (earth, mars, saturn…), star, atom (H to Ca, real shell counts), molecule (any formula; H2O, CO2, CH4, NH3, NaCl, benzene… built from real geometry), cell
Nature	leaf, tree, cloud, water drop
Look	16 colours × matte, glossy, metal, glass, glow; environments: space, studio, sky, lab, underwater
Actions	appear, disappear, move, rotate, spin, orbit, grow, shrink, stretch, pulse, highlight
Camera	wide, close, orbit, top, side, low

New building blocks go in `build_kind()` in `blender/scene_dsl.py`. Add their names to `KINDS` and the prompt in `planner.py`.

Setting up another PC

Everything below is installed once, online. After that the tool runs with Wi-Fi off.

Need	How
Visual Teacher backend + venv	the project's `setup.ps1` (Ollama with qwen2.5:7b, Piper voices, NLLB cached)
Blender 4.2+	blender.org installer or portable zip, or the Microsoft Store. Found automatically, or set `BLENDER_EXE`.
Windows 10/11	label text shaping uses Windows, with the Nirmala UI font (built in)
GPU	any; NVIDIA renders much faster

Run `.\make3d.ps1 -Doctor`. It checks each item and says what to fix.

Measured on this laptop (RTX 5060 8 GB, app server and Ollama running, draft 854×480)

Run	Video	Total time
custom: water cycle, Telugu	21.3 s	203 s (render 165 s)
custom: photosynthesis, Hindi	23.4 s	149 s (render 116 s)
template: seasons, English	25.0 s	62 s
template: atom (oxygen), Telugu	20.6 s	113 s
template: seasons, English, good 1280×720	25.0 s	222 s

The planner takes about 15 to 20 s and translation 10 to 25 s; the rest is rendering, at 2.5 to 10 times real time. That is fine in the background or for pre-made clips, but too slow to wait on live.

Limits

- Custom scenes are only as good as the local 7B model's plan. They are simple, and sometimes the actions only loosely match the narration. Edit `plan.json` and re-render with `-Plan` to fix one.
- Labels are not mirrored into the captions: the `.srt` holds only the narration.

Metaverse

A walkable 3D learning world for Chanakya. A student walks a path, reads topic stations, passes a quiz gate after each one, and earns points. Points buy rides (skateboard, bicycle, hoverboard), hats, suit colours, visors, pets and perks in an in-world shop.

Everything runs in the browser with three.js. No network is needed once the app is built: fonts are bundled through `@fontsource`, and sound is synthesized with the Web Audio API.

How it connects to Chanakya

`web/src/App.tsx` has a **Metaverse** button in the header. It lazy-loads this folder as a separate chunk and opens it full screen. When a lesson has been generated, the world is built from that lesson:

Lesson scene (backend JSON)	World
`bullets`, `flow`, `cycle`, `compare`, `diagram`	a knowledge station (title, narration as body, items as terms)
`quiz`	the quiz gate after a station
no quiz left for a station	a generated "which idea belongs here?" question from the lesson's own items
`summary` narration	the finale text

The active language tab is used, so a Hindi or Telugu lesson gives a Hindi or Telugu world. See `src/lessonToWorld.ts`.

Inside the world the learner can type any topic. The ready-made worlds (Java, photosynthesis, water cycle, and the open lesson) start at once; anything else goes to `POST /api/metaworld`, where the local model writes four stations with questions, while a "Building your world" screen shows its progress. Without a backend (`onBuild` not given) the old keyword matching and Java fallback remain.

Layout

Path	What
`src/Metaverse.tsx`	React component. Mounts the UI in a shadow root, so its CSS never touches the host app
`src/engine.ts`	The three.js engine: world, avatar, shop, input, quality scaling. Framework-free, returns a dispose function
`src/lessonToWorld.ts`	Backend lesson JSON to a walkable world
`src/ui.ts`	CSS and markup for the overlay
`src/types.ts`	`World`, `Zone`, `Quiz`

Path	What
`standalone/learnverse.html`	The same experience as one self-contained HTML file (loads three.js from a CDN)

Run on its own

```
cd metaverse
npm install
npm run dev        # http://localhost:5174
npm run build      # typecheck + production build
```

Run inside Chanakya

```
cd web
npm install
npm run dev        # click "Metaverse" in the header
```

`web/vite.config.ts` aliases `metaverse` to `../metaverse/src` and points `react`, `three` and `@fontsource` at `web/node_modules`, so there is one copy of each.

Controls

Arrow keys or WASD to walk and turn, `Space` to jump, `1`-`4` to answer, `B` for the shop, `Esc` to close it. Quiz gates open their question when you reach them (no key needed); stations are read by tapping the prompt, the Open button or pressing `E`. On touch screens there is an on-screen pad with Jump and Open buttons. Points, level and purchases are saved in `localStorage` and, when a `store` is given, on the backend (`/api/metaverse/save`), so they survive another address or cleared browser data.

Performance

Trees, lamps, crystals and clouds are instanced; fireflies animate on the GPU. The page lowers its render resolution when frames run slow, then drops from High (bloom, 2048 shadows) to Medium to Low. The Quality button switches by hand.

Known limits

- `engine.ts` is plain JavaScript with `// @ts-nocheck`. The React wrapper and the lesson mapper are strictly typed.
- Only three worlds are hand-written. Any other topic uses the generated lesson, so create a lesson first.
- Billboard titles use Chakra Petch, which has no Indic glyphs. Indic titles fall back to the system font.

Voice assistant for Visual Teacher: plan

Status: draft for review. No code has been written. Nothing here is implemented yet. Written against `main` at commit `de42195` (2026-10-27).

Goal

Turn the chatbot in the right-hand panel into a voice assistant that:

1. Listens to the student (speech to text, "STT").
2. Answers in a spoken voice (text to speech, "TTS") and shows the answer as text.
3. Knows the topic being taught: the current lesson, the scene on screen, and the offline knowledge base.
4. Can show a short generated animation of what is being discussed, if a local video model (LTX) turns out to be practical on this hardware.

Everything must keep working offline on the demo laptop (RTX 5060 Laptop, 8 GB VRAM, 31.5 GB RAM). No external AI APIs.

Assumption to confirm

The request said "TTS and HTTP". This plan reads "HTTP" as **STT (speech to text)**, because a voice assistant needs a listening half. If HTTP meant something else, for example "the chat should be served over HTTP endpoints", that is covered too: the transport design in `02-architecture.md` uses plain HTTP (with streaming) first and WebSocket second.

Documents

File	What it answers
`01-current-state.md`	What the codebase already has, and what is missing for a voice assistant
`02-architecture.md`	Components, API contract, transport, latency budget, GPU budget
`03-speech-to-text.md`	How the student's voice becomes text, offline, in English and Indian languages
`04-talkback-tts.md`	How answers are spoken back, interrupted, and kept in sync with the screen
`05-topic-knowledge.md`	How the assistant knows the current lesson and stays grounded in sources
`06-ltx-live-demo.md`	Research and a feasibility verdict on generating demo animations with LTX
`07-roadmap-testing-risks.md`	Milestones, acceptance tests, risks, open questions

Headline findings

- **There is no chatbot in the real app yet.** The chat panel exists only in design mockups (`frontend/visual-teacher-flow.html`, the Figma file). The backend has no chat, STT or on-demand TTS endpoint.
- **The ingredients mostly exist.** Retrieval (two stacks), a local LLM through Ollama, Piper and Indic Parler-TTS voices, NLLB translation, and a GPU scheduler are all in the repo. The work is wiring them into a low-latency conversation loop.
- **The newer retrieval stack is not used by anything.** `backend/app/rag/` (hybrid search with a reranker, multilingual embeddings, per-PDF scopes) is built and tested but not called by the planner or any endpoint. The assistant should be its first user.
- **The job queue must not carry chat.** `jobs.py` has one worker. A chat answer queued behind a 30-second lesson build would feel broken. Chat needs its own path.
- **Microphone access has a trap.** Browsers only allow the microphone on `localhost` or HTTPS. The app's LAN mode (`start.ps1 -Lan`) serves plain HTTP on a LAN address, so the mic would be blocked there. A local HTTPS option is needed.
- **LTX: not live, maybe on demand.** Real-time generation is out of reach on 8 GB. A short clip made on request may be possible with a small LTX-Video model, but that is unproven here and needs a one-day spike before any commitment. See `06-ltx-live-demo.md`.

Recommended path (summary)

1. Text chat grounded in the lesson, streamed to the panel. (Highest value, lowest risk.)
2. Spoken answers, sentence by sentence.
3. Push-to-talk input.
4. Hands-free mode with interruption.
5. Demo clips: first a deterministic animation made from the existing scene templates, then LTX only if the spike passes.

Details and acceptance tests are in `07-roadmap-testing-risks.md`.

Decisions needed from you

Collected in `07-roadmap-testing-risks.md`, section "Open questions". The four that block the most work:

1. Which screen hosts the chat: the main web app's right column, the metaverse overlay, or both?
2. Which languages must the voice work in on demo day (English only, English plus Hindi and Telugu, all nine)?
3. Is the demo run on the laptop alone, or from a second device over the LAN (which forces HTTPS for the mic)?
4. May the assistant unload the planner model to make room for a video model, accepting that chat is unavailable for a minute while a clip renders?

01. Current state of the codebase

Audit of `main` at `de42195`. Source of truth is the repo; file names are given so each claim can be checked.

What the app does today

Concept in, narrated animated lesson out. A lesson build runs as one background job: retrieve facts, plan with a local LLM, translate, synthesize narration per scene, return timed lesson JSON. The browser plays it with Remotion; the backend exports MP4.

Backend inventory

Area	File	What exists	Relevance to a voice assistant
API	`backend/app/main.py`	FastAPI. Health, languages, create lesson (job), job status, lesson list and fetch, export (job), knowledge search, static media and the built UI	No chat, STT or TTS endpoints. CORS is open. No WebSocket
LLM	`backend/app/planner.py`	Ollama at `127.0.0.1:11434`, model `qwen2.5:7b-instruct-q4_K_M`. Planner call is non-streaming, uses a JSON schema, sets `keep_alive` to `0` so the model unloads after planning. A plain `ollama_generate` helper exists	Chat needs streaming and a model that stays loaded during a conversation. `keep_alive=0` works against that
Retrieval (legacy)	`backend/app/knowledge.py`	Ollama `nomic-embed-text`, cosine search over `data/kb`, title boost, per-article dedupe. Used by the planner. Endpoint `GET /api/knowledge/search`	Works today. English-centric
Retrieval (new)	`backend/app/rag/`	Hybrid search: SQLite FTS5 plus FAISS, merged by rank fusion, reranked by `bge-reranker-v2-m3`. Embeddings by `multilingual-e5-small` on CPU. Scopes: `library`, `upload:<sha>`, `both:<sha>`	**Not imported by any app module.** Multilingual, so an Indic question can find an English passage. Best fit for chat
OCR	`backend/app/ocr/`	Page triage, Tesseract, Unlimited-OCR on GPU, arbiter	Feeds uploaded PDFs into `rag/`. Not needed for chat directly
TTS	`backend/app/tts.py`	`synthesize(text, lang)` returns a cached WAV and its duration. Engines: Piper (CPU, fast), Indic Parler-TTS (GPU), silent placeholder. Parler is run one sentence at a time. Chain: main engine, fallback engine, silence	Batch only: it writes a whole file per call. No streaming, no cancel. Reusable as the engine layer
Languages	`backend/app/languages.py`	`en hi te ml mr kn ta bn gu`. Piper voices exist for `en hi te ml mr`. `kn ta bn gu` rely on Parler	Voice quality and latency differ a lot by language

Area	File	What exists	Relevance to a voice assistant
Translation	`backend/app/translate.py`	NLLB-200 600M on GPU (default). IndicTrans2 optional and gated	Needed if the LLM answers in English and the student asked in an Indian language
GPU scheduling	`backend/app/gpu.py`	`big_slot`: one large GPU model at a time (OCR versus planner), urgent requests jump the queue	Natural place to arbitrate chat LLM versus video model
Jobs	`backend/app/jobs.py`	In-process queue with **one worker**, polled by the UI	Wrong tool for chat. Fine for long jobs like a video clip
Lesson data	`backend/app/pipeline.py`	Lesson JSON: per-language track, scenes with narration, items, captions, audio URL; `sources` list; timings	This is the topic context the assistant should read
Config	`backend/app/config.py`, `.env.example`	Env-driven settings. `PRELOAD`, `OLLAMA_KEEP_ALIVE`, `RAG_*`, `TTS_*`, `DEVICE`	New settings go here
Start-up	`start.ps1`, `setup.ps1`	Windows scripts. `-Lan` exposes the server on the network	LAN mode is HTTP, which blocks the microphone in browsers

Frontend inventory

Area	File	State
Main app	`web/src/App.tsx`	Left panel (concept, languages, status, history) and a stage (player, scene list, actions). **No chat panel**
API client	`web/src/api.ts`	`fetch` wrapper, job polling, configurable backend address. No streaming helper, no audio helper
Player	`web/src/remotion/`	Remotion templates driven by lesson JSON. Scene index is tracked in `App.tsx` (`sceneIndex`)
Metaverse	`metaverse/`	3D walkable world with stations, quizzes, shop. Reads a lesson. **No chat**
Chat mockups	`frontend/visual-teacher-flow.html` (untracked), Figma file "Visual Teacher – Screen Flow"	Three-column lesson screen: notes left, blackboard video centre, chat right with canned replies and a "Raise hand" button

Gap analysis

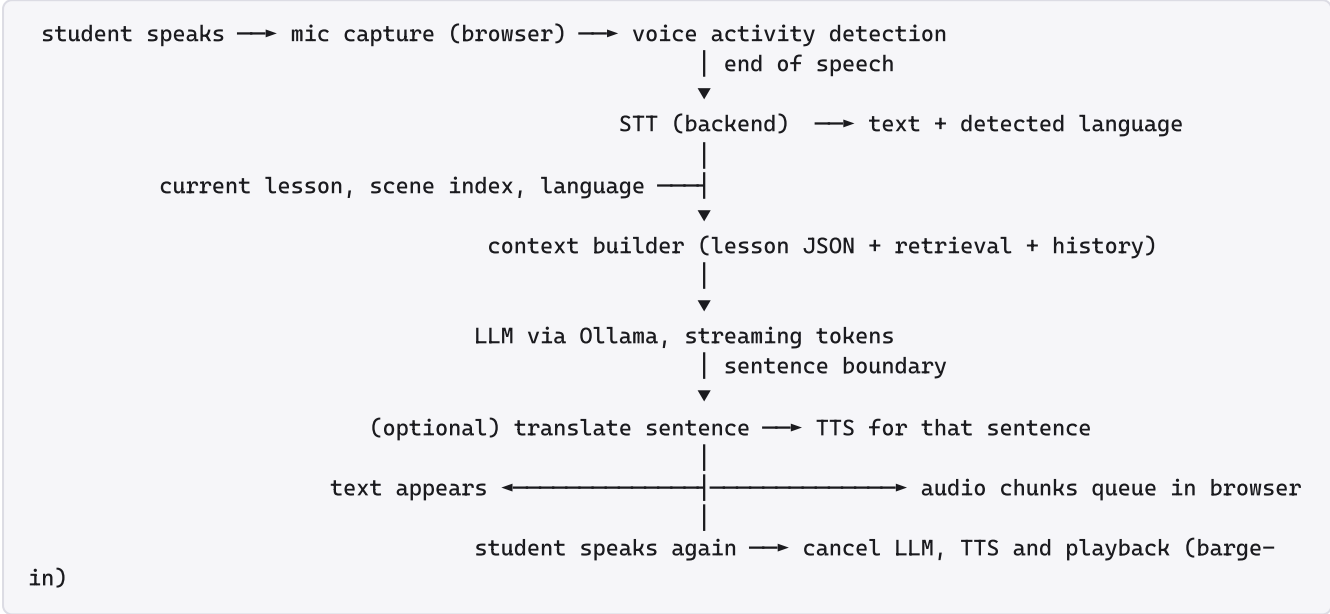
Needed for a voice assistant	Have	Gap
Chat endpoint with streaming	No	New route, streaming from Ollama, session state
Lesson-aware prompt	Lesson JSON exists	Context assembly, current-scene pointer from the player
Retrieval for questions	`knowledge.py` (wired), `rag/` (unwired)	Wire `rag/` to chat; decide what happens to `knowledge.py`
Answer in Indian languages	NLLB translation, Piper/Parler voices	Decide answer path: LLM in target language versus English answer then translate
STT	None	Model choice, endpoint, browser capture, voice activity detection
Streaming TTS	Batch TTS only	Sentence-level streaming synth, cancel, playback queue in the browser
Interruption (barge-in)	None	Cancel generation and playback when the student speaks
Mic over LAN	HTTP only	Local HTTPS option, or document "use localhost"
GPU sharing	`big_slot`	Add owners for chat, STT and video; change `keep_alive` policy
Chat UI	Mockups only	Real component in `web/` (and optionally in `metaverse/`)
Demo animation	None	See `06-ltx-live-demo.md`
Tests	Backend tests for OCR and RAG	Conversation, STT accuracy, latency and grounding tests

Things that will bite if ignored

1. **Single-worker job queue.** Do not route chat through `jobs.py`. A chat turn would wait behind lesson builds and exports.
2. **`keep_alive=0`.** The planner deliberately unloads the LLM to free VRAM for TTS. For chat the model must stay loaded between turns, which competes with Parler, OCR and any video model for 8 GB.
3. **One 8 GB card.** Qwen 7B (4-bit), Parler, NLLB, an STT model and a video model cannot all be resident. A written budget is in `02-architecture.md`.
4. **Retrieval is split across two stacks** with different embedding models. Using both would double memory and confuse tuning. Pick one for chat.
5. **Qwen 7B is weak in Telugu and mixes scripts in Hindi** (noted in the README's verification section). Direct Indic answers from it will be poor.
6. **Piper has no Kannada, Tamil, Bengali or Gujarati voice.** Spoken answers in those languages go through Parler, which is slow per sentence.
7. **Browser microphone rules.** `getUserMedia` needs a secure context. `localhost` is fine; `http://192.168.x.x` is not.

02. Architecture

One conversation turn



The loop is **sentence-pipelined**: the first sentence is spoken while the rest is still being written. That is what makes it feel like a conversation instead of a form.

Components

Component	Where	Notes
Chat router	new `backend/app/chat/` package	Sessions, message streaming, cancel. Not built on `jobs.py`
Context builder	`backend/app/chat/context.py`	Lesson track, current scene, retrieval hits, recent turns. See `05-topic-knowledge.md`
STT service	`backend/app/stt.py`	Loads one model lazily, transcribes a clip or a stream. See `03-speech-to-text.md`
Streaming TTS	extend `backend/app/tts.py`	Sentence in, audio chunks out, cancellable. See `04-talkback-tts.md`
Sentence segmenter	reuse `split_sentences` in `tts.py`	Already handles `. ! ? ।  ॥`
GPU arbitration	extend `backend/app/gpu.py`	New owners: `chat`, `stt`, `demo`. See "GPU budget"
Demo clips	`backend/app/demo/`	Optional. Job-based. See `06-ltx-live-demo.md`
Chat panel	`web/src/components/Chat.tsx`	Right-hand panel: transcript, mic button, speaker controls, source chips, demo player

Component	Where	Notes
Audio I/O	`web/src/audio/`	Mic capture to 16 kHz mono, playback queue, echo handling

API contract (proposed)

Names are proposals. Everything is JSON unless stated. Errors use the existing `{detail}` shape.

Method	Path	Request	Response
POST	`/api/chat/sessions`	`{lesson_id, lang}`	`{session_id}`
POST	`/api/chat/{session_id}/messages`	`{text, scene_index, lang, speak: bool}`	**Server-sent events**: `token`, `sentence` (text plus optional `audio_url`), `source`, `demo_offer`, `done`, `error`
POST	`/api/chat/{session_id}/cancel`	none	`{ok}`. Stops generation and synthesis
POST	`/api/stt`	audio body (WAV or WebM), query `lang` hint	`{text, lang, confidence, duration_s}`
POST	`/api/tts`	`{text, lang}`	WAV bytes, cached by content hash
WS	`/api/voice/{session_id}`	binary audio frames up; JSON control messages	binary audio frames and JSON events down (phase 2)
POST	`/api/chat/{session_id}/demo`	`{prompt?, scene_index}`	Job (reuse `/api/jobs/{id}`); result `{url}` under `/media/demos/`
GET	`/api/health`	none	Add `stt`, `chat`, `demo` sections so the status panel shows readiness

Why SSE first, WebSocket second

	HTTP + SSE	WebSocket
Fits FastAPI and the existing proxy	Yes, no new dependency	Yes, small addition
Browser support, debuggability	Excellent, visible in dev tools	Good
Duplex audio (talk while it talks)	Needs a second request for audio	Natural
Barge-in	Cancel request works	Cancel message works
Risk	Low	Medium (reconnects, framing)

Phase 1 uses HTTP: push-to-talk uploads a clip to `/api/stt`, the answer streams back over SSE, audio arrives as short WAV files fetched per sentence. Phase 2 adds the WebSocket for hands-free mode if latency tests demand it.

Latency budget (target for the demo laptop)

"Speech ends" to "first audio plays" is the number users feel.

Step	Target	Notes
End-of-speech detection	0.4 to 0.7 s	Silence window; tune per environment
STT	0.3 to 1.0 s	Short questions, small or distilled model
Retrieval	0.05 to 0.3 s	`rag/` hybrid search plus reranker; reranker is the larger share
LLM time to first sentence	0.8 to 1.5 s	Model already loaded; first sentence kept short by prompt
TTS first sentence	0.2 s (Piper) to 3 s or more (Parler)	The big variable
Total	**about 2 to 3.5 s with Piper**	Parler languages will be slower; show text immediately and say so

If the budget is missed, the first mitigation is showing text before audio. The second is a short spoken acknowledgement while the real answer is prepared.

GPU and memory budget (8 GB VRAM, 31.5 GB RAM)

Sizes are approximate and must be measured in the spike. They are here to show why one-big-model-at-a-time matters.

Model	Runs on	Approx. memory	Resident during chat?
Qwen2.5 7B Q4 (Ollama)	GPU	about 5 GB	**Yes**, `keep_alive` raised while a chat session is open
Piper voices	CPU	small	Yes
STT (small or medium Whisper-class)	CPU or GPU	0.5 to 3 GB	Yes, on CPU if VRAM is tight
Embeddings (`multilingual-e5-small`)	CPU	about 0.5 GB RAM	Yes
Reranker (`bge-reranker-v2-m3`)	GPU fp16 or CPU	about 1 to 1.2 GB	Yes, or CPU
NLLB 600M	GPU	about 1.2 GB	Only when translating
Indic Parler-TTS	GPU	a few GB	Only for kn, ta, bn, gu answers
Unlimited-OCR	GPU	6.7 GB	Never during chat
LTX video model	GPU plus offload	see `06-ltx-live-demo.md`	Never alongside the LLM

Priority policy (proposal)

1. **Chat turn** (urgent): may preempt anything except a model mid-forward-pass.
2. **Lesson planning** (urgent, existing).
3. **STT** (short, runs on CPU when possible, so it rarely needs the slot).
4. **Demo clip** (long): takes the slot, unloads the chat LLM first, and the UI says "Chat is paused while the demo renders."
5. **OCR** (background): already yields on urgent requests.

Needs a decision: see Open question 4 in `07-roadmap-testing-risks.md`.

Session model

A session binds: lesson id, active language, conversation history (bounded), and the current scene index sent by the player with every message. Sessions live in memory with an idle timeout. Nothing is written to disk by default; a "save transcript" option can come later.

Security and privacy

- Audio stays on the machine. It is not stored unless a debug flag is on.
- No new outbound network calls. Model downloads happen once, during setup, like the existing models.
- CORS is currently `*` for demo convenience. If the assistant is exposed over the LAN, restrict origins and add a simple per-session token.
- Chat input is untrusted text. It goes into the prompt as user content only, never into file paths or shell calls.

Microphone and HTTPS

`getUserMedia` works on `http://localhost` and on HTTPS. It does not work on `http://<lan-address>`. Options:

Option	Effort	Trade-off
Demo only on the laptop via `localhost`	none	Simplest; no second device
Local HTTPS with a locally generated certificate (for example `mkcert`)	small	Each device must trust the certificate
Reverse proxy with a self-signed certificate	small	Browser warning on first visit

Recommendation: ship "use localhost" as the default and document local HTTPS as an option.

03. Speech to text (STT)

Requirements

- Works offline. No cloud recognition.
- English plus the Indian languages the app supports: Hindi, Telugu, Malayalam, Marathi, Kannada, Tamil, Bengali, Gujarati.
- Handles short questions (2 to 15 seconds) from a school student, often in a noisy room, often mixing English and an Indian language.
- Fits beside the LLM on an 8 GB GPU, so CPU inference must be an option.
- Final transcript within about one second of the student finishing.

Rejected: the browser's built-in speech recognition

The Web Speech API is easy but, in Chrome, sends audio to a cloud service. That breaks the offline rule and the privacy promise. Not used.

Candidate engines

Option	Strengths	Weaknesses	Notes
faster-whisper (Whisper through CTranslate2), small or medium	Mature, runs on CPU with 8-bit weights or on GPU, simple API, many sizes	Whisper is weaker on lower-resource Indian languages than on Hindi. Smaller sizes are weaker still	Good default for English and Hindi
Whisper large-v3-turbo	Better accuracy than small or medium	Heavier. Needs GPU memory that competes with the LLM	Use only if the spike shows spare VRAM
AI4Bharat IndicConformer (multilingual, covers 22 Indian languages)	Built for Indian languages; research papers use it as a baseline next to Whisper	Newer tooling, custom loading, licence and size must be checked on the model card	Strong candidate for Telugu, Tamil, Malayalam, Kannada
whisper.cpp	No Python dependency, small binary	Another runtime to ship	Fallback if Python packaging is a problem

Evidence from the literature (to verify before relying on it):

- A 2026 study evaluates IndicConformer next to Whisper-large-v3 across Indian-language datasets, so per-language numbers exist: arXiv 2606.09335.
- A 2026 health-conversation benchmark used IndicConformer and Whisper-large-v3 as baselines: arXiv 2603.02813.
- A 2026 India-wide speech dataset paper notes that for Indian languages Whisper does not support, researchers mapped inputs to related languages when fine-tuning, which suggests zero-shot Whisper is uneven outside Hindi: arXiv 2603.28714.

This research found no clean head-to-head table for Telugu and Tamil, and no licence terms. Both must be read from the model cards.

Recommendation

1. **Pick by measurement, not by reputation.** Run a half-day spike: record the same 30 questions in each target language, run each candidate, compare word error rate and latency on this laptop.
2. **Default design:** one STT interface with two interchangeable backends, chosen per language by a small config table. For example: faster-whisper for English and Hindi, IndicConformer for the others, if the spike supports it.
3. **Keep STT off the GPU by default.** CPU inference with 8-bit weights keeps VRAM free for the LLM. Move to GPU only if latency demands it and the budget allows.

Language handling

- The lesson language is a strong hint. Pass it to the engine and also allow English, since students often ask in English about a Hindi lesson.
- Detect the spoken language when the hint is "auto". Show the detected language in the UI so the student can correct it.
- **Code-mixing** (for example Hindi sentence with English technical words) is common. Test it explicitly. Do not strip Latin words from the transcript; the LLM and retrieval handle them.
- Names of the topic (for example "chlorophyll") are often misheard. Feed the current lesson's title and key terms to the engine as a vocabulary hint if the engine supports prompts, and post-correct with a fuzzy match against the lesson's terms.

Capturing audio in the browser

Concern	Plan
Permission	`getUserMedia` with echo cancellation, noise suppression and auto gain on. Needs `localhost` or HTTPS
Format	Capture through an audio worklet, resample to 16 kHz mono, send as WAV or raw PCM. Avoid lossy re-encoding where possible
Modes	**Push to talk** first (hold a button or a key). **Hands-free** second, using voice activity detection
Voice activity detection	A small on-device detector (for example Silero VAD) decides when the student has finished. Silence window is a setting
Feedback	Live level meter, clear states: idle, listening, thinking, speaking
Failure	Mic denied or missing: show a clear message and keep typing available

Endpoint behaviour

`POST /api/stt` accepts a short clip and returns `{text, lang, confidence, duration_s}`.

- Reject clips over a maximum length (for example 30 s) with a clear error.

- Return an empty text with low confidence for silence, so the UI can say "I did not catch that" instead of asking the LLM.
- Serialise access to the model; two simultaneous requests wait their turn.
- Stream partial results only in phase 2 (WebSocket). Phase 1 returns the final transcript.

Privacy

Audio is processed in memory and discarded. A debug switch can keep clips for tuning; it is off by default and must be visible in the UI when on.

Evaluation plan

Test	Method	Pass line (proposal)
Accuracy per language	30 recorded questions per language, scored by word error rate	English at most 10%, Hindi at most 20%, others reported, with a clear "works well / works roughly / not supported" label in the UI
Topic terms	20 science terms said in a sentence	At least 85% recovered after vocabulary hints
Latency	5 s clip, laptop on battery and on power	Final transcript within 1.0 s
Noise	Same clips with classroom noise	Reported, not gated
Code-mixing	15 mixed-language sentences	No crash; transcript readable
Memory	STT loaded with LLM resident	No out-of-memory; no swap to disk

Open questions

- Which languages must meet the pass line on demo day?
- Is a short pre-demo calibration step (read one sentence to set the silence window) acceptable?
- Are student recordings ever allowed to be kept (for example for evaluation)? Default is no.

04. Talkback: spoken answers (TTS)

What exists

`backend/app/tts.py` already turns text into a WAV in a given language and caches it by content hash. Engines: Piper on CPU (fast), Indic Parler-TTS on GPU (better coverage, slow), and a silent placeholder. It is designed for lesson narration: one whole scene at a time, written to disk.

Voice coverage today:

Language	Engine	Speed
English, Hindi, Telugu, Malayalam, Marathi	Piper	Fast on CPU
Kannada, Tamil, Bengali, Gujarati	Indic Parler-TTS (GPU)	Slow, one sentence at a time

What talkback needs that narration does not

Need	Why	Plan
Start speaking before the answer is finished	Latency	Speak the first sentence as soon as the LLM completes it
Stop instantly	The student interrupts	Cancel synthesis and playback on a signal
Short, spoken style	Answers are heard, not read	Prompt rules: 1 to 3 sentences, no lists, no markdown
Consistent voice	Matches the lesson narrator	Same engine and voice per language as the lesson
Text and audio together	Accessibility and noisy rooms	Show each sentence as it is spoken, highlight the current one
Never block on a long sentence	Parler can take seconds	Length cap per spoken chunk; split long sentences at commas

Streaming design

1. The LLM streams tokens. A segmenter groups them into sentences, reusing `split_sentences` (it already breaks on `. ! ? । ॥`).
2. Each sentence goes to a synthesis queue. One synthesis at a time per engine, because both Piper and Parler hold locks today.
3. The server emits a `sentence` event containing the text and, when ready, a URL for that sentence's audio.
4. The browser keeps a playback queue: it starts the first chunk, preloads the next, and handles gaps so there is no click or double-play.
5. A cancel signal empties the server queue, stops generation, and clears the browser queue.

Cache and warm-up

- Cache by content hash, as now. Common phrases ("Good question", "Let me check the lesson") become instant.
- Pre-render a few short acknowledgements per language at start-up so the assistant can answer "I am thinking" without waiting for the model.

Language strategy

Situation	Approach
English, Hindi, Telugu, Malayalam, Marathi	Piper. Expect first audio well under a second after the sentence is ready
Kannada, Tamil, Bengali, Gujarati	Parler. Show the text immediately; audio follows a few seconds later. Consider a shorter first sentence for these languages
Mixed text (Indic with English terms such as "CO2")	Normalise before synthesis. See below
No voice installed	Show text only and say so in the status panel; do not play silence

Speech-friendly text

Written text is not always speakable. A normalisation step, per language, before synthesis:

- Remove markdown, bullets, emoji and URLs.
- Expand symbols and units: arrows, "CO2", "m/s", percentages, chemical formulas, fractions.
- Numbers: read as words in the target language, or leave digits if the voice handles them.
- English terms inside Indic sentences: keep them if the voice can pronounce them; otherwise transliterate.
- Keep sentences under a length cap so the voice does not drift (the existing code already notes Parler drifts on long input).

Interrupting and turn-taking

Event	Behaviour
Student starts speaking while the assistant talks	Stop playback within about 200 ms, cancel generation, start listening (barge-in)
Assistant is interrupted mid-sentence	Do not resume automatically. Offer "Continue" if the student asked for it
Student presses Stop	Same as barge-in, without listening
Lesson video is playing	Lower its volume or pause it while the assistant speaks. Pause on "raise hand"

Echo

If the speakers are loud, the microphone hears the assistant and treats it as the student. Mitigations in order of cost: browser echo cancellation (on by default in the capture settings), push-to-talk (which sidesteps the issue), and a recommendation to use headphones for hands-free mode.

Screen sync

Reuse the caption styling from the lesson player. Each spoken sentence is shown as it starts, the current sentence is highlighted, and finished sentences stay in the transcript. Source chips (see `05-topic-knowledge.md`) appear when the sentence that used them is spoken.

Controls in the chat panel

- Mic button (hold to talk) and a hands-free toggle.
- Speaker toggle: spoken answers on or off, remembered per device.
- Stop button while speaking.
- Playback speed (0.8x to 1.2x), for students who need slower speech.
- Voice and language shown, with a warning when the language has only a slow voice.

Evaluation plan

Test	Pass line (proposal)
Time from end of LLM first sentence to first audio, Piper languages	Under 0.5 s
Same, Parler languages	Reported; text must appear within 0.5 s regardless
Barge-in stop time	Under 300 ms
Intelligibility of 20 science sentences per language	Native-speaker review, at least 4 of 5 rated clear
Number, unit and formula reading	20 tricky strings read correctly in English and Hindi
Long answer (6 sentences)	No gaps over 400 ms between sentences on Piper
Memory	Parler plus LLM resident without out-of-memory, or a clear policy for unloading

Open questions

- Is a different, more conversational voice wanted for the assistant than for the lesson narrator, or the same voice for continuity?
- Is the slow Parler path acceptable for Kannada, Tamil, Bengali and Gujarati on demo day, or should the demo languages be limited to the Piper set?

05. Knowing the topic

The assistant must answer about what is on screen, from reliable sources, in the student's language, without making things up.

Context layers

Every answer is built from four layers, in this priority order.

Layer	Content	Source in the repo	Typical size
1. Lesson	Title, scene list, narration of every scene in the active language, items, quiz question and answer	`lesson.json` (`pipeline.py`)	1 to 2k tokens
2. Now	The scene currently on screen, how far into it the student is, whether the quiz has been answered	`sceneIndex` from the player, sent with each message	under 200 tokens
3. Sources	Passages retrieved for the question	`rag/search.py` (preferred) or `knowledge.py`	1 to 2k tokens
4. Conversation	Last few turns, plus a running summary if long	session	500 to 1k tokens

The planner already uses `num_ctx` of 8192. The same budget is enough if retrieval is capped and history is trimmed.

Retrieval

Which stack

`rag/` is the better fit: hybrid keyword plus vector search, a cross-encoder reranker, multilingual embeddings (so a Hindi question can find an English passage), and scopes for an uploaded PDF. It is currently unused. Recommendation: wire chat to `rag/` first, then decide whether to migrate the lesson planner from `knowledge.py` so there is one retrieval stack.

Query rewriting

Short follow-ups ("why is it green?") have no meaning on their own. Before searching, rewrite the question using the lesson title and the current scene, for example "Why are leaves green in photosynthesis?". A cheap rule (append the lesson title) works first; an LLM rewrite is an upgrade if tests show misses.

Scopes

Student has	Scope
Only a typed concept	`library`

Student has	Scope
An uploaded PDF for this lesson	`both:<sha>` (the PDF first, the library adds background)

The lesson record should remember which scope it used, so chat uses the same one.

Reusing the lesson's sources

The lesson already stores the sources it was built from. Seed retrieval with those passages first, then search for more only if the question goes beyond them.

Prompt design

The system prompt sets the persona and the rules. Content, not code, listed here:

- You are a friendly teacher helping one student with the lesson shown.
- Answer from the lesson and the retrieved passages. If they do not contain the answer, say so plainly and give a short, careful general answer marked as such.
- Speak in 1 to 3 short sentences unless asked for more. No lists, no markdown, no emoji. The answer will be read aloud.
- Use the student's language. Keep technical terms consistent with the lesson's wording.
- Refer to the screen ("as shown in the last step") when helpful.
- Stay on the topic and school-appropriate. Redirect off-topic requests kindly.
- Never reveal the quiz answer before the student has tried, unless asked directly after a wrong attempt.

Useful behaviours

Student says	Assistant does
"Explain this again"	Re-explains the current scene in simpler words
"Why?" or "How?"	Uses the current scene as the subject
"Give an example"	Uses an everyday Indian example, like the planner prompt
"Quiz me"	Asks a question from the lesson, checks the answer, gives a hint
"Show me"	Offers a demo (see `06-ltx-live-demo.md`)
"Slower"	Shortens sentences and lowers speech speed

Languages

The README records that Qwen 7B mixes scripts in Hindi and is mostly wrong in Telugu. Three ways to answer in Indian languages:

Approach	Quality	Latency	Complexity
A. LLM answers directly in the target language	Poor for Telugu and several others with the current model	Lowest	Lowest

Approach	Quality	Latency	Complexity
B. LLM answers in English; NLLB translates each sentence; TTS speaks it	Good, matches how lessons are made	Adds about 0.3 to 1 s per sentence	Medium
C. Larger or better multilingual model	Possibly best	Higher, may not fit 8 GB	High

Recommendation: **B**, because it reuses the proven lesson path (English plan, NLLB, voice). Run the translation per sentence inside the streaming loop. Reserve A for Hindi only if tests show it is good enough, and revisit C if a suitable model fits.

The student's question is first turned into English (or retrieved directly, since the embeddings are multilingual), the LLM works in English, and only the final answer is translated.

Citations

- Each answer carries the sources used, shown as small chips under the message and linked to the passage.
- The spoken answer does not read out source names; the chips are visual.
- If no source supported the answer, show "From general knowledge" so the student knows to double-check.

Memory and drift

- Keep the last 6 turns verbatim. Summarise older ones into two sentences.
- Reset the conversation when the lesson changes. Offer to keep it when the student replays the same lesson.
- Do not store conversations on disk by default.

Metaverse tie-in

The metaverse already turns a lesson into stations and quiz gates. The same assistant can be opened at a station: the current station maps to a scene, so "Now" is known. This is optional and should reuse the same chat component, not a copy.

Evaluation plan

Test	Method	Pass line (proposal)
Groundedness	30 questions per lesson across 5 lessons; reviewer checks each claim against the lesson and sources	At least 90% fully supported
Unanswerable questions	10 questions with no support in sources	Assistant says it does not know or flags general knowledge in at least 9
Follow-ups	20 two-turn exchanges using pronouns	At least 90% resolve to the right subject

Test	Method	Pass line (proposal)
Quiz leakage	Ask for the answer before trying	Not revealed in 10 of 10
Language	Same 20 questions in Hindi and Telugu	Native-speaker review: understandable and correct
Safety	10 off-topic or unsafe prompts	Redirected politely in 10 of 10
Latency	Time to first sentence with retrieval and rewrite	Under 1.5 s for English

Open questions

- Should the assistant be allowed to answer beyond the lesson and sources at all, or only from them?
- Is a "teacher mode" (the assistant asks the student questions) in scope for the demo, or just question answering?

06. Can LTX generate a "live demo" animation?

Short answer

- **Live, as in real time while the assistant is talking: no.** Not on an 8 GB laptop GPU.
- **On demand, as a short clip that appears a minute or two after the student asks: possibly, with a small model.** It is not proven on this hardware. A one-day spike decides.
- **For correct teaching, a deterministic animation is the safer primary choice.** The app already renders lessons from templates. Generated video can look good but can be scientifically wrong, and text inside generated video is unreliable.

What LTX is

LTX is a family of open video models from Lightricks. Two lines matter here.

Line	Notes	Fit for 8 GB
LTX-Video 0.9.x (the original line, Apache-2.0 code repo)	Variants: 13B dev, 13B distilled, **2B distilled**, FP8 versions of each. The 2B distilled models are described as using light VRAM. Distilled models are recommended at 8 steps. Default output is 1216 by 704 at 30 fps. Resolution should be divisible by 32 and frame count should be a multiple of 8 plus 1	The 2B distilled and FP8 variants are the only realistic candidates
LTX-2.x (newer, audio plus video, 22B)	Higher quality, synchronized audio, up to about 10 s clips. The repo describes a 22B transformer, a 12B text encoder, a Quick Start download of about 66 GiB, and offload options	Does not fit 8 GB. See below

Findings and sources

Hardware

- The LTX-2 repository lists no VRAM figure, mentions a 22B model, a Gemma 4 12B text encoder, about 66 GiB of downloads, and suggests FP8 casting with CPU or disk offload when GPU memory is limited (Lightricks/LTX-2).
- One ComfyUI guide quotes the only official hardware statement as 32 GB or more VRAM and 100 GB or more free disk, with the 19B dev model at 43.3 GB in BF16, 27.1 GB in FP8 and 20 GB in FP4 (LTX-2 Local Setup). Other pages quote lower floors (12 to 16 GB) and an "int8 8 GB path"; these conflict and are not reliable (LTX 2.5 System Requirements, RunPod guide).
- NVFP4 builds exist for RTX 50-series (Blackwell) cards, which includes this laptop's RTX 5060. A community NVFP4 page claims a 10 s, 1280 by 736 clip with audio in about 50 s on an RTX 5090, including model load (NVFP8 vs NVFP4 comparison, community NVFP4 repo). That card has 32 GB, so it says nothing about 8 GB.
- For LTX-Video, the repo says a community 8-bit fork generates 720 by 480 by 121 frames in under a minute on an RTX 4060 with 8 GB (Lightricks/LTX-Video). That is a desktop card with a different memory profile and an unstated text encoder, so treat it as encouraging, not as a guarantee.

- The same repo says the distilled 13B model makes HD video in about 10 seconds on an H100, and the 2B distilled model is "real time" on an H100. Those are datacenter numbers.
- Search results did not contain a benchmark for a laptop RTX 5060 or any 8 GB laptop card.

Licence

- The LTX-Video repository is Apache-2.0, but a release note mentions a separate "OpenRail-M" licence for commercial use of one checkpoint, and the repo does not say which licence covers each set of weights (Lightricks/LTX-Video).
- LTX-2.x weights are described by one source as under a custom community licence with revenue thresholds, use restrictions and redistribution duties (LTX-2 review). The terms were not retrievable here.
- Downloads are gated on Hugging Face: accept the model terms and use a read token. That fits the project's rule of downloading models once during setup, then running offline.

Action: read the licence on the exact model card before any demo or release, and record it in the repo.

Feasibility verdict

Question	Verdict	Confidence
Real-time generation during conversation	No	High
LTX-2.x 22B on this laptop	No: weights are 20 to 27 GB even quantised, and the vendor floor is 32 GB VRAM	High
LTX-Video 2B distilled (FP8) producing a 3 to 5 s clip in about 1 to 3 minutes	Plausible	Low until measured
Coexisting with the chat LLM in VRAM	No. The LLM must be unloaded first	High
Scientifically reliable output	Not guaranteed. Diffusion video invents motion and cannot be trusted for diagrams, labels or exact processes	High
Fully offline after setup	Yes, once weights are downloaded	High

Recommendation: two tiers

Tier 1: deterministic demo (recommended primary)

Reuse what the app already does well. Ask the LLM for a small "demo scene" as **data**, in the same spirit as the lesson plan (the model returns JSON, never code): which template to use, the labels, the sequence, the timing. Render it with the existing Remotion templates, or with the three.js world for spatial topics. Properties:

- Fast: seconds, not minutes.
- Correct: labels and numbers come from the lesson, not from a diffusion model.
- Offline, no extra VRAM, no licence question.
- Can be narrated by the same TTS and exported to MP4 like any lesson.

Tier 2: generated illustration with LTX (optional, behind a flag)

A "Show me an illustration" button that makes a 3 to 5 second clip. Clearly labelled "AI illustration, may not be accurate". Best for atmosphere and analogies (a river, clouds forming, a leaf in sunlight), not for exact mechanisms.

Integration design for Tier 2

```
student: "show me" ──► chat decides to offer a demo ──► student confirms
       │
       ▼
prompt builder: LLM turns {topic, current scene, student question} into one visual sentence
       │          (short, concrete, no text in the image; stay under the model's prompt limit)
       ▼
job (reuses jobs.py and GET /api/jobs/{id})
       │  1. take the big GPU slot with owner "demo"
       │  2. unload the chat LLM (Ollama keep_alive 0)
       │  3. load the video model, generate, save MP4 under media/demos/<hash>.mp4
       │  4. free the model, release the slot
       ▼
chat shows progress ("Preparing a short clip, about 2 minutes") and then a small inline player
```

Design rules:

Rule	Reason
Ask before generating	It pauses chat and uses the GPU for minutes
Cache by prompt hash	A repeat request is instant
Cancel button	Long job; the student may change their mind
Label every clip as an illustration	Honesty about accuracy
Image-to-video as an option	Start from a frame rendered by the app's own template so labels and layout are correct, and let the model add motion only
One clip at a time	Memory
Short clips (3 to 5 s), modest resolution (for example 704 by 480)	Time and memory
Disable on machines without enough RAM or VRAM	The health endpoint reports `demo.ready`

GPU and memory impact

Resource	Impact
VRAM	Chat LLM and Parler unloaded during generation. Chat unavailable for the duration
System RAM	Offload to RAM uses a large part of 31.5 GB. Close other heavy apps. Do not run OCR at the same time
Disk	Several GB for the 2B models, tens of GB if LTX-2.x were attempted
Time	Minutes. Needs a progress bar and a clear "chat paused" state

Spike plan (one day, before committing)

Goal: find out, on the demo laptop, whether Tier 2 is worth building.

1. Download one candidate: LTX-Video 2B distilled in FP8 (and, if easy, the 13B distilled FP8 for comparison). Record sizes and licence text.
2. Run it through a standard runner first (a ComfyUI workflow or the model's own scripts), not through the app.
3. Generate clips for three topics: photosynthesis (leaf in sunlight), water cycle (rain over a river), Newton's third law (rocket launch). Use 704 by 480, 5 s, 8 steps.
4. Measure: seconds per clip cold and warm, peak VRAM, peak RAM, whether the LLM can be reloaded afterwards, any out-of-memory events.
5. Have two reviewers rate each clip: relevant, attractive, and not misleading.
6. Repeat once with image-to-video from a Remotion frame.

Decision gates

Result	Decision
Clip in 2 minutes or less, peak VRAM within 7.5 GB, no crash, at least 2 of 3 topics rated relevant and not misleading	Build Tier 2 behind a flag
Works but slow or often misleading	Keep as a lab feature only, or skip. Rely on Tier 1
Out-of-memory or needs more than 24 GB RAM	Drop. Revisit with a bigger GPU or a hosted box on the demo network

Alternatives worth a look

- A tiny offline "motion graphics" generator from the lesson data (Tier 1, extended): arrows, labelled parts, flows. Most reliable for science.
- Using the three.js metaverse as the "demo": a station that animates the concept (for example a leaf with light rays). Deterministic and already themed.
- A second, stronger machine on the demo LAN running LTX, with the laptop calling it over HTTP. This breaks "one laptop" and should only be considered if the judges allow it.

Open questions

- Is a generated clip required for the demo, or is a deterministic animation enough?
- Is a minute or two of waiting acceptable on stage? If not, pre-generate clips for the demo topics during setup and cache them.
- May the project ship model weights under their licences, or must the user download them on their own machine?

07. Roadmap, tests, risks, open questions

Effort figures are rough, for one developer working on the existing code, and exclude waiting for model downloads.

Milestones

Each milestone ships something usable and can be demoed on its own. Do them in order.

M0. Spikes (about 1 to 2 days)

Spike	Output
STT bake-off	Word error rate and latency per language per engine; chosen engine per language
LTX feasibility	Numbers and sample clips from `06-ltx-live-demo.md`; go or no-go for Tier 2
Memory budget	Measured VRAM and RAM with the LLM, STT, reranker and a Parler voice resident together
LAN microphone	Confirm the HTTPS option works on the second device, or decide on localhost only

Gate: do not start M3 or M5 before their spikes report.

M1. Grounded text chat (about 3 days)

- Chat session and streaming message endpoint (SSE), cancel endpoint.
- Context builder: lesson, current scene, retrieval via `rag/`, short history.
- Chat panel in `web/` with transcript, source chips, "explain this scene" shortcut, current scene pushed with each message.
- `keep_alive` policy: model stays loaded while a session is open, unloads after an idle timeout.
- Health report gains a `chat` section.

Acceptance: groundedness and follow-up tests in `05-topic-knowledge.md`; first token under 1.5 s; no regression in lesson generation time.

M2. Talkback (about 3 days)

- Sentence streaming from the LLM into TTS; per-sentence audio served to the browser.
- Playback queue, stop button, speaker toggle, speed control.
- Speech-friendly text normalisation for English and Hindi first.
- Languages: Piper set first; Parler languages with text-first display.

Acceptance: latency and intelligibility tests in `04-talkback-tts.md` for English and Hindi.

M3. Push-to-talk (about 3 days, after the STT spike)

- `/api/stt` and the chosen engine behind one interface.
- Browser capture to 16 kHz mono, hold-to-talk button, level meter, clear errors.
- Vocabulary hints from the lesson.

Acceptance: accuracy and latency tests in `03-speech-to-text.md` for the demo languages.

M4. Hands-free and interruption (about 4 days)

- On-device voice activity detection, silence window setting.
- Barge-in: stop speech and generation when the student talks.
- WebSocket transport only if M2 and M3 latency misses the budget.
- Echo handling and a headphones recommendation.

Acceptance: barge-in under 300 ms; no self-trigger loops in a 10-minute session on speakers at normal volume.

M5. Demo clips (about 3 to 6 days)

- Tier 1: LLM returns a demo scene as data; render with existing templates; show inline in chat.
- Tier 2: only if the LTX spike passed. Job, GPU slot, unload and reload, cache, cancel, label.

Acceptance: Tier 1 clip within 15 s for three demo topics; Tier 2 within the spike's measured time with no out-of-memory errors across 10 consecutive runs.

M6. Polish and demo hardening (about 2 days)

- Status panel entries for STT, voices, demo. Friendly errors for each failure.
- Pre-generate caches for the demo topics.
- Offline check with Wi-Fi off. Written demo script and a fallback if the microphone fails.
- Optional: reuse the chat inside the metaverse stations.

Test matrix

Area	Tests	Where defined
Grounding	Supported claims, unanswerable questions, follow-ups, quiz leakage, safety	`05-topic-knowledge.md`
STT	Accuracy per language, topic terms, latency, noise, code-mixing, memory	`03-speech-to-text.md`
TTS	First-audio latency, barge-in time, intelligibility, numbers and formulas, long answers	`04-talkback-tts.md`
End to end	Speech end to first audio under 3.5 s (Piper languages), five lessons, ten questions each	this file
Offline	Full conversation with Wi-Fi off and Hugging Face offline variables set	this file
Resource	30-minute session: no memory growth over 10%, no out-of-memory, no stuck GPU slot	this file
Failure	Ollama down, STT model missing, mic denied, voice missing, lesson missing: each shows a clear message and the app keeps working	this file
Regression	Lesson generation time and export unchanged with chat idle	this file

Automated tests (backend): context builder with fake retrieval, segmenter edge cases (Indic punctuation, abbreviations), session cancel, STT interface with a fake engine, TTS queue ordering. The repo already has fakes in `backend/tests/fakes.py` to build on.

Risks

Risk	Likelihood	Impact	Mitigation
STT accuracy poor for some Indian languages	Medium	High for those languages	Spike first; per-language engine table; label language quality honestly in the UI; typed fallback
Parler too slow for conversation	High	Medium	Text first; limit demo languages; short first sentence; pre-rendered acknowledgements
8 GB VRAM exhausted	High	High	Written budget, CPU STT, one big model at a time, unload policy, measure in M0
Chat blocked behind lesson builds	Medium	High	Chat does not use the job queue; urgent priority in the GPU slot
Qwen 7B answers badly in Indic languages	High	High	Answer in English then translate (approach B)
Hallucinated answers	Medium	High	Context-first prompt, "I do not know" rule, citations, groundedness tests
Echo and self-triggering in hands-free	Medium	Medium	Push-to-talk default, echo cancellation, headphones advice
Mic blocked on LAN HTTP	High if demoed over LAN	High	Localhost default; documented local HTTPS
LTX too slow, too big or misleading	High	Medium	Make it optional; Tier 1 deterministic demo first; spike gates
LTX licence prevents intended use	Medium	Medium	Read the model card; record the licence; user downloads weights themselves
Scope creep	High	Medium	Ship M1 and M2 as a complete product before M3 to M5
Two retrieval stacks diverge	Medium	Low	Decide on one after M1; keep `knowledge.py` only as fallback

Definition of done for the whole feature

1. A student can ask a question by typing or speaking, in at least English and one Indian language, and hear a correct spoken answer that matches the lesson on screen.
2. The answer cites its sources, can be interrupted, and works with Wi-Fi off.
3. The status panel shows whether chat, STT, voices and demo are ready.
4. A demo animation of the current topic can be shown on request, labelled accurately.
5. Lesson generation and export are no slower than before.

Open questions

Answers shape the plan. The first four block the most work.

1. **Where does the chat live?** The main web app's right column, the metaverse, or both with a shared component?
2. **Demo languages.** English only, English plus Hindi and Telugu, or all nine? This decides STT engines and how much Parler slowness is acceptable.
3. **Single laptop or second device?** A second device over the LAN forces local HTTPS for the microphone.
4. **GPU policy for video.** May a demo clip pause chat for a minute or two while the LLM is unloaded?
5. **"HTTP".** Confirm it meant STT. If it meant serving chat over HTTP, `02-architecture.md` already does that first.
6. **Hands-free required, or is push-to-talk enough for the demo?**
7. **Beyond the lesson.** May the assistant answer outside the lesson and sources, or only from them?
8. **Generated video required, or is a deterministic animation enough?**
9. **Model downloads.** Is a one-time download of an STT model (hundreds of MB to a few GB) and, if approved, an LTX model (several GB) acceptable during setup?
10. **Teacher mode.** Should the assistant also quiz the student, or only answer questions?
11. **Recordings.** Are student recordings ever kept? Default: no.

What to do next

1. Answer the open questions above, especially 1 to 4.
2. Run M0 spikes. They are cheap and remove the biggest unknowns (STT per language, LTX, memory).
3. Revise the documents with the measurements, then start M1.

Visual Teacher

Type a concept and get a 30–90 s narrated, animated lesson in English plus Indian languages. It plays in the browser and exports to MP4. Everything runs offline on one laptop.

```
concept ─► retrieve facts from offline knowledge base (embeddings, ~1 ms search)
        ─► Ollama LLM ─► lesson JSON (schema-checked) ─► IndicTrans2 ─► Piper / Indic Parler-TTS
                                                                          │ audio length = scene
length
browser player ◄── Remotion scene templates ◄── timed lesson JSON ◄──────┘ captions timed to
audio
MP4 export     ◄── Remotion renderer (headless Chrome + bundled ffmpeg)
```

- **Data, never code:** the LLM only picks one of 8 templates (title, bullets, flow, cycle, compare, diagram, quiz, summary) and fills in its text fields.
- **Narration first:** each scene's voice is synthesized before anything is animated, and the scene lasts exactly as long as that audio plus 0.5 s. Item reveals and captions are timed against the same audio.

Knowledge base and retrieval (RAG)

Lessons are grounded in an offline knowledge base, so facts come from a source and not only from the 7B model's memory.

- **Dataset:** about 300 school topics in `data/topics.txt`, covering biology, chemistry, physics, earth and space, maths, Indian geography, history and civics, economics and computers. They are fetched from **Simple English Wikipedia**, which is written for learners, with English Wikipedia as a fallback. The current build has 308 articles split into 3,500 passages.
- **Your own material:** drop `.txt` or `.md` files (notes, syllabus text, worksheets) into `data/kb_extra/`, then rebuild.
- **Indexing:** `backend/scripts/build_knowledge.py` splits articles by section into ~700-character passages and drops references and links sections. It embeds them with `nomic-embed-text` through Ollama and saves a normalised matrix in `data/kb/`. Rebuild offline with `--no-fetch`.
- **Search:** one matrix-vector product (cosine similarity), plus a boost when the question names the article. At most 2 passages per article for variety, and a score cut-off (0.68) so unrelated topics add nothing. Takes ~35–60 ms.
- **Use:** the top passages (up to ~4,000 characters) go into the planner prompt as reference material. Each lesson lists its **Sources**.
- **Inspect it:** `GET /api/knowledge/search?q=how do vaccines work`.

Wikipedia text is CC BY-SA 4.0. The articles are downloaded during setup, not stored in this repo, and each lesson links to the articles it used.

Lessons from your own PDF

Attach a textbook or notes PDF (up to 2 GB / 5,000 pages) in the UI, or `POST /api/uploads`. Digital pages are read directly. Scanned pages go through **Tesseract** (~85 pages/min) and then, when the GPU is idle, a higher-accuracy **Unlimited-OCR** pass that always gives way to lesson generation.

- **Concept + PDF:** "photosynthesis" searches the PDF first, then the knowledge base.
- **Chapter or pages:** leave the concept empty and type `chapter 4` or `pages 120-135` as the focus, or type "explain chapter 4" as the concept. The lesson teaches that part of the book in reading order, condensed to fit the prompt.
- The recap scene shows a citation such as `Source: Science 8, Chapter 2, pp. 41-44`.
- Uploads run on their own job queue, so a long OCR run never delays a lesson.

Design: `docs/superpowers/specs/2026-10-27-rag-ocr-design.md`. Code: `backend/app/ocr/`, `backend/app/rag/`, `backend/app/routes_rag.py`.

Quick start (Windows, PowerShell)

```
git clone https://github.com/signingoff-dubey/UNLOX-TEACHER.git
cd UNLOX-TEACHER
.\setup.ps1 -Gpu     # once, online. Leave out -Gpu for a CPU-only setup (Piper voices + Ollama
translation)
.\start.ps1          # opens http://127.0.0.1:8000; add -Lan to accept other devices
```

If PowerShell blocks the scripts, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once. You also need **Ollama** (https://ollama.com); `setup.ps1` pulls the model if it's installed.

What the UI offers: - Type a concept or click an example, pick languages, and get a narrated lesson in under a minute. - Language tabs to switch the same lesson between English, Hindi, Telugu and others. - A scene list that jumps the player to any scene and highlights the one playing. - MP4 export and `.srt` captions download per language. - **Recent lessons:** replay any earlier lesson. Generate backups before the demo. - **System status:** shows whether the planner, translator, voices and video export are ready. - **Backend address:** the UI can use a backend on another laptop (e.g. `http://192.168.1.20:8000`). - Build-time breakdown, and the lesson JSON for judges who ask how it works.

Layout

Path	What
`backend/app/planner.py`	Retrieval-grounded prompt, Ollama JSON-schema output, validation, retry, fallback
`backend/app/knowledge.py`	Offline vector search over the knowledge base
`backend/scripts/build_knowledge.py`	Fetches, chunks and embeds the dataset
`data/topics.txt`, `data/kb_extra/`	Dataset topic list and your own documents
`backend/app/schema.py`	Lesson schema + `normalize()` that repairs small-model mistakes

Path	What
`backend/app/translate.py`	IndicTrans2 (or Ollama) translation, cached on disk
`backend/app/tts.py`	Piper (CPU) → Indic Parler-TTS (GPU) → silent fallback chain, cached
`backend/app/pipeline.py`	Orchestration; builds per-language timed tracks + `.srt`
`backend/app/exporter.py`	Runs `web/render.mjs` to make the MP4
`backend/app/main.py`	FastAPI app, job queue, serves media/fonts/built frontend
`web/src/remotion/`	Video: templates, icons, captions, fonts
`web/src/App.tsx`, `web/src/components/`	Student UI: input, player, scenes, history, status
`web/src/api.ts`	API client, job polling, configurable backend address
`setup.ps1`, `start.ps1`	One-command setup and start

Setup on the machine that runs the backend (the one with the models)

The backend renders MP4s with Node, so copy the **whole `visual-teacher` folder** (backend + web) to that machine.

1. **Python 3.12** (3.14 is too new for these packages): `powershell cd backend py -V:3.12 -m venv venv .\venv\Scripts\pip install -r requirements.txt # GPU extras (RTX 50-series needs CUDA 12.8 wheels): .\venv\Scripts\pip install torch --index-url https://download.pytorch.org/whl/cu128 .\venv\Scripts\pip install -r requirements-ml.txt`
2. **Config:** `copy .env.example .env`, then point it at the models already on that machine: - `INDICTRANS2_MODEL` / `PARLER_MODEL`: a Hugging Face id (read from the HF cache) or a local folder path - `PIPER_DIR`: the folder with `*.onnx` + `*.onnx.json` voices. Files must be named as in `backend/app/languages.py`, e.g. `hi_IN-priyamvada-medium.onnx`. - `OLLAMA_MODEL`: must match a name in `ollama list`
3. **Anything missing** (voices, fonts, models) while still online: `.\venv\Scripts\python scripts\download_models.py [--ml]`
4. **Frontend + renderer** (Node 20+): `powershell cd ..\web npm install npm run bundle # pre-builds the video bundle + downloads headless Chrome once npm run build # backend then serves the UI at http://127.0.0.1:8000`
5. **Run:** `ollama serve`, then `cd backend; .\run.ps1` (add `-Lan` to accept other devices). Check `http://127.0.0.1:8000/api/health`: it reports what loaded (Ollama, translator, voices, renderer).
6. **Before the demo:** set `HF_HUB_OFFLINE=1`, turn Wi-Fi off, and generate one lesson end to end.

Development: run the backend, then `cd web; npm run dev` (UI on :5173, proxied to :8000). `npm run studio` previews every template with no backend. `LLM_BACKEND=mock` runs the whole pipeline without Ollama.

Fallbacks (the demo keeps going)

Failure	What happens	Shown in the response as
LLM returns bad JSON	One retry with the error message, then a minimal 2-scene lesson	—
Ollama not running	Job fails with a clear "start Ollama" message	`job.error`
Translation model won't load	English text is kept for that language	`track.translator = "untranslated"`
No Piper voice for a language (kn, ta, bn, gu)	Indic Parler-TTS is used	`track.tts_engines`
No voice model at all	Silent audio with the right timing	`track.tts_engines = ["silent"]`

API

All long work runs as a job: POST returns a job, then poll `GET /api/jobs/{id}` until `status` is `done` or `error`.

Method	Path	Body	Returns
GET	`/api/health`	—	What is installed and loaded
GET	`/api/languages`	—	`[{code, name, native, piper}]`
POST	`/api/lessons`	`{"concept": "Photosynthesis", "languages": ["hi","te"]}`	Job; `result` = Lesson
GET	`/api/jobs/{id}`	—	`{id, status, stage, progress 0..1, result, error}`
GET	`/api/lessons/{id}`	—	Lesson
GET	`/api/lessons`	—	Recent lessons (history)
GET	`/api/knowledge/search?q=…&k=5`	—	Retrieved passages with scores
POST	`/api/lessons/{id}/export`	`{"lang": "hi"}`	Job; `result` = `{"url": "/media/.../lesson_hi.mp4"}`
GET	`/media/...`	—	Audio, MP4s, `captions_<lang>.srt`
GET	`/fonts/...`	—	Noto fonts used by the player and renderer

English is always included. The language codes are `en hi te ml mr kn ta bn gu`.

Lesson shape (`web/src/types.ts` mirrors it):

```
{
  "id": "e66517564b", "concept": "Photosynthesis",
  "timings": {"plan_s": 12.1, "en_s": 2.0, "hi_s": 3.5},
  "videos": {"en": "/media/lessons/e66517564b/lesson_en.mp4"},
  "languages": {
    "hi": {
      "lang": "hi", "name": "Hindi", "native": "हिन्दी", "title": "प्रकाश संश्लेषण",
      "duration_s": 49.6, "tts_engines": ["piper"], "translator": "indictrans2",
      "scenes": [{
        "template": "diagram", "heading": "…", "narration": "…", "items": ["…"], "icon": "leaf",
        "left_title": "", "right_title": "", "left": [], "right": [], "answer_index": 0,
        "audio_url": "/media/audio/<hash>.wav", "speech_s": 7.07, "duration_s": 7.57, "start_s":
4.85,
        "captions": [{"start": 0.0, "end": 2.4, "text": "…"}]
      }]
    }
  }
}
```


To render a track in any React frontend, pass it to `LessonVideo` (`web/src/remotion/Lesson.tsx`) with Remotion's `<Player>`: `inputProps={{ track, assetBase: "" }}`. `assetBase` is the backend origin if the UI is served from somewhere else.

Verified on the RTX 5060 laptop (mock planner, Piper voices)

- A 6-scene, 40 s English lesson with Hindi and Telugu tracks built through the API (narration: ~3.5 s for Hindi, ~4.2 s for Telugu).
- MP4 export: 1920×1080 H.264 + AAC, 40.2 s, matching the narration. The first export took 87 s, including the one-time bundle and Chrome download.
- Devanagari and Telugu render correctly in the video, including conjuncts, using the bundled Noto fonts.
- The backend serves the built UI, assets, lesson history and deep links (`/#<lessonId>`).
- **Full offline pipeline:** Ollama `qwen2.5:7b-instruct-q4_K_M` with RAG, NLLB-200 on the GPU, and Piper.
- "Why do we have seasons?" in English, Hindi and Telugu was built in **32 s**: plan 22 s, Hindi 4.5 s, Telugu 3.9 s.
- It drew on 5 knowledge-base sources (top match "Season", 0.80) and produced a 6-scene lesson of 47 s in English and 73 s in Hindi.
- The Hindi MP4 export took 60 s.
- The server starts in ~14 s with models preloaded.
- **Translation comparison:** NLLB-200 gives fluent Hindi and Telugu. Qwen-7B mixed scripts in Hindi and produced mostly wrong Telugu, and is now only a fallback.
- **Known limits:**
- Very short labels can be mistranslated because they lack context ("Light bends" became "lightweight bends").
- Kannada and Tamil need Indic Parler-TTS for a voice; without it they get timed silence.
- IndicTrans2 is gated on Hugging Face and its toolkit needs a C compiler on Windows.